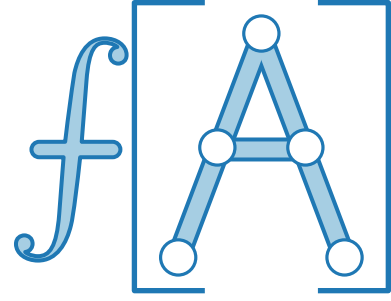
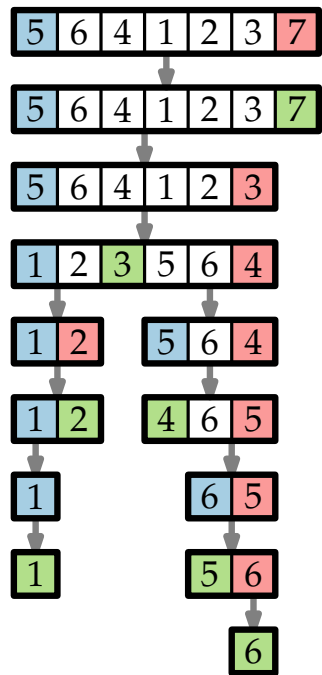




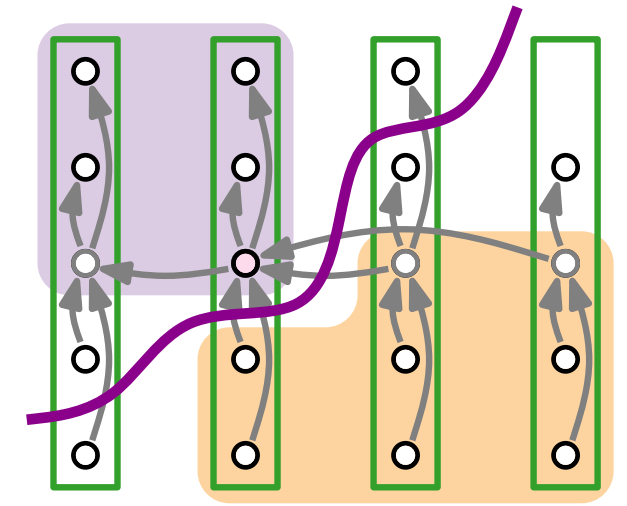
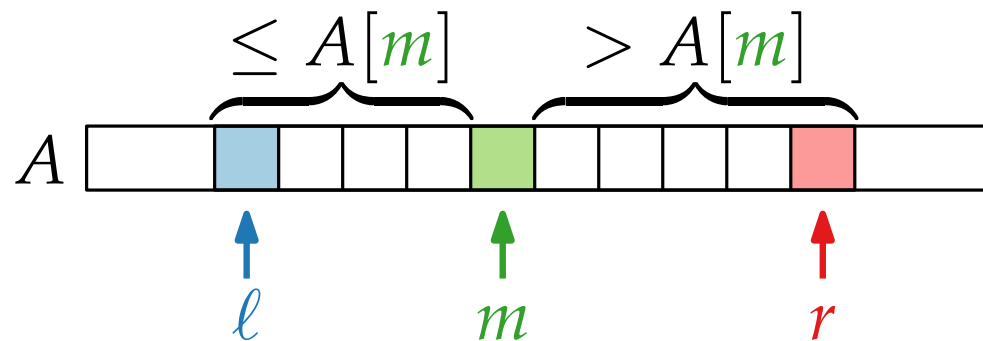
Fortgeschrittene Algorithmen

4. Vorlesung

Randomisierte Algorithmen II: Das Auswahlproblem

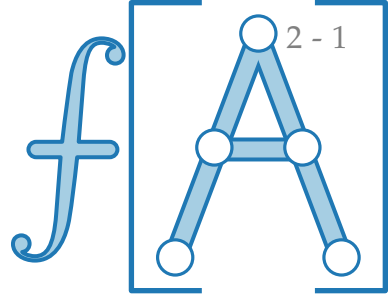


Teil I: RANDOMIZEDQUICKSORT



Wiederholung: QUICKSORT

QUICKSORT($A, \ell = 1, r = A.length$)

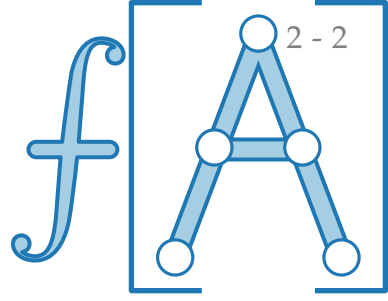


Wiederholung: QUICKSORT

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
    |
```



Wiederholung: QUICKSORT

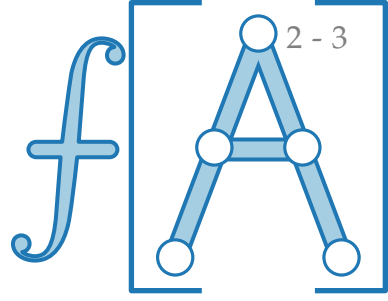
QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{PARTITION}(A, \ell, r)$

 QUICKSORT($A, \ell, m - 1$)

 QUICKSORT($A, m + 1, r$)



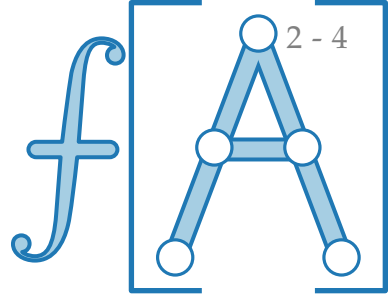
Wiederholung: QUICKSORT

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

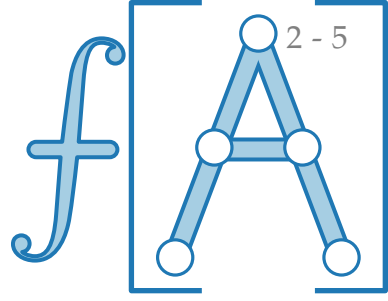
```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

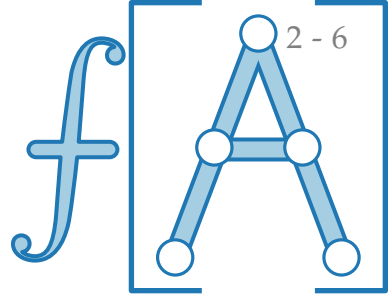
```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
    |  $m = \text{PARTITION}(A, \ell, r)$   
    | QUICKSORT( $A, \ell, m - 1$ )  
    | QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

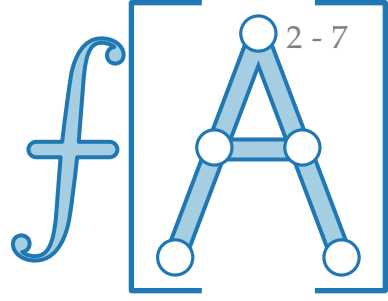
```
    | if  $A[j] \leq pivot$  then  
      | |  $A[i] \rightleftharpoons A[j]$   
      | |  $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Was passiert hier?

Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

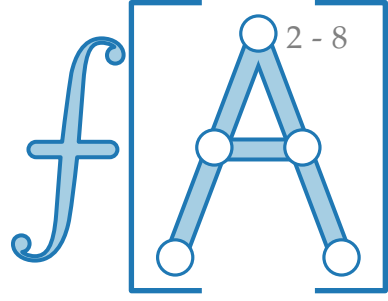
```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Was passiert hier?



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

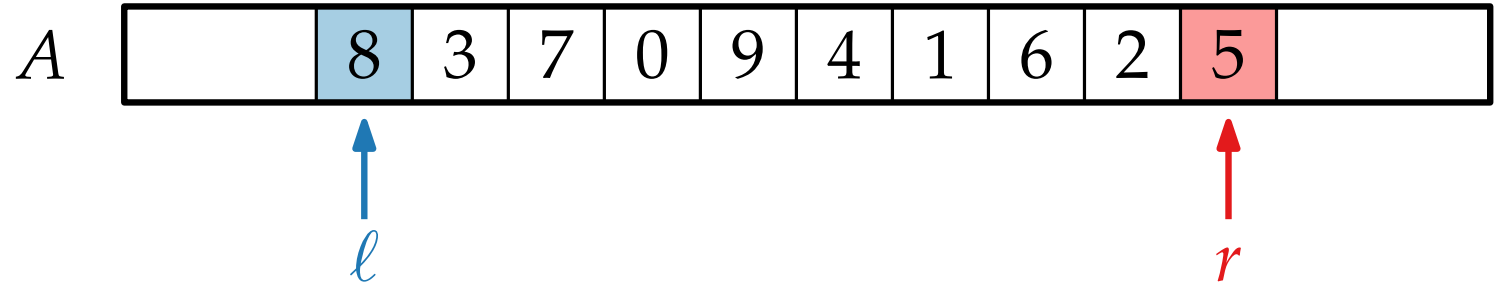
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

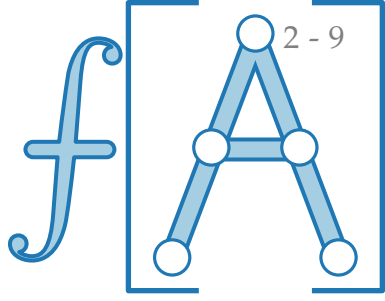
```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

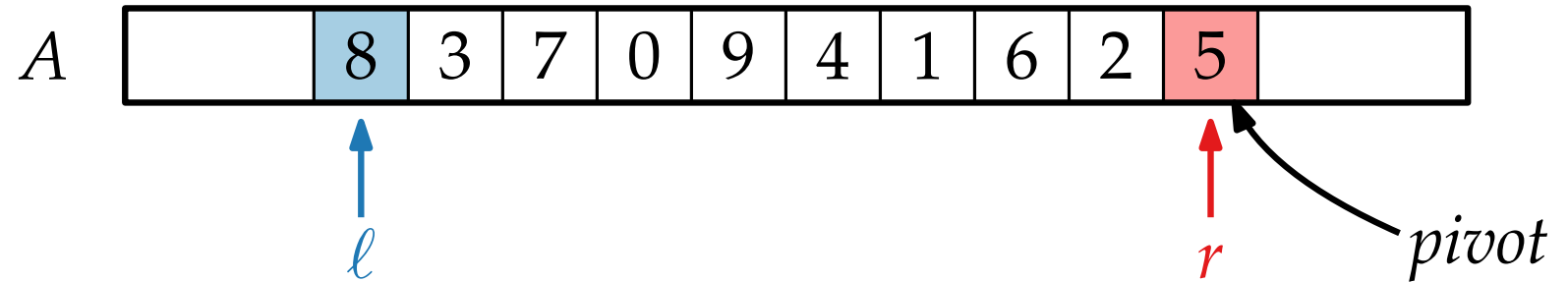
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

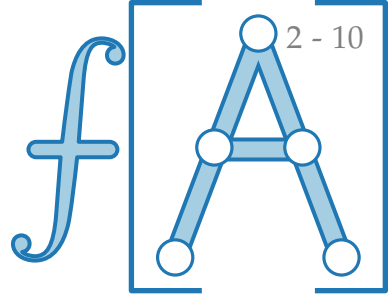
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

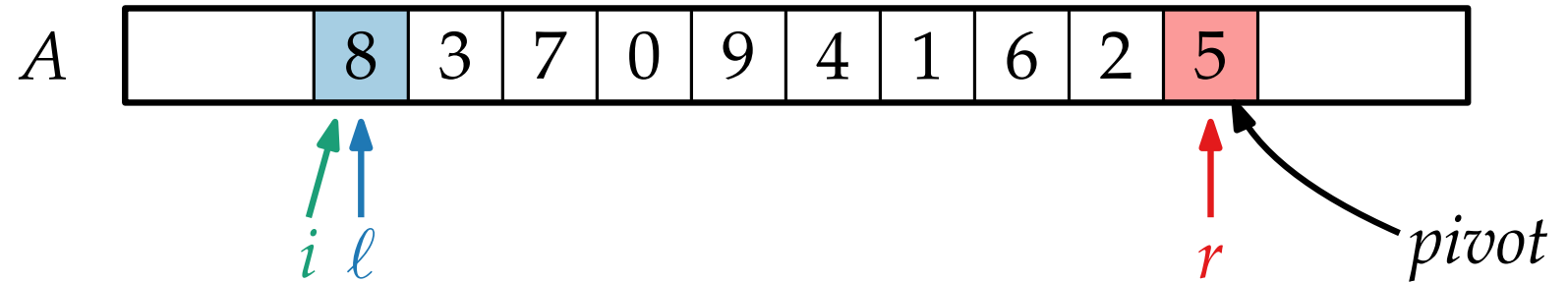
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

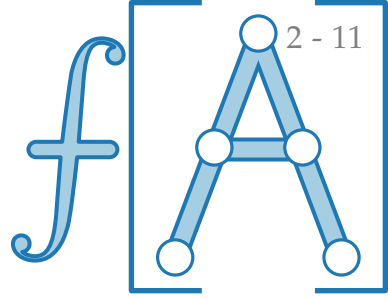
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

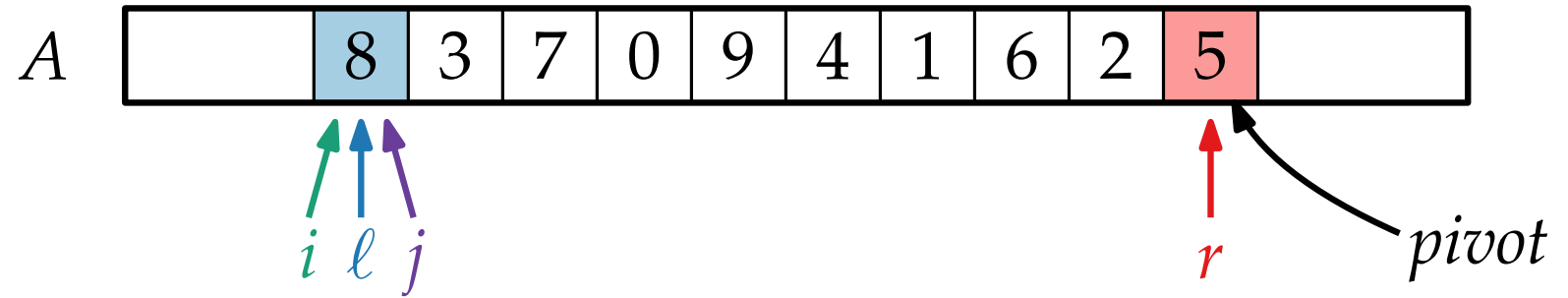
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

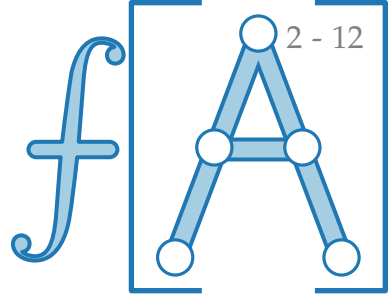
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

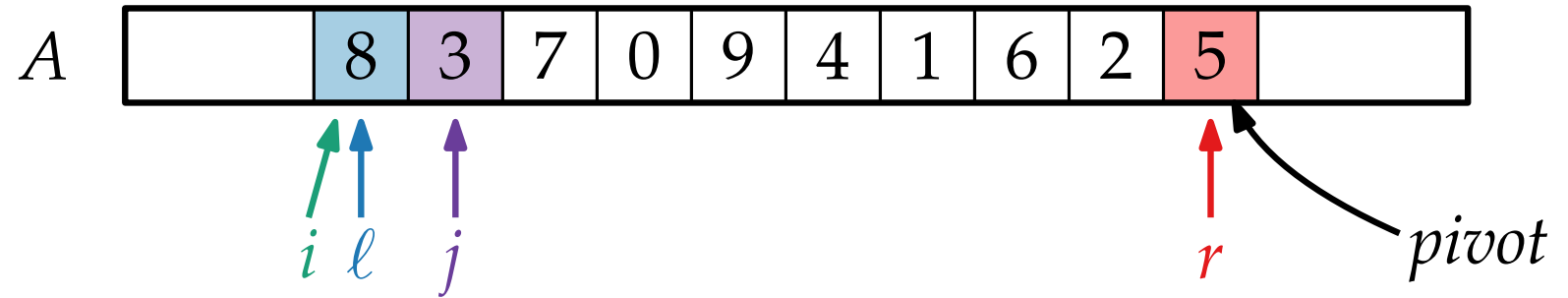
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

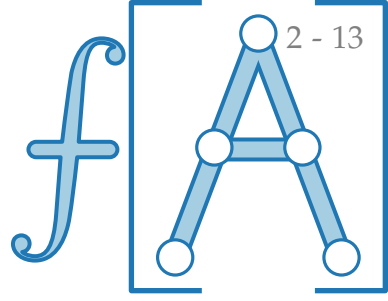
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

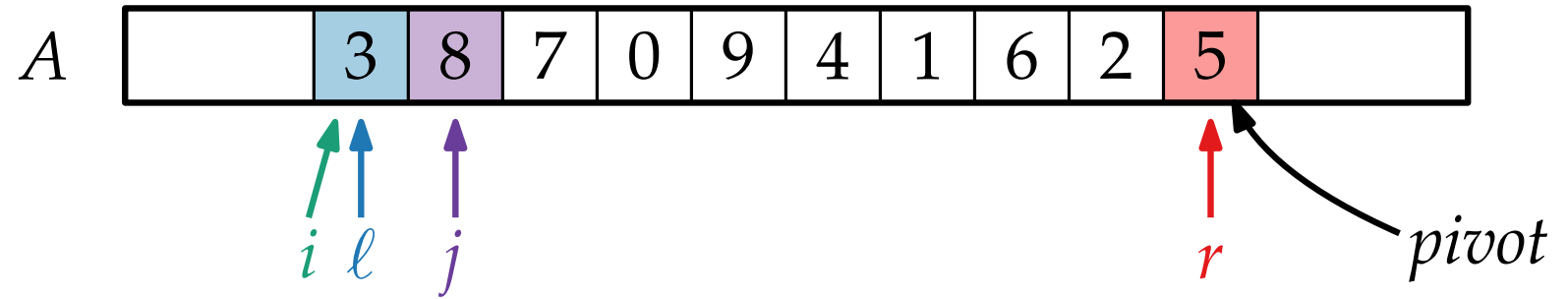
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

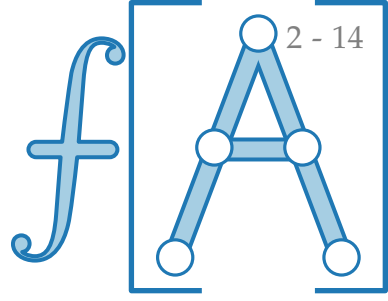
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

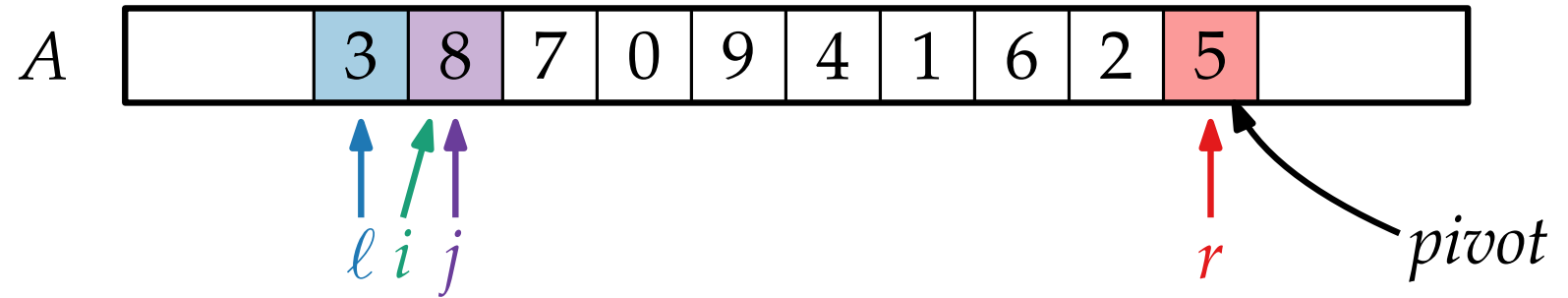
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

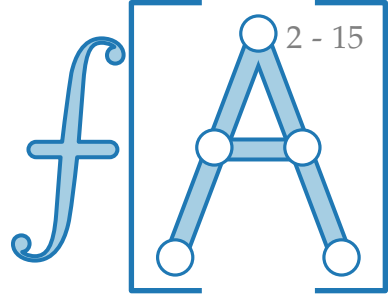
```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

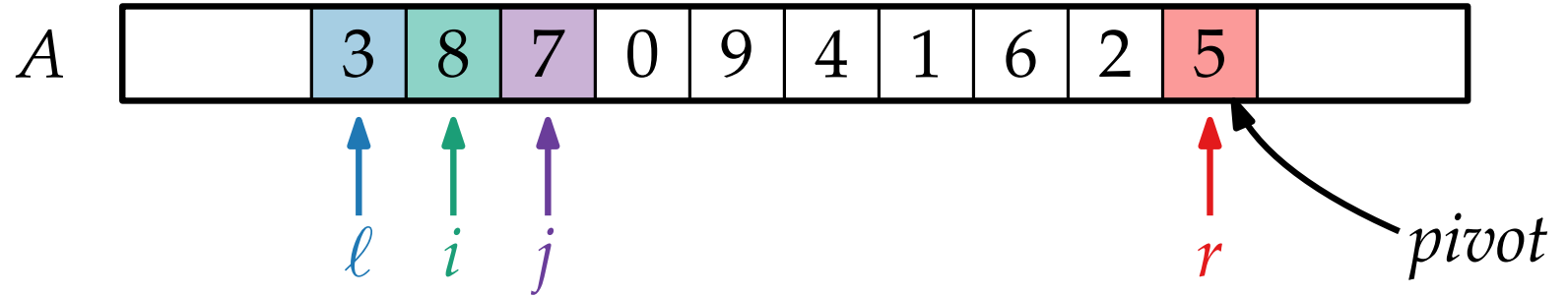
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

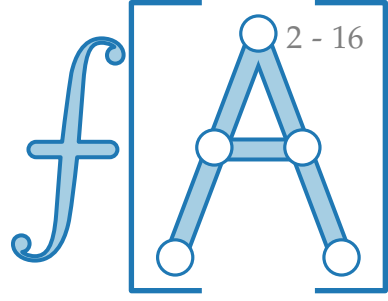
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

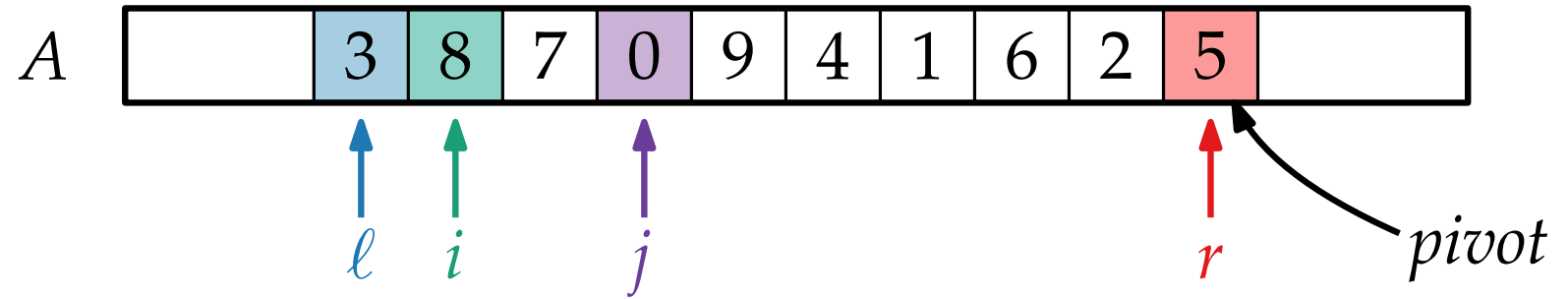
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

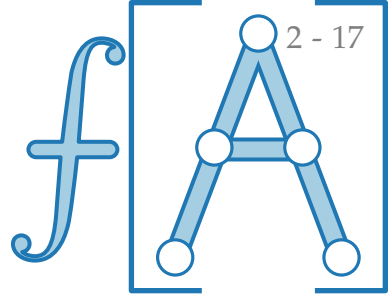
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

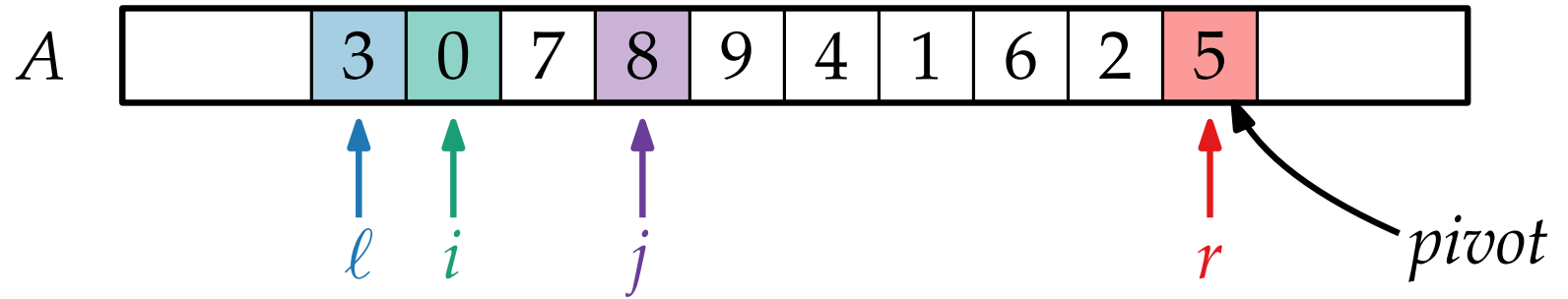
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

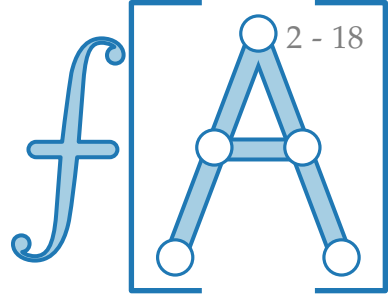
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

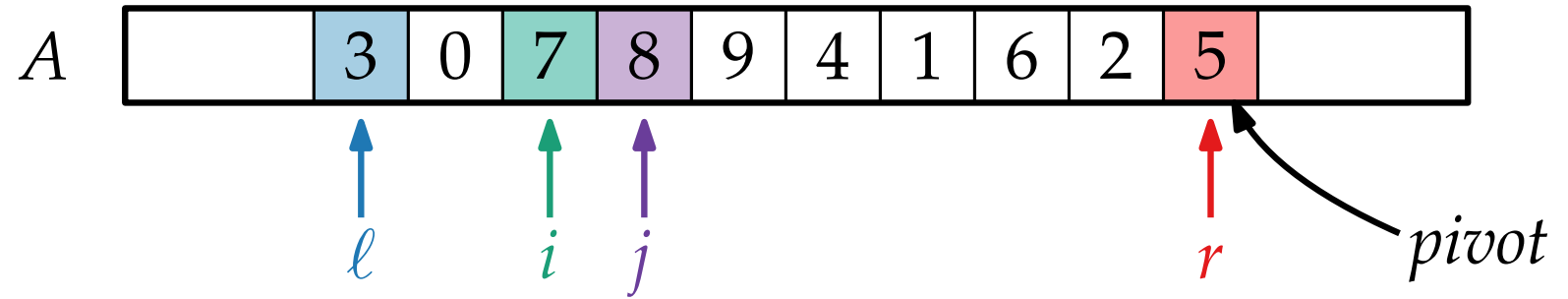
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

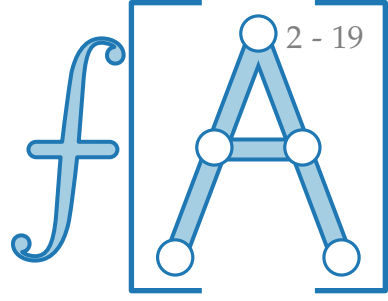
```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

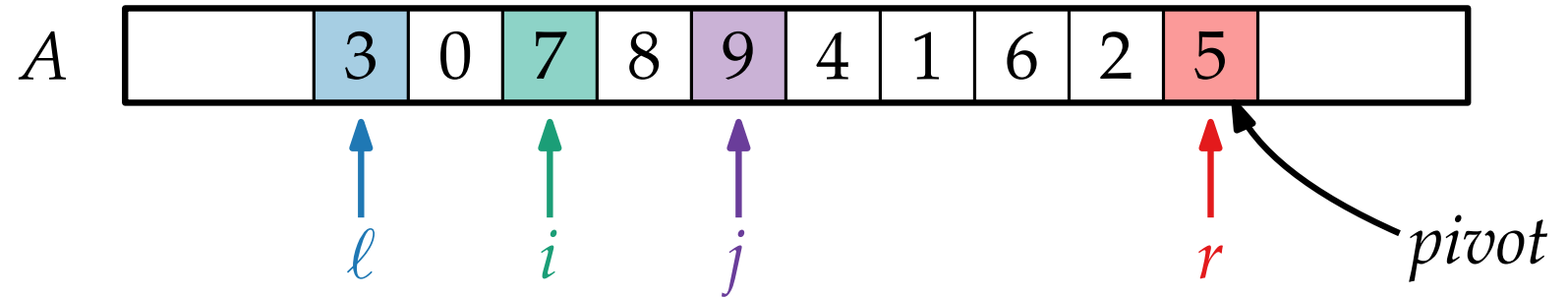
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

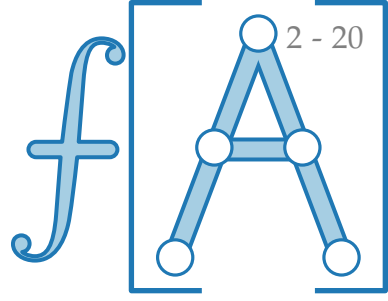
```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

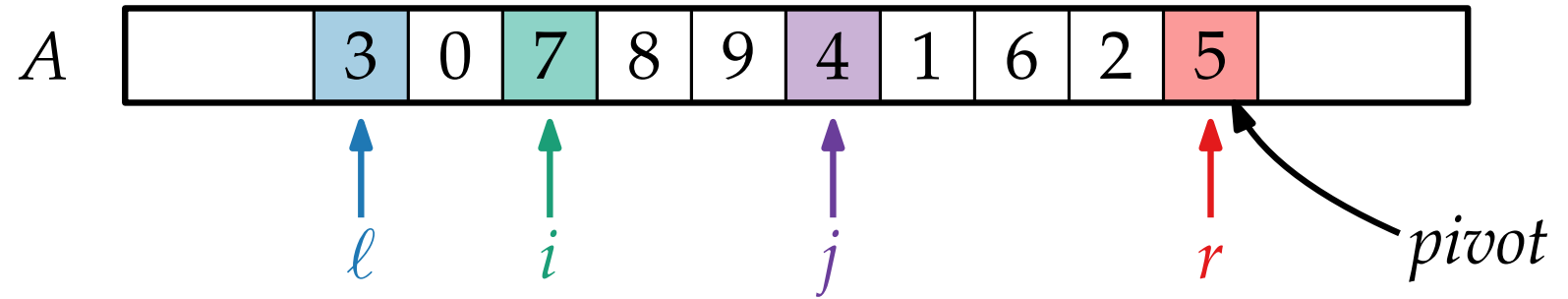
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

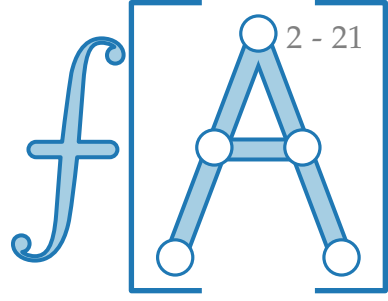
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

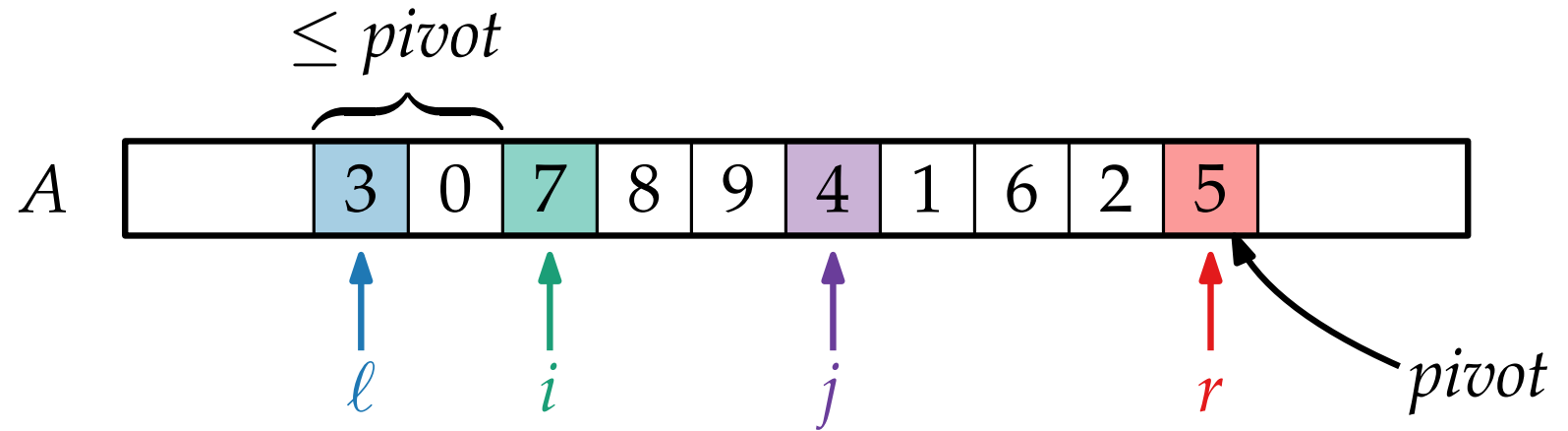
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

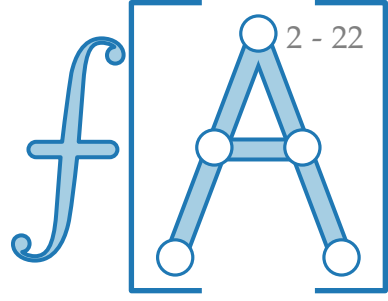
```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

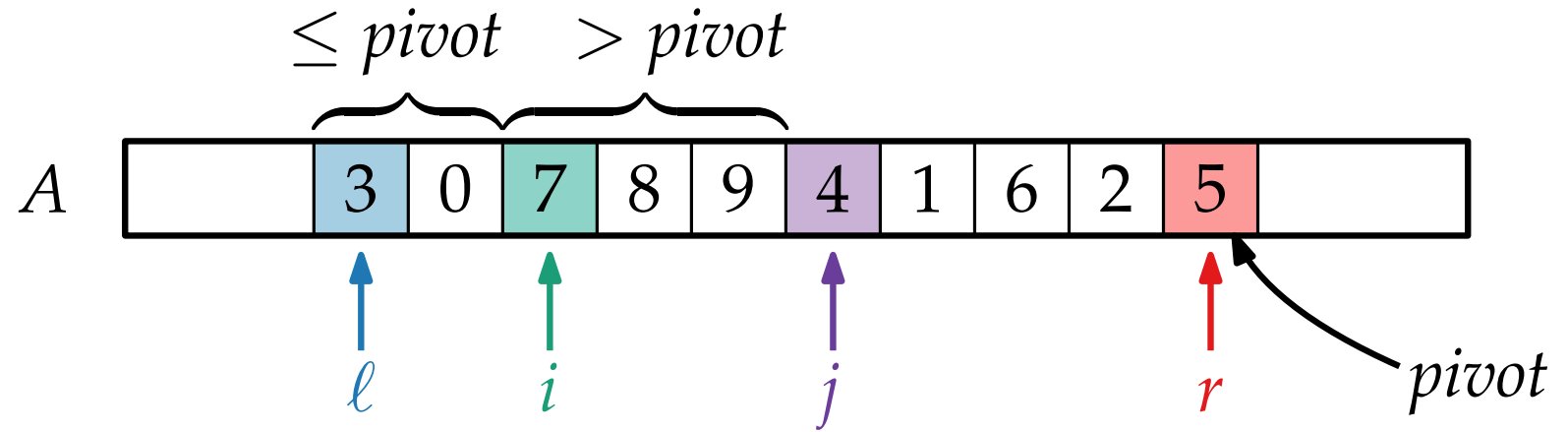
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

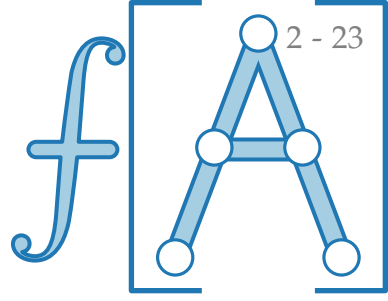
```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

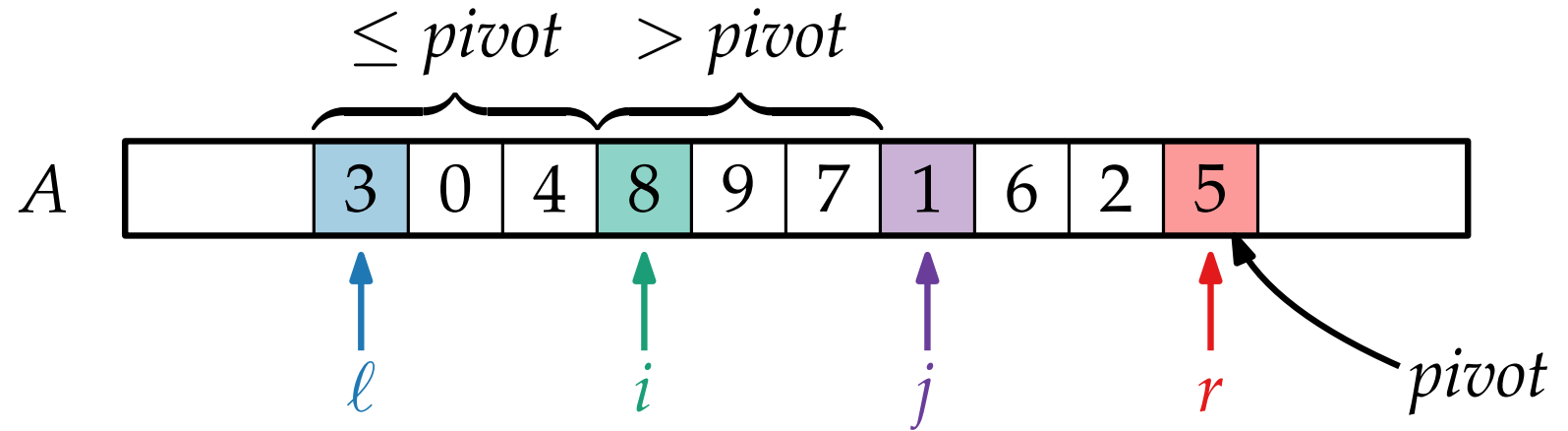
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

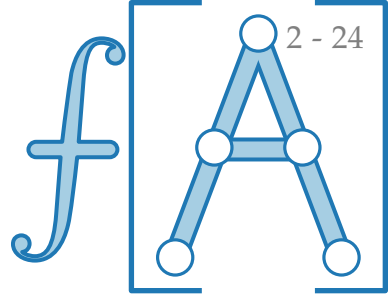
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

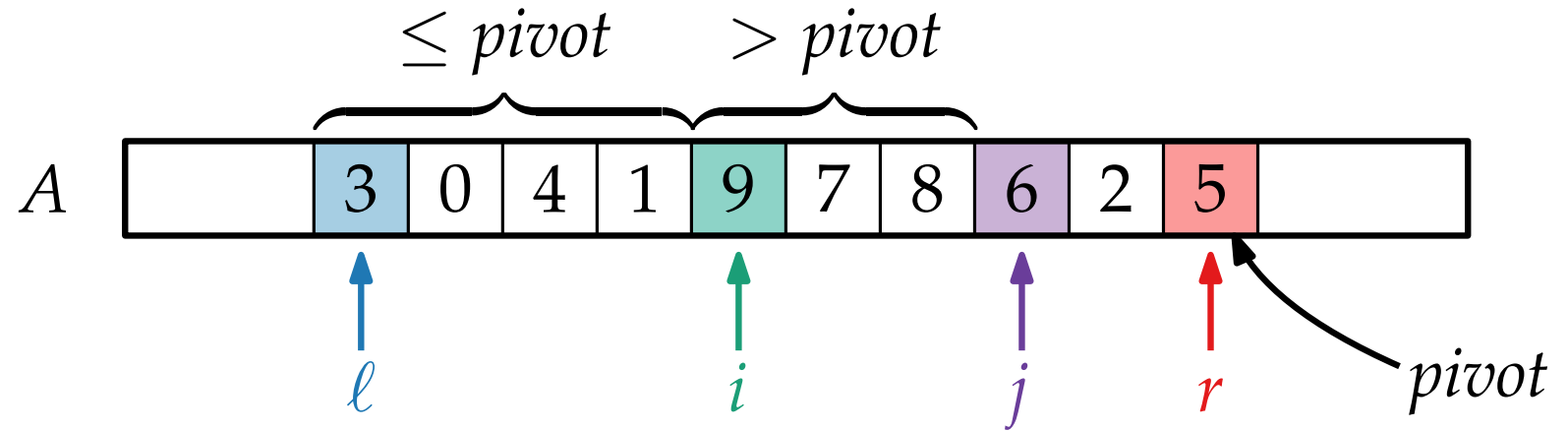
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

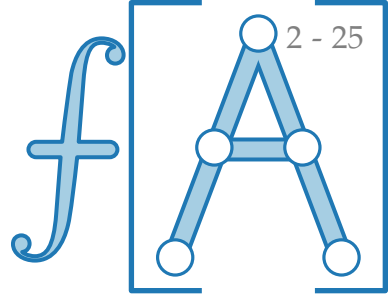
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

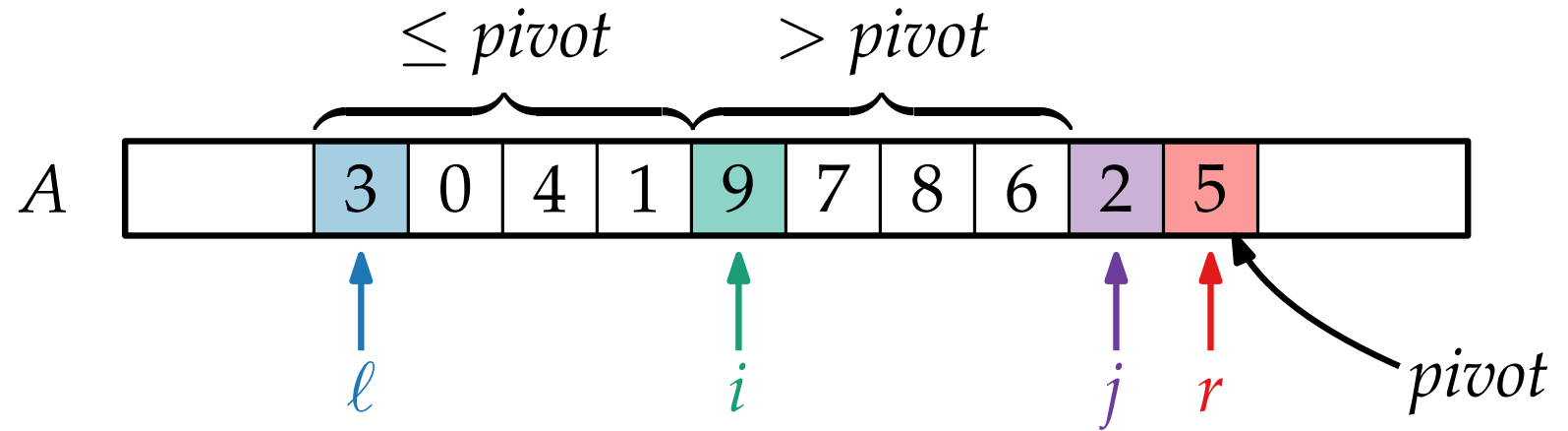
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

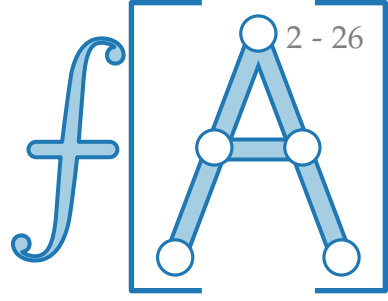
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

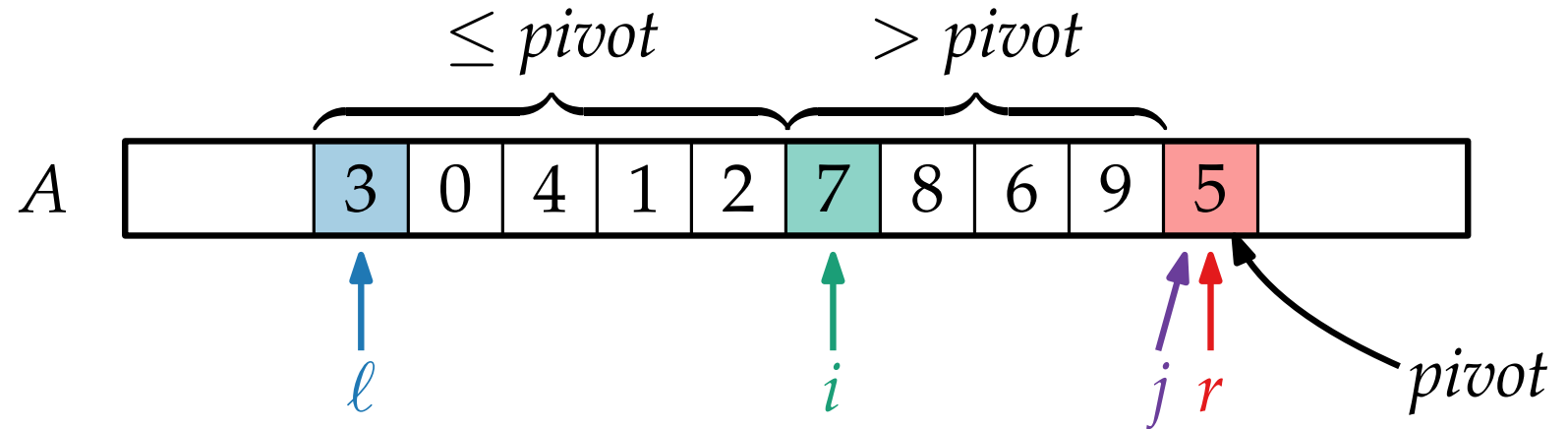
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

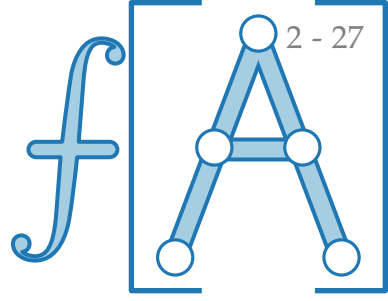
```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Wiederholung: QUICKSORT



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

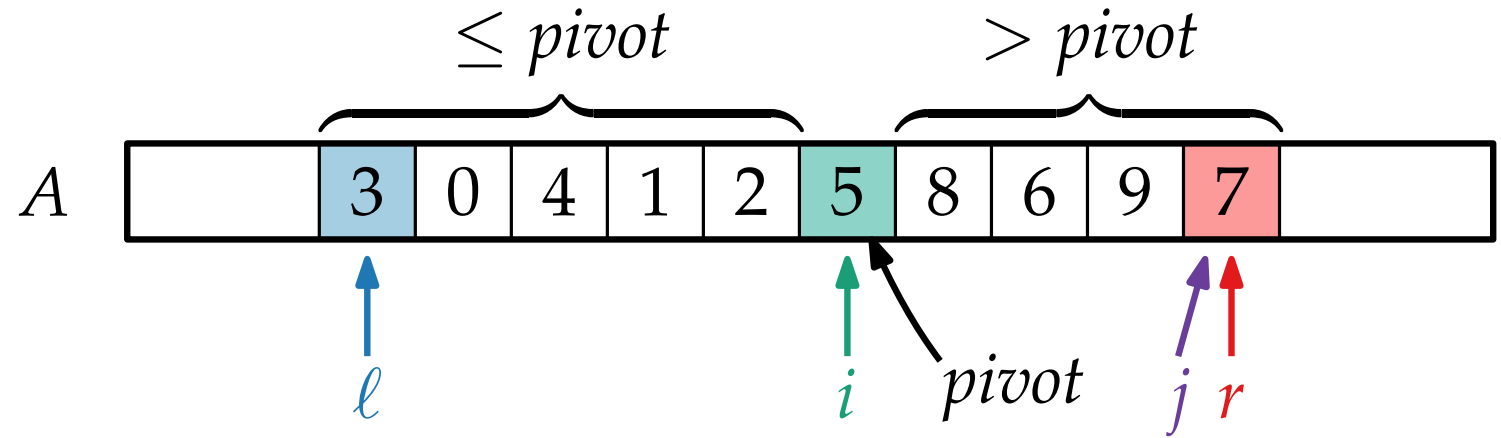
```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

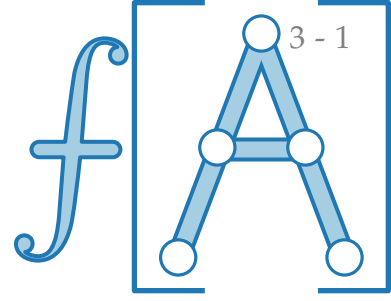
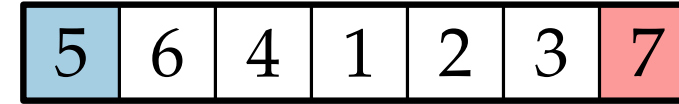
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

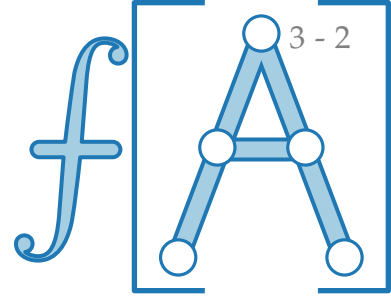
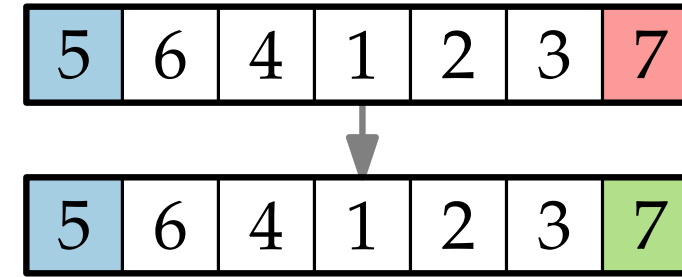
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

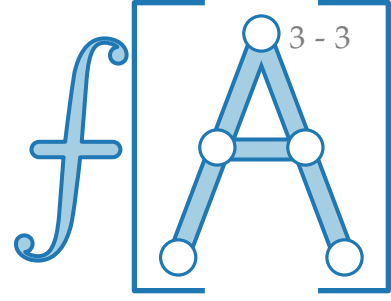
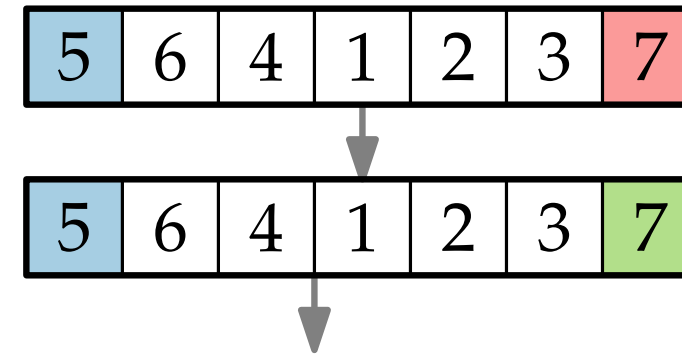
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

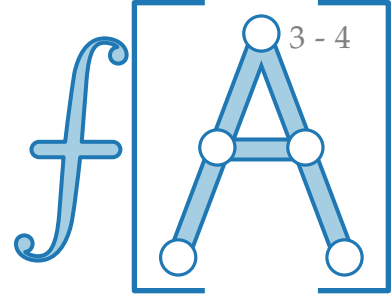
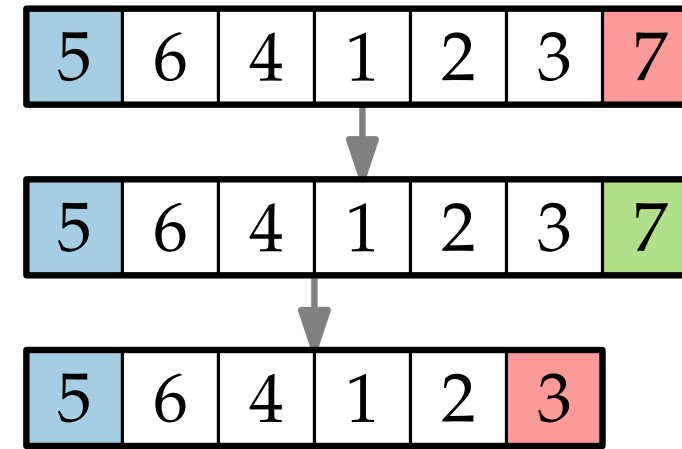
```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )  
  if  $\ell < r$  then  
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )  
  pivot =  $A[r]$   
   $i = \ell$   
  for  $j = \ell$  to  $r - 1$  do  
    if  $A[j] \leq \text{pivot}$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$   
   $A[i] \leftrightarrow A[r]$   
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

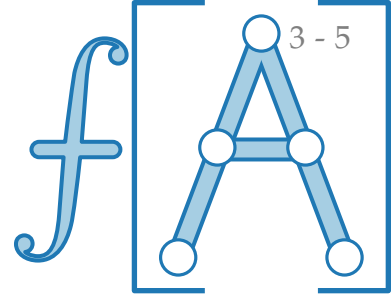
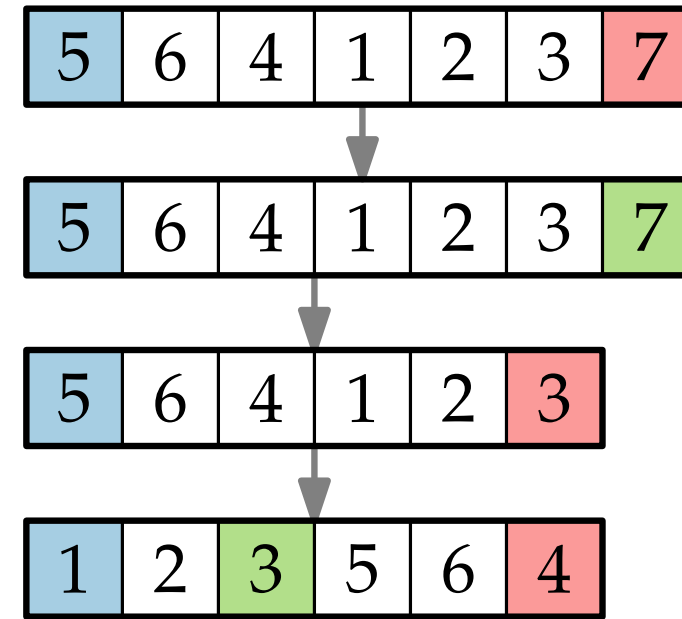
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

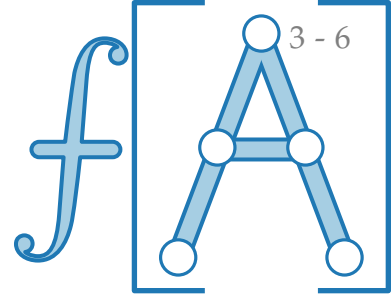
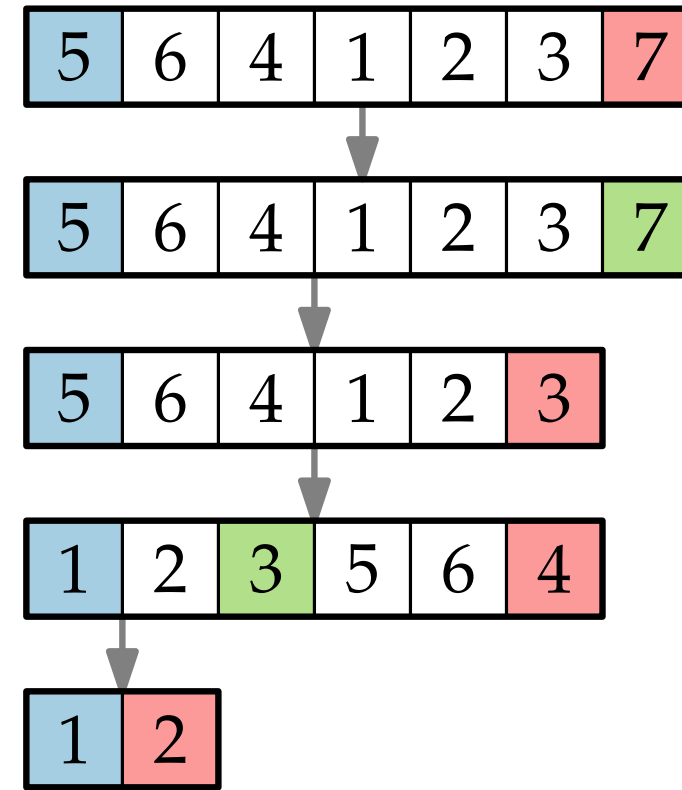
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

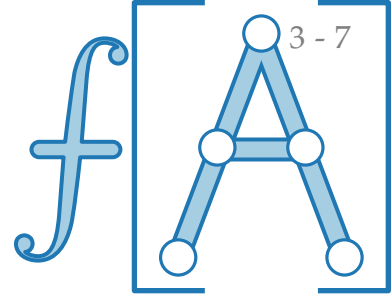
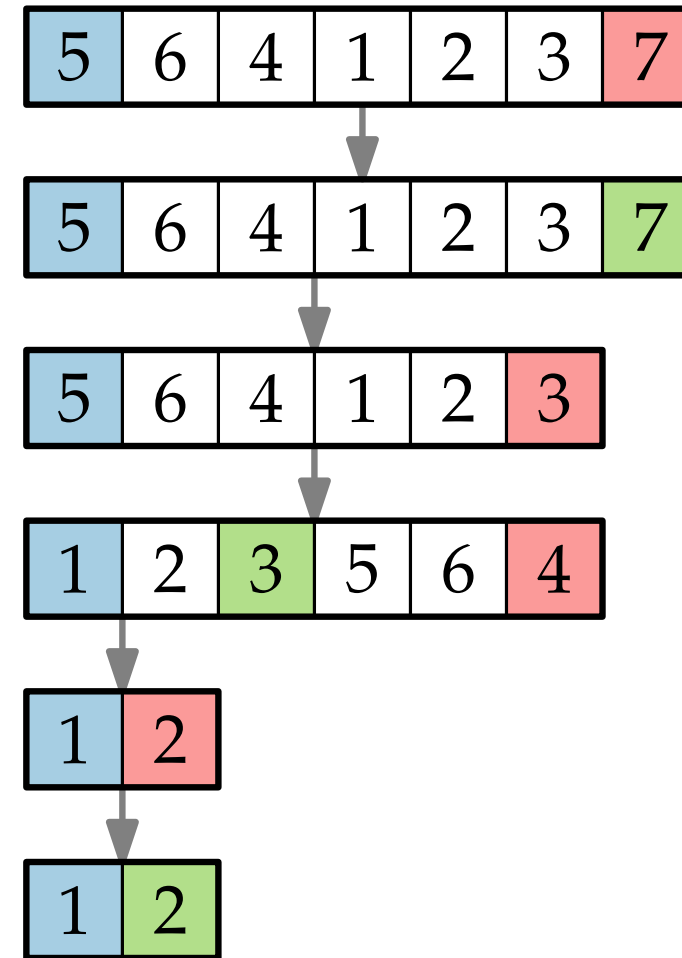
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

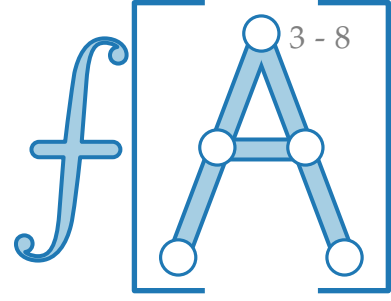
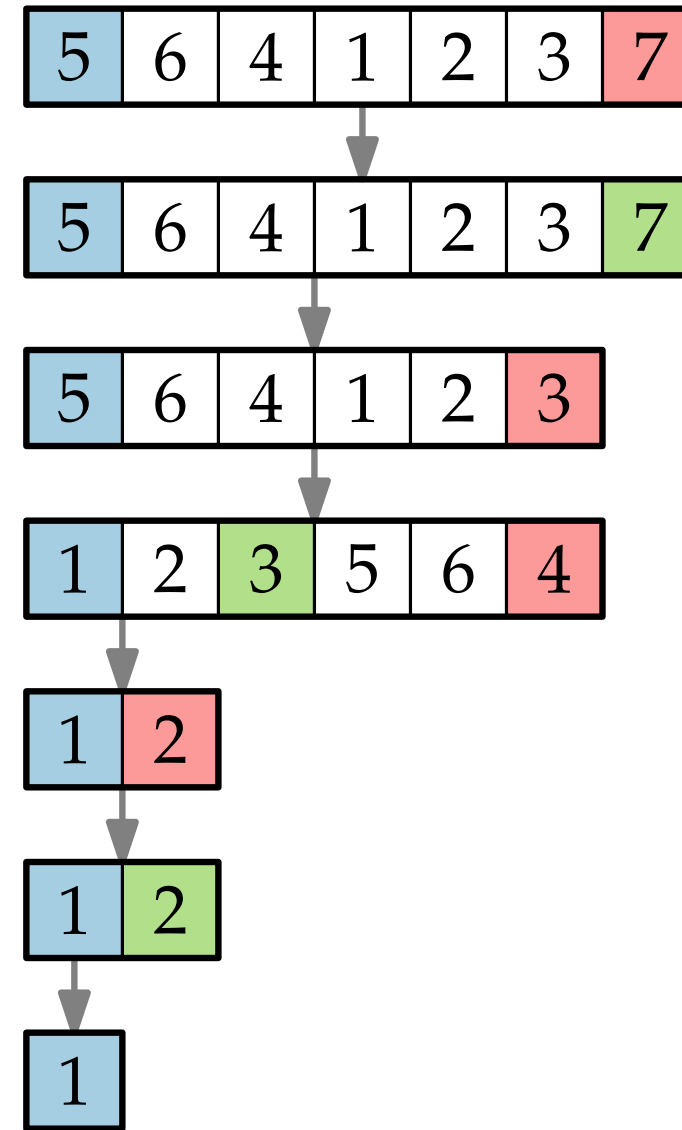
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$$m = \text{PARTITION}(A, \ell, r)$$
$$\text{QUICKSORT}(A, \ell, m - 1)$$
$$\text{QUICKSORT}(A, m + 1, r)$$

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

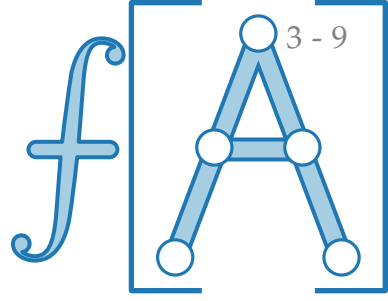
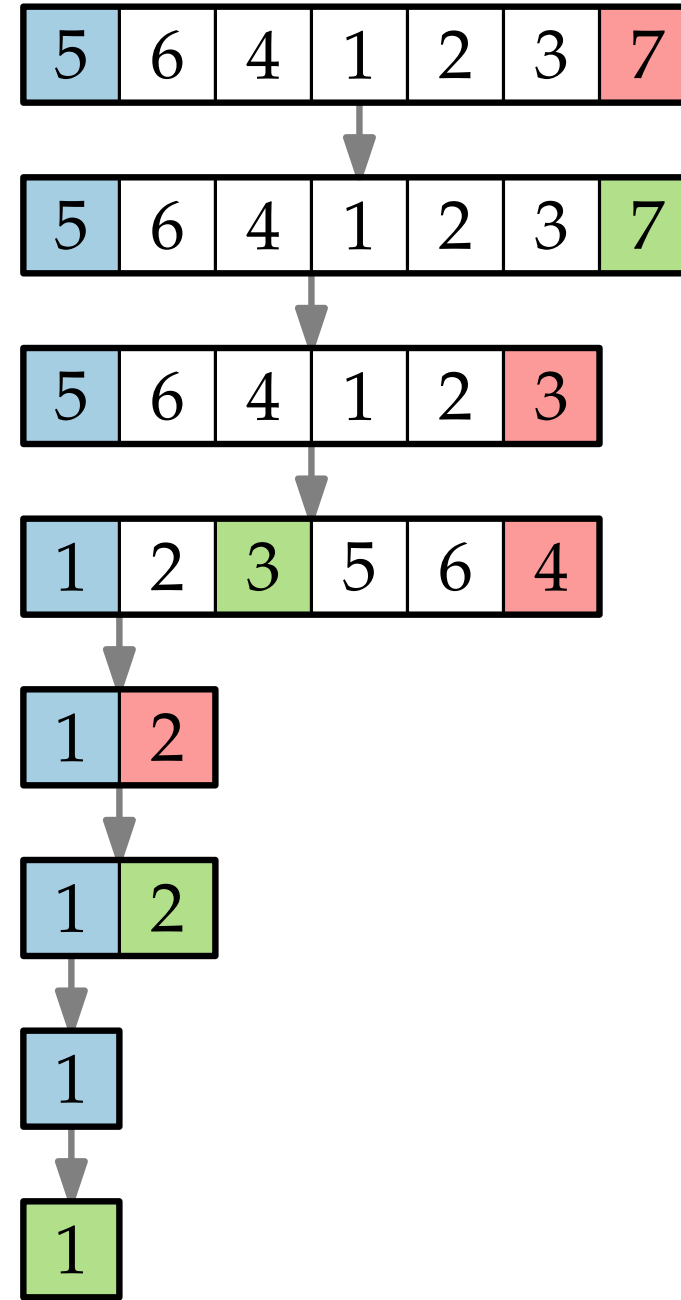
$$pivot = A[r]$$
$$i = \ell$$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$$A[i] \leftrightarrow A[j]$$
$$i = i + 1$$
$$A[i] \leftrightarrow A[r]$$

```
return i
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

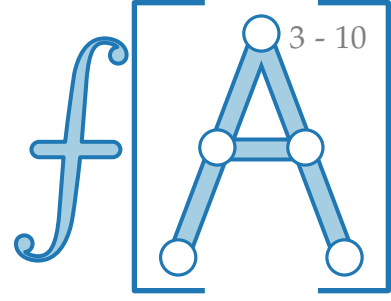
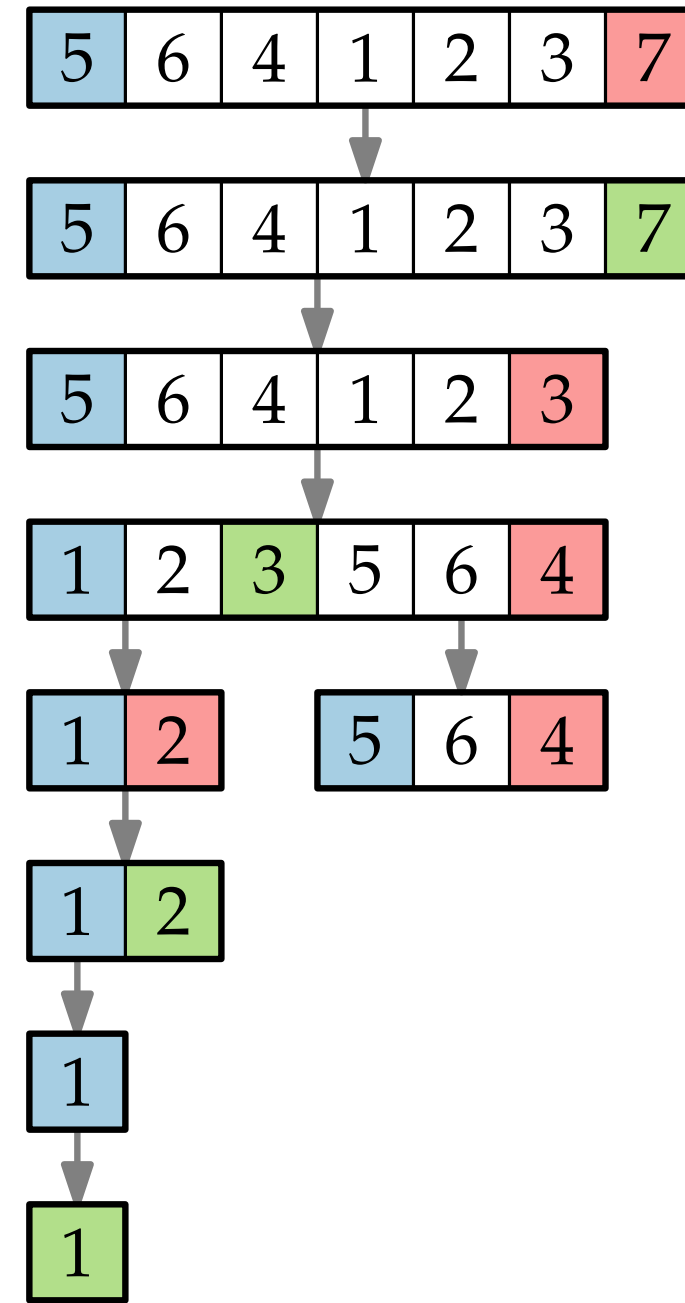
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

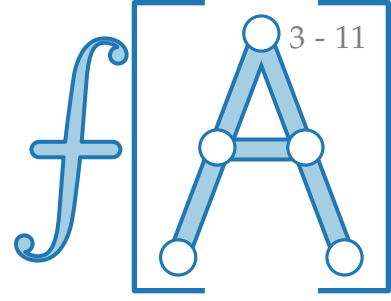
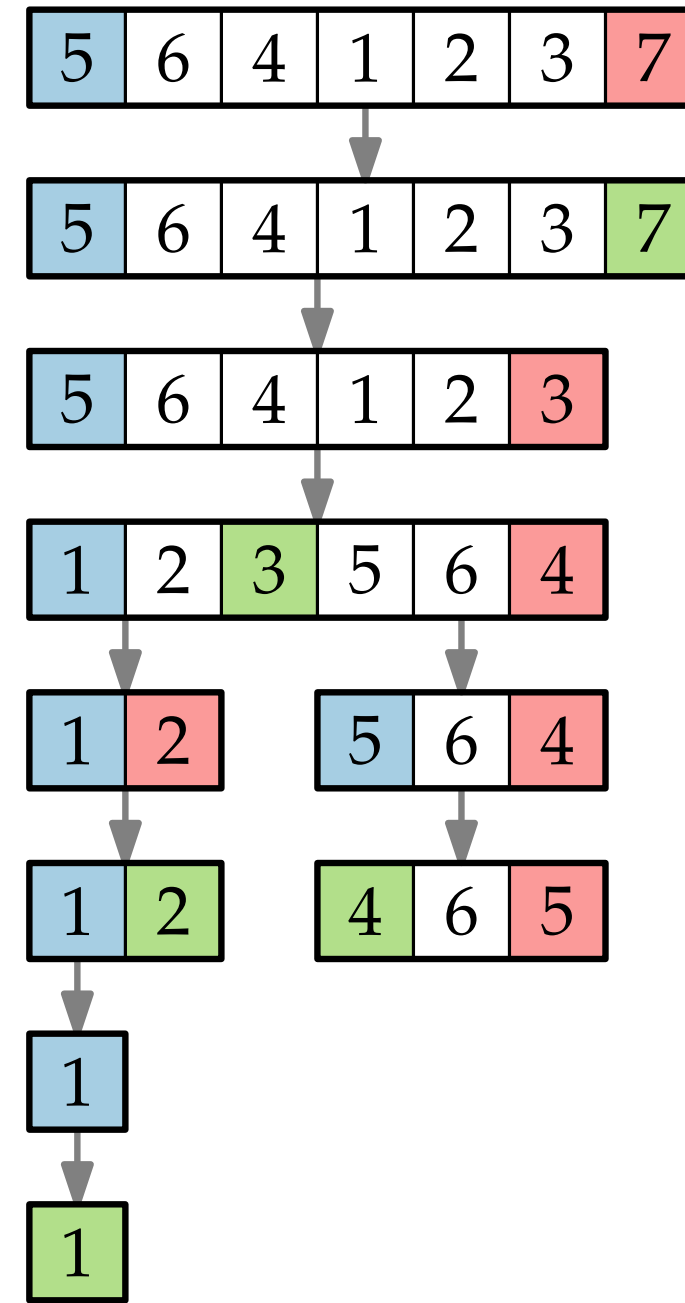
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

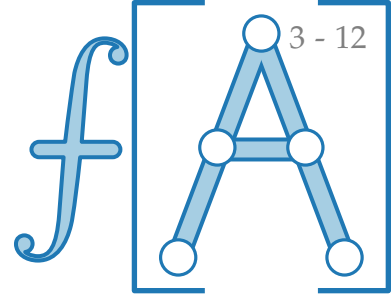
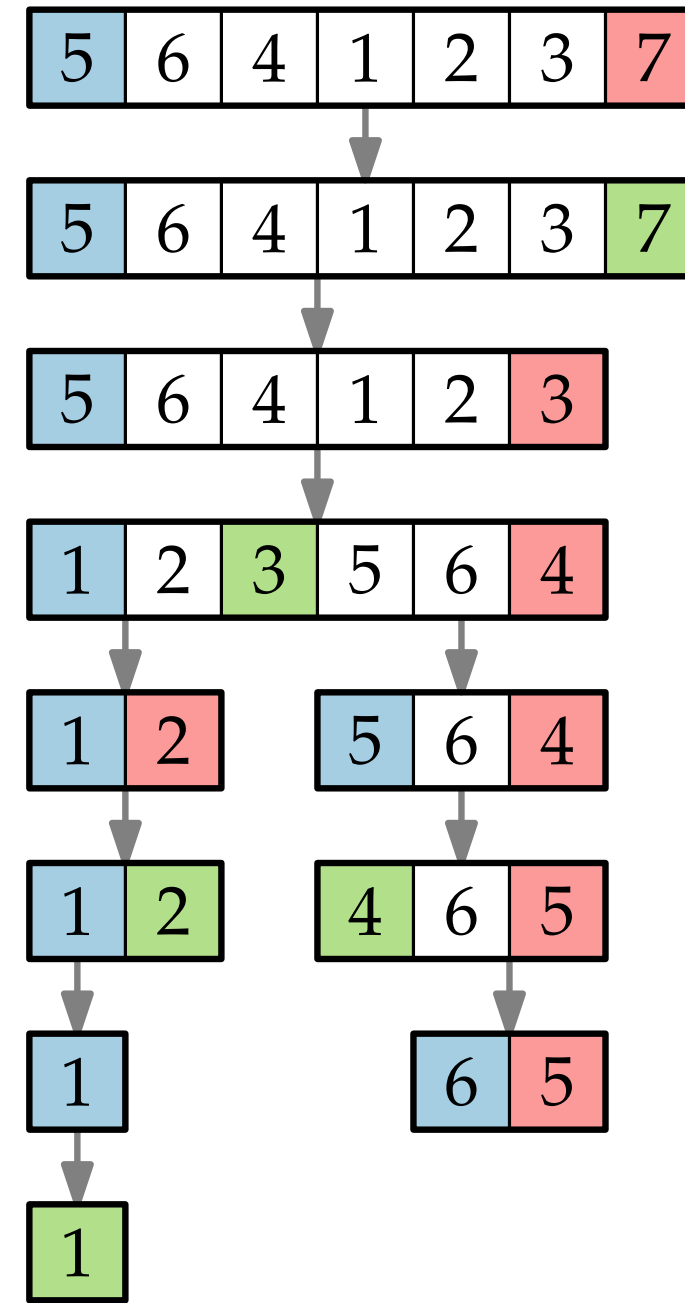
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

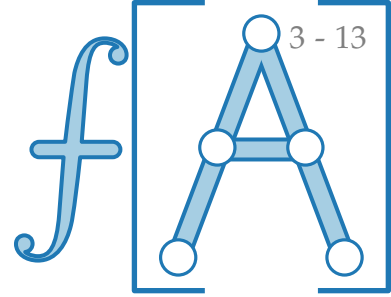
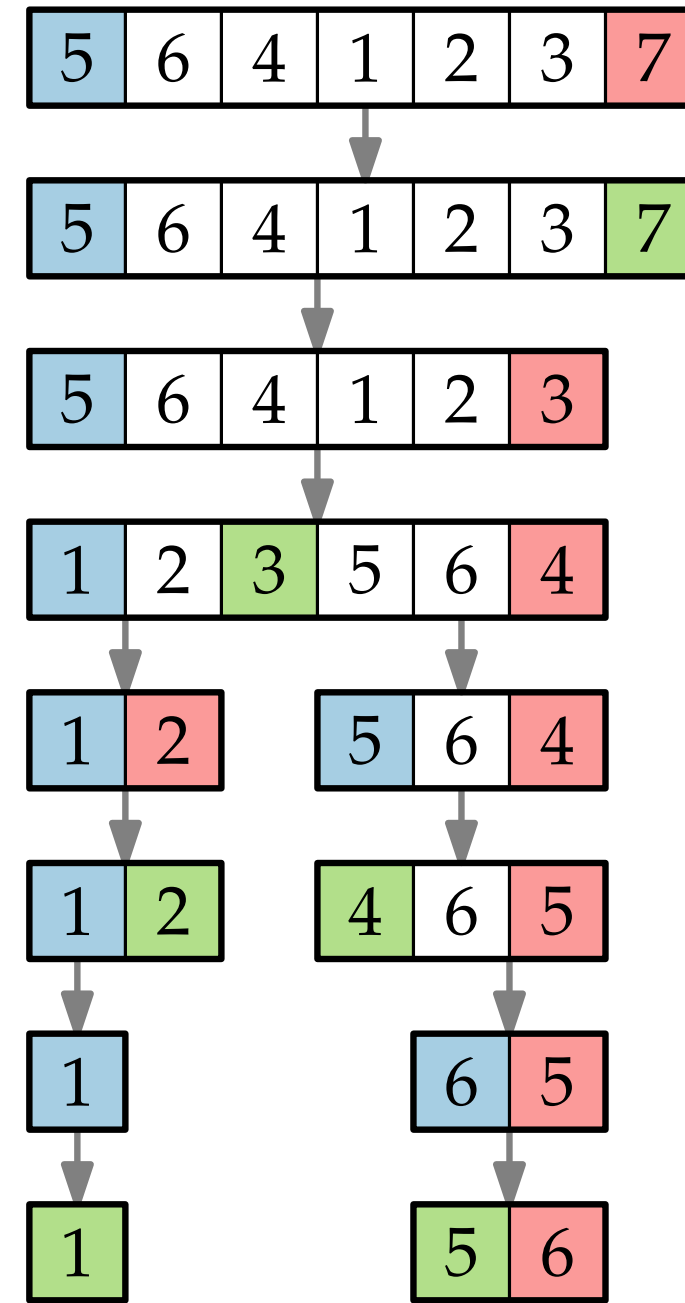
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

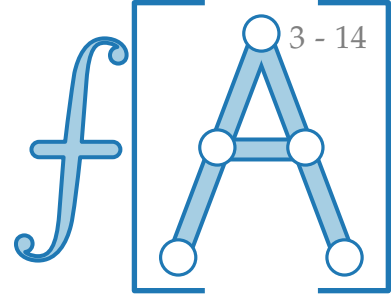
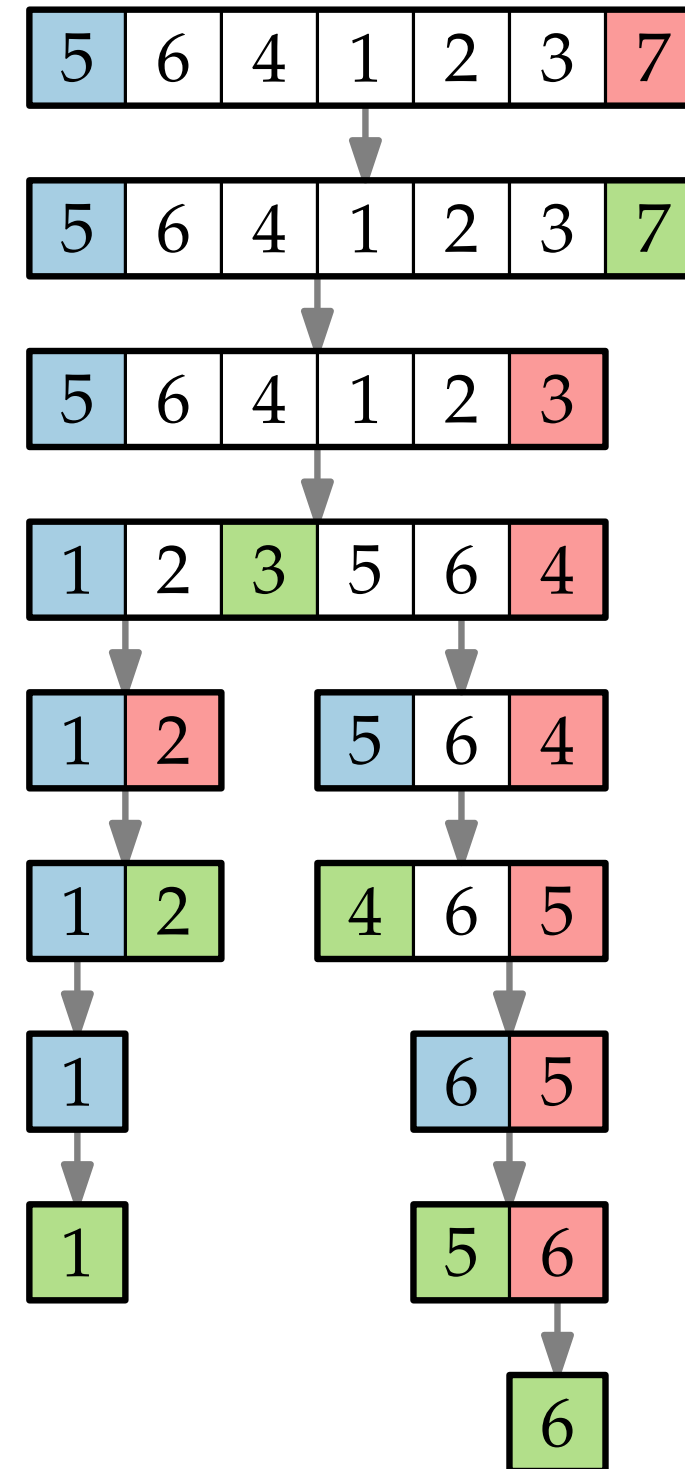
```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \leftrightarrow A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```



Ein Beispiel

QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{PARTITION}(A, \ell, r)$
 QUICKSORT($A, \ell, m - 1$)
 QUICKSORT($A, m + 1, r$)

int PARTITION(int[] A , int ℓ , int r)

$pivot = A[r]$

$i = \ell$

 for $j = \ell$ to $r - 1$ do

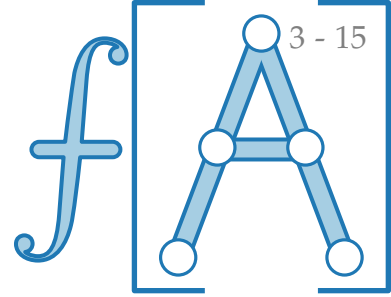
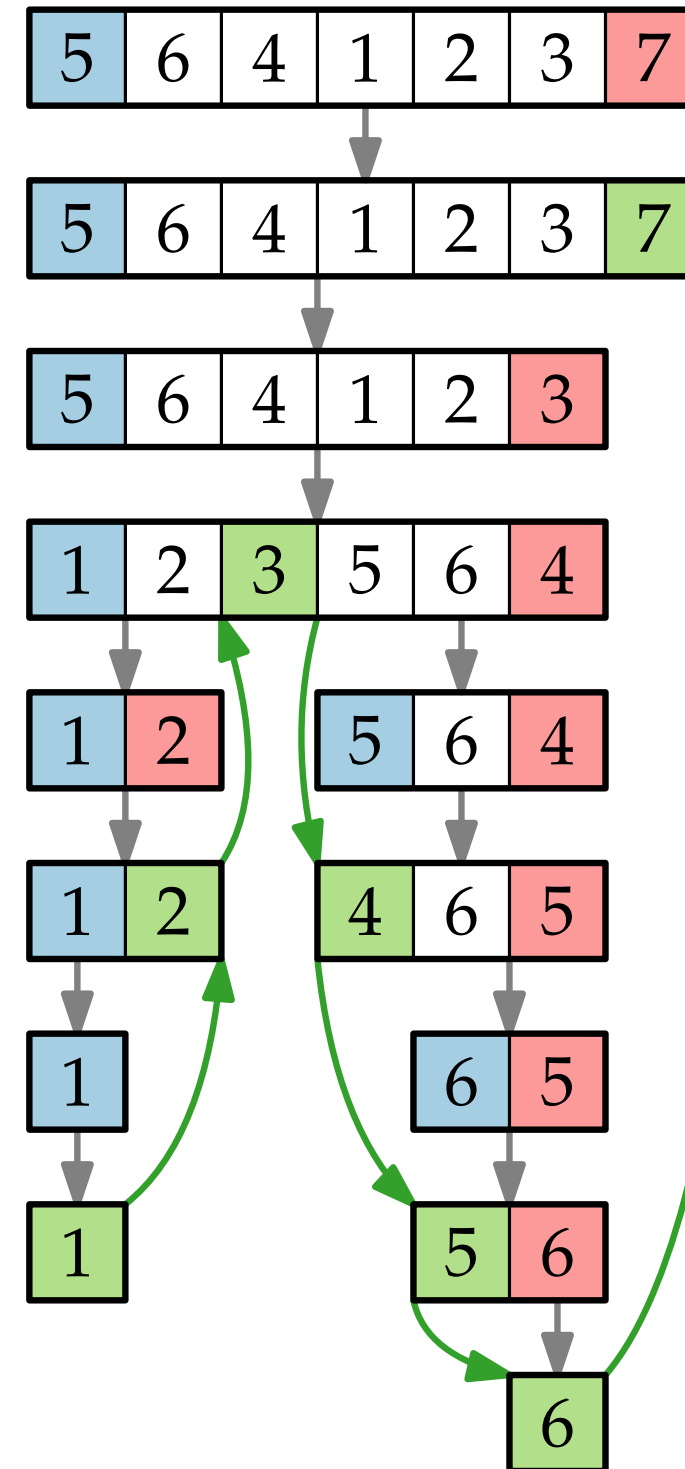
 if $A[j] \leq pivot$ then

$A[i] \leftrightarrow A[j]$

$i = i + 1$

$A[i] \leftrightarrow A[r]$

 return i



Ein Beispiel

QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{PARTITION}(A, \ell, r)$
 QUICKSORT($A, \ell, m - 1$)
 QUICKSORT($A, m + 1, r$)

int PARTITION(int[] A , int ℓ , int r)

$pivot = A[r]$

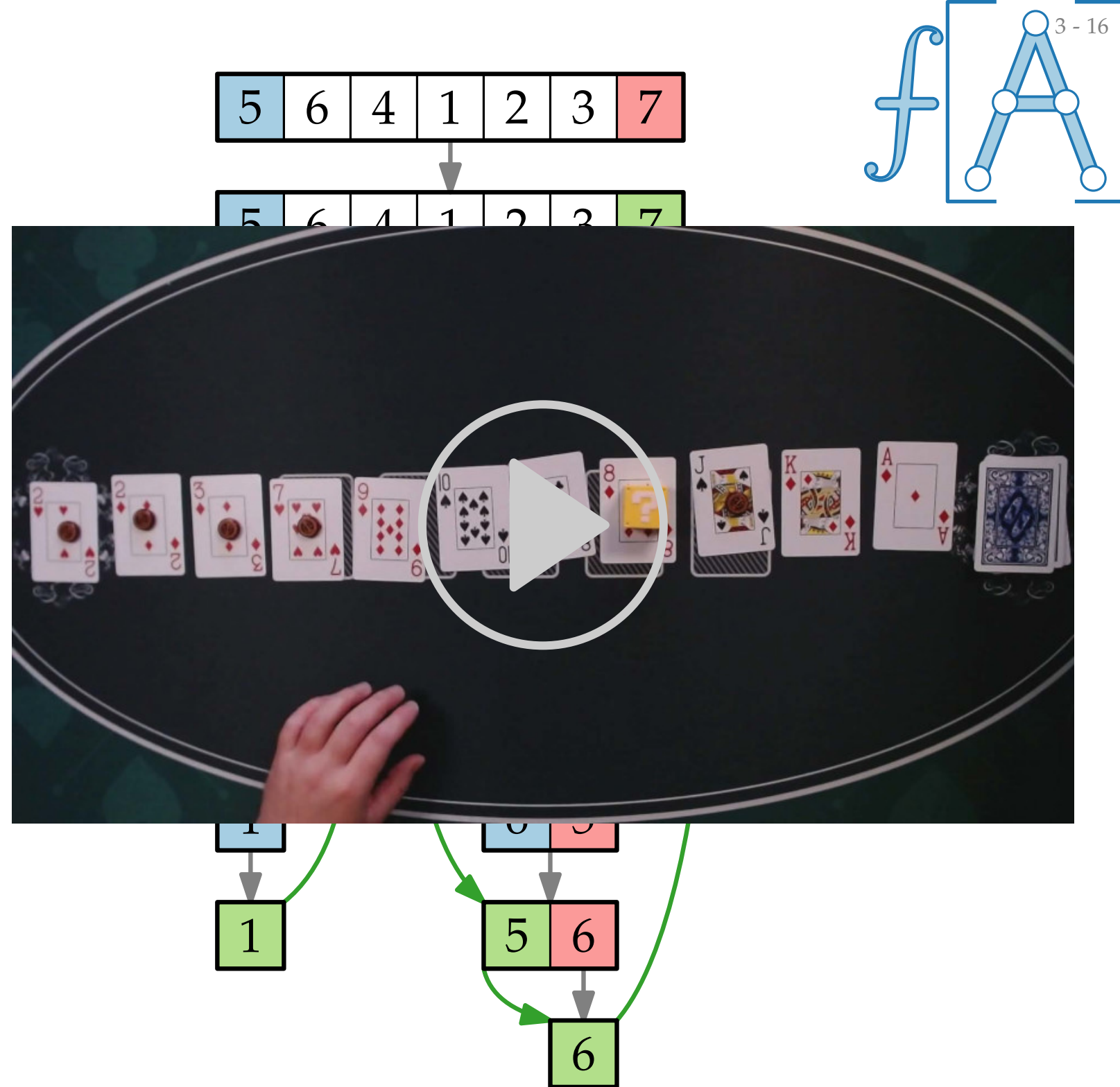
$i = \ell$

 for $j = \ell$ to $r - 1$ do

 if $A[j] \leq pivot$ then
 $A[i] \leftrightarrow A[j]$
 $i = i + 1$

$A[i] \leftrightarrow A[r]$

 return i



Ein Beispiel

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \leftrightarrow A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \leftrightarrow A[r]$ 
```

```
  return  $i$ 
```

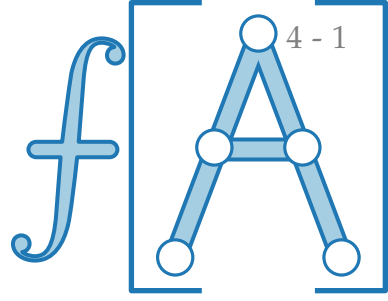
3 - 17

Demo.

<https://algo.uni-trier.de/demos/sort.html>

6

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Laufzeit

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

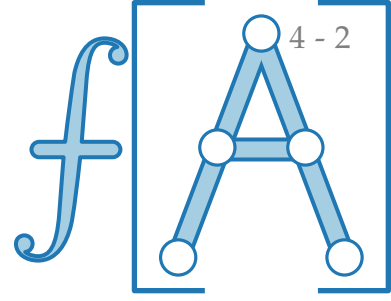
```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!



Laufzeit

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

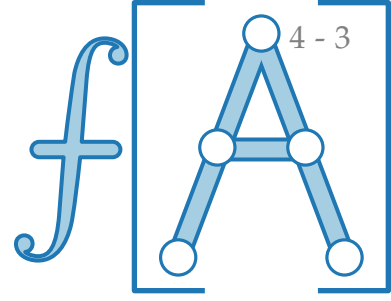
```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!



Laufzeit

```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

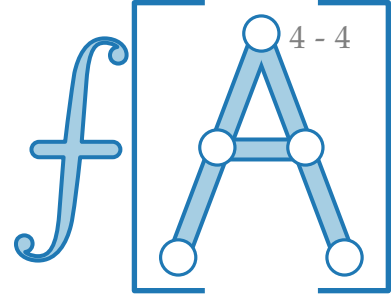
```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

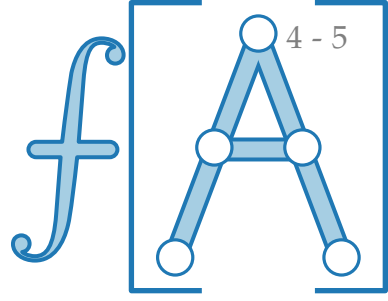
Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer**

Vergleiche.



Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
  pivot =  $A[r]$ 
```

```
   $i = \ell$ 
```

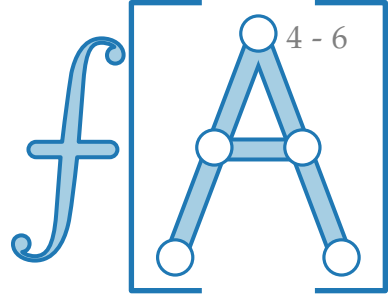
```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq \text{pivot}$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

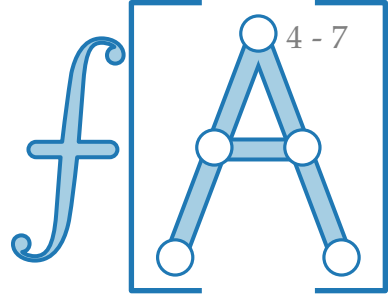
Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

$T_{\text{QS}}(n) =$

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1)$$

```
int PARTITION(int[ ] A, int  $\ell$ , int  $r$ )
```

```
  pivot =  $A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq \text{pivot}$  then
```

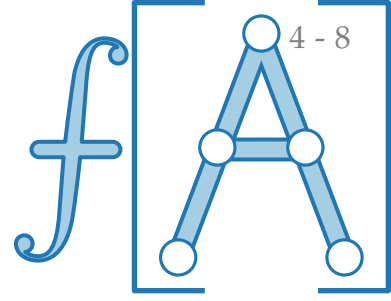
```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

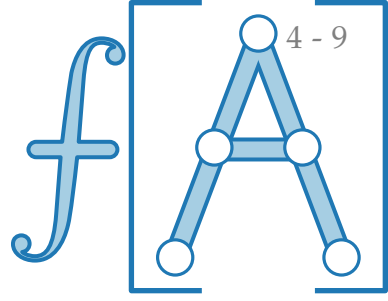
Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m)$$

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
    [  $m = \text{PARTITION}(A, \ell, r)$   
      QUICKSORT( $A, \ell, m - 1$ )  
      QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    [ if  $A[j] \leq pivot$  then  
      [  $A[i] \rightleftharpoons A[j]$   
        [  $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

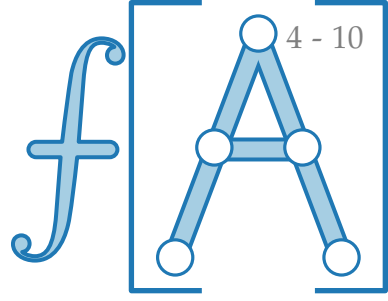
Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

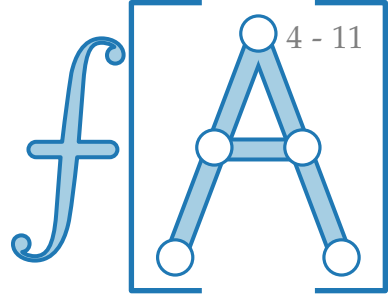
Wovon hängt dann die Laufzeit ab?

$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

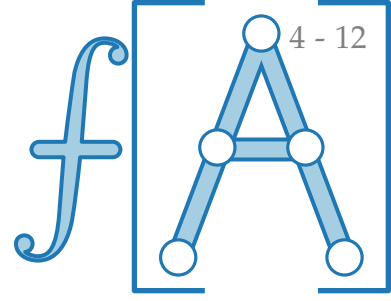
$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

$$T_{\text{QS}}(n) = T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1$$

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

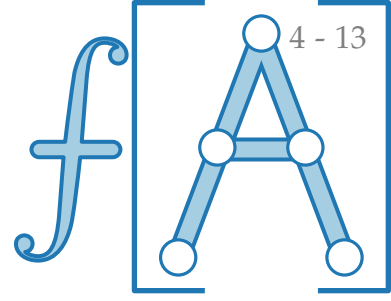
$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

$$T_{\text{QS}}(n) = T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1$$

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
    [  $m = \text{PARTITION}(A, \ell, r)$   
      QUICKSORT( $A, \ell, m - 1$ )  
      QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
      [  $A[i] \rightleftharpoons A[j]$   
         $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

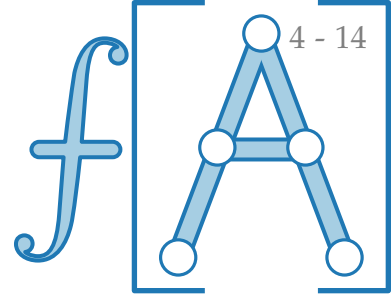
$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m ⁰ immer erstes Element

$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \end{aligned}$$

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
    [  $m = \text{PARTITION}(A, \ell, r)$   
      QUICKSORT( $A, \ell, m - 1$ )  
      QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
      [  $A[i] \rightleftharpoons A[j]$   
         $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

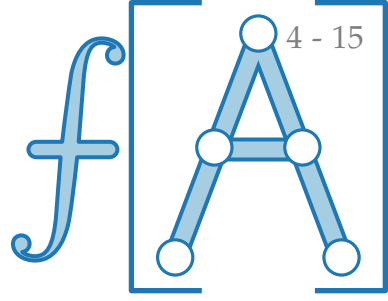
$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \end{aligned}$$

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

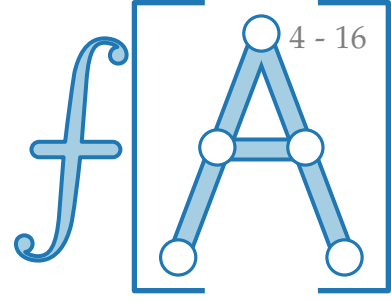
$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \\ &= T_{\text{QS}}(1) + 1 + 2 + \cdots + n - 2 + n - 1 \end{aligned}$$

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
    [  $m = \text{PARTITION}(A, \ell, r)$   
      QUICKSORT( $A, \ell, m - 1$ )  
      QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then  
      [  $A[i] \rightleftharpoons A[j]$   
         $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

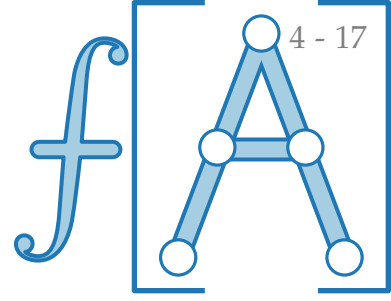
$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \\ &= T_{\text{QS}}(1) + 1 + 2 + \dots + n - 2 + n - 1 \end{aligned}$$

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

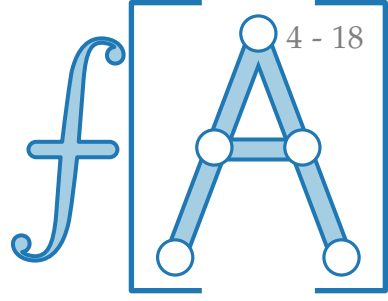
$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \\ &= T_{\text{QS}}(1) + 1 + 2 + \cdots + n - 2 + n - 1 \\ &= \sum_{i=0}^{n-1} i \end{aligned}$$

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

```
        if  $A[j] \leq pivot$  then
```

```
             $A[i] \rightleftharpoons A[j]$ 
```

```
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

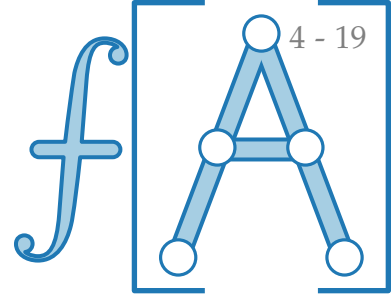
$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \\ &= T_{\text{QS}}(1) + 1 + 2 + \dots + n - 2 + n - 1 \\ &= \sum_{i=0}^{n-1} i \in \Theta(n^2) \end{aligned}$$

2. Extremfall: m immer mittleres Element

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

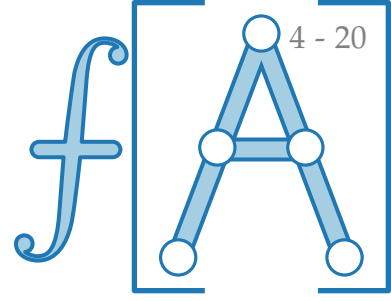
1. Extremfall: m immer erstes Element

$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \\ &= T_{\text{QS}}(1) + 1 + 2 + \cdots + n - 2 + n - 1 \\ &= \sum_{i=0}^{n-1} i \in \Theta(n^2) \end{aligned}$$

2. Extremfall: m immer mittleres Element

$$T_{\text{QS}}(n) \approx$$

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$   
    QUICKSORT( $A, \ell, m - 1$ )  
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
     $pivot = A[r]$ 
```

```
     $i = \ell$ 
```

```
    for  $j = \ell$  to  $r - 1$  do
```

```
        if  $A[j] \leq pivot$  then  
             $A[i] \rightleftharpoons A[j]$   
             $i = i + 1$ 
```

```
     $A[i] \rightleftharpoons A[r]$ 
```

```
    return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

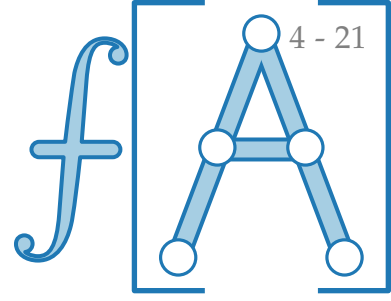
1. Extremfall: m immer erstes Element

$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \\ &= T_{\text{QS}}(1) + 1 + 2 + \dots + n - 2 + n - 1 \\ &= \sum_{i=0}^{n-1} i \in \Theta(n^2) \end{aligned}$$

2. Extremfall: m immer mittleres Element

$$T_{\text{QS}}(n) \approx 2T_{\text{QS}}(n/2) + n - 1$$

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
    [  $m = \text{PARTITION}(A, \ell, r)$ 
      QUICKSORT( $A, \ell, m - 1$ )
      QUICKSORT( $A, m + 1, r$ )
    ]
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
  pivot =  $A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    [ if  $A[j] \leq \text{pivot}$  then
      [  $A[i] \rightleftharpoons A[j]$ 
         $i = i + 1$ 
      ]
    ]
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

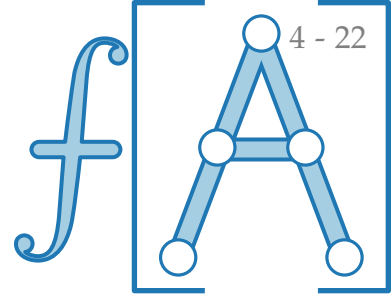
$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \\ &= T_{\text{QS}}(1) + 1 + 2 + \dots + n - 2 + n - 1 \\ &= \sum_{i=0}^{n-1} i \in \Theta(n^2) \end{aligned}$$

2. Extremfall: m immer mittleres Element

$$T_{\text{QS}}(n) \approx 2T_{\text{QS}}(n/2) + n - 1 \in$$

(siehe MERGESORT)

Laufzeit



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
    [  $m = \text{PARTITION}(A, \ell, r)$ 
      QUICKSORT( $A, \ell, m - 1$ )
      QUICKSORT( $A, m + 1, r$ )
    ]
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
  pivot =  $A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    [ if  $A[j] \leq \text{pivot}$  then
      [  $A[i] \rightleftharpoons A[j]$ 
         $i = i + 1$ 
      ]
    ]
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Zähle Anzahl der Vergleiche!

Beob. PARTITION benötigt **immer** $r - \ell$ Vergleiche.

Wovon hängt dann die Laufzeit ab?

$$T_{\text{QS}}(n) = T_{\text{QS}}(m - 1) + T_{\text{QS}}(n - m) + n - 1$$

1. Extremfall: m immer erstes Element

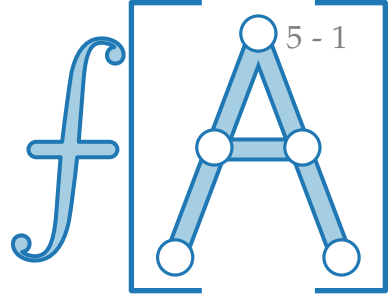
$$\begin{aligned} T_{\text{QS}}(n) &= T_{\text{QS}}(0) + T_{\text{QS}}(n - 1) + n - 1 \\ &= (T_{\text{QS}}(n - 2) + n - 2) + n - 1 \\ &\vdots \\ &= T_{\text{QS}}(1) + 1 + 2 + \dots + n - 2 + n - 1 \\ &= \sum_{i=0}^{n-1} i \in \Theta(n^2) \end{aligned}$$

2. Extremfall: m immer mittleres Element

$$T_{\text{QS}}(n) \approx 2T_{\text{QS}}(n/2) + n - 1 \in \Theta(n \log n)$$

siehe MERGESORT

Pivot Auswahl



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

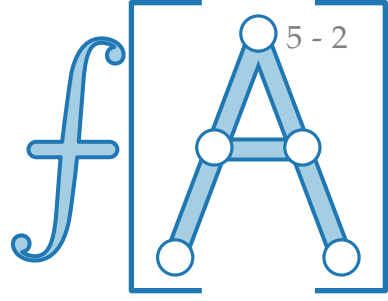
```
       $A[i] \rightleftharpoons A[j]$ 
```

```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Pivot Auswahl



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

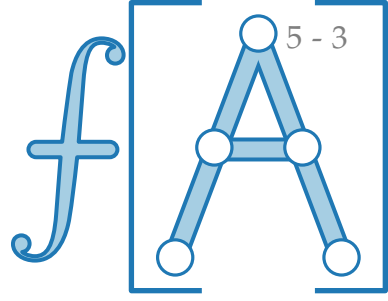
```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Pivot Auswahl

Idee: Steck Zufall in den Algorithmus!



```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```

```
int PARTITION(int[ ]  $A$ , int  $\ell$ , int  $r$ )
```

```
   $pivot = A[r]$ 
```

```
   $i = \ell$ 
```

```
  for  $j = \ell$  to  $r - 1$  do
```

```
    if  $A[j] \leq pivot$  then
```

```
       $A[i] \rightleftharpoons A[j]$ 
```

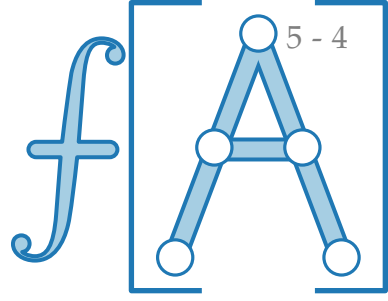
```
       $i = i + 1$ 
```

```
   $A[i] \rightleftharpoons A[r]$ 
```

```
  return  $i$ 
```

Pivot Auswahl

Idee: Steck Zufall in den Algorithmus!



RANDOMIZEDPARTITION(A, ℓ, r)

int PARTITION(int[] A , int ℓ , int r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ to $r - 1$ do

 if $A[j] \leq pivot$ then

$A[i] \rightleftharpoons A[j]$

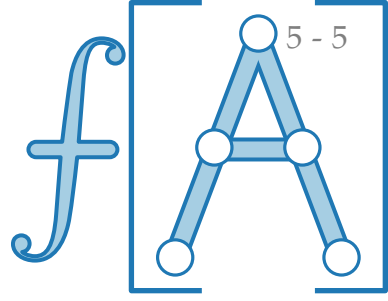
$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Pivot Auswahl

Idee: Steck Zufall in den Algorithmus!



RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$

int PARTITION(int[] A , int ℓ , int r)

$\text{pivot} = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq \text{pivot}$ **then**

$A[i] \rightleftharpoons A[j]$

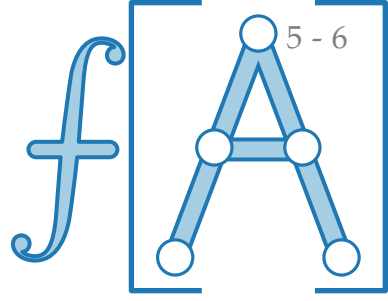
$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Pivot Auswahl

Idee: Steck Zufall in den Algorithmus!



RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$

Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

int PARTITION(int[] A , int ℓ , int r)

$\text{pivot} = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq \text{pivot}$ **then**

$A[i] \rightleftharpoons A[j]$

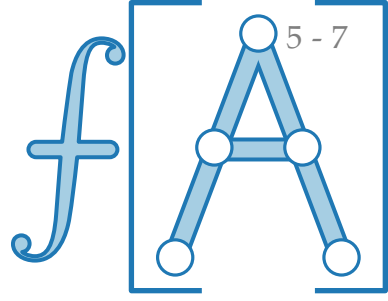
$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Pivot Auswahl

Idee: Steck Zufall in den Algorithmus!



RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$

Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

int PARTITION(int[] A , int ℓ , int r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

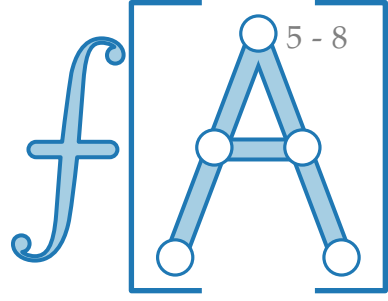
$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Pivot Auswahl

Idee: Steck Zufall in den Algorithmus!



RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

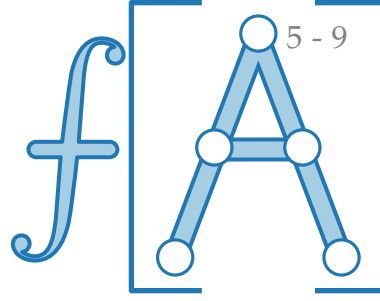
$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

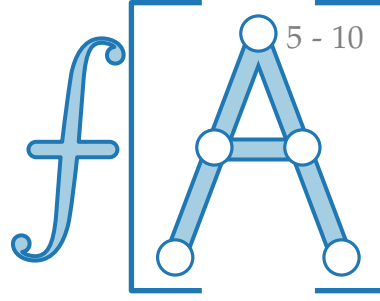
$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!



Demo.

<https://algo.uni-trier.de/demos/sort.html>

Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

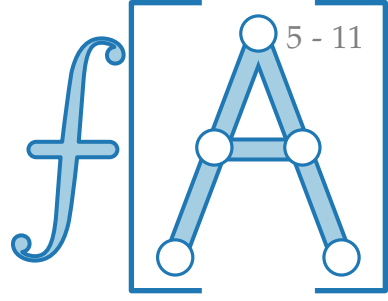
$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

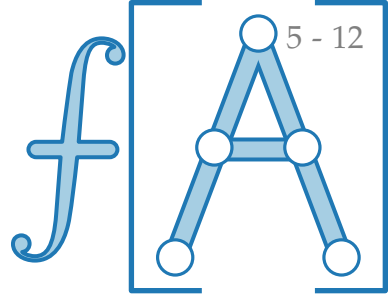
$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

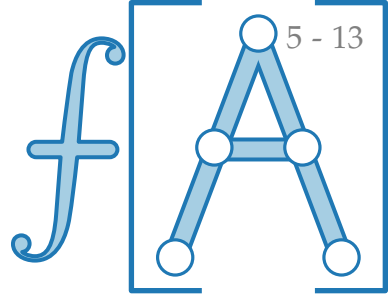
return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

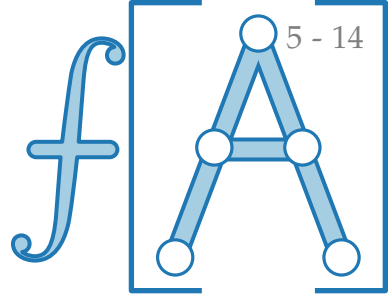
Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

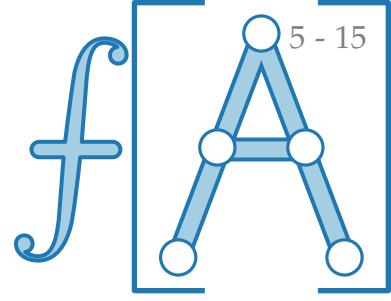
Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.

7	6	5	1	2	3	4
---	---	---	---	---	---	---



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$\text{pivot} = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq \text{pivot}$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

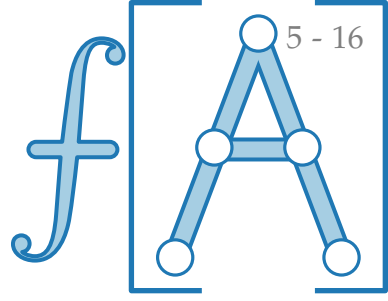
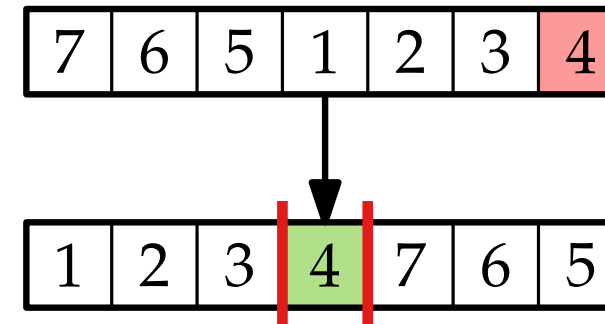
Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$\text{pivot} = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq \text{pivot}$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

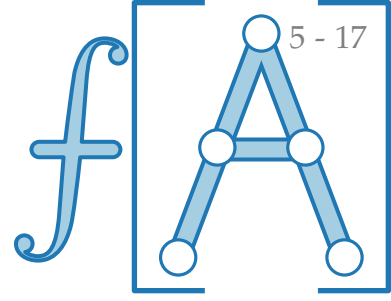
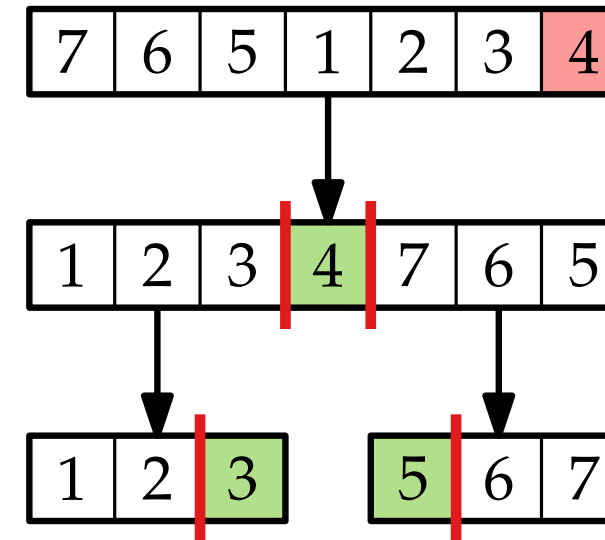
Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$\text{pivot} = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq \text{pivot}$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

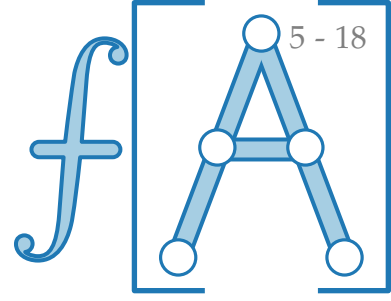
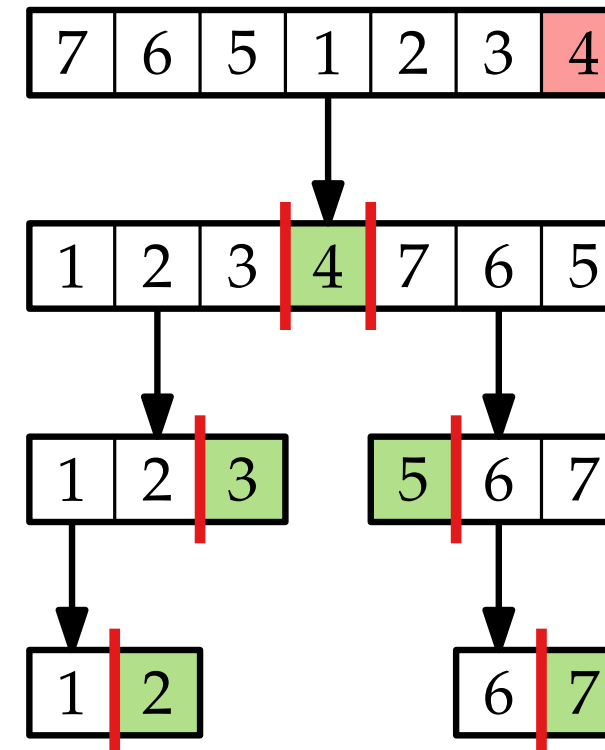
Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$\text{pivot} = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq \text{pivot}$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

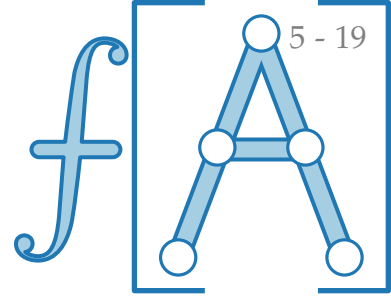
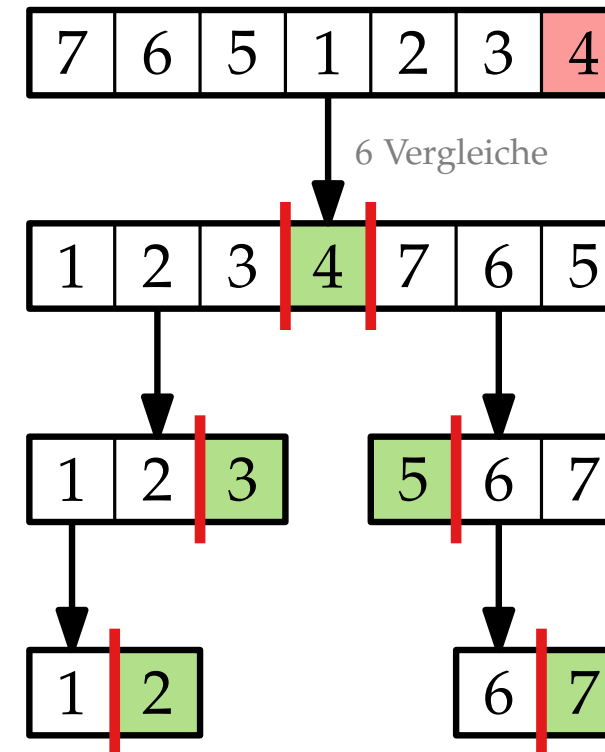
Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$\text{pivot} = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq \text{pivot}$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

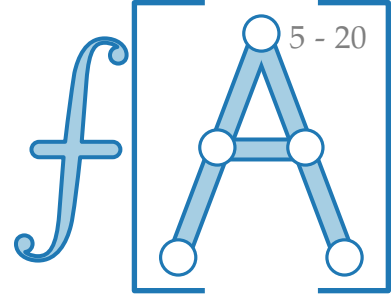
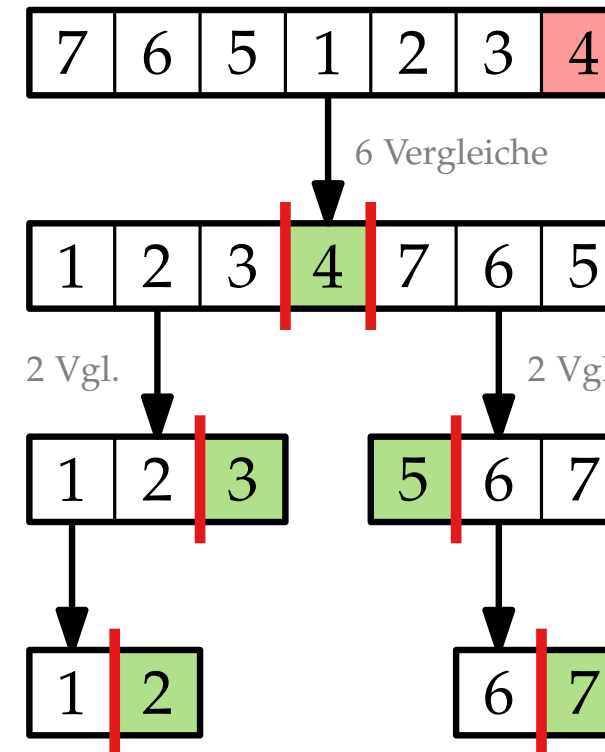
Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return PARTITION(A, ℓ, r)

int PARTITION(int[] A , int ℓ , int r)

$\text{pivot} = A[r]$

$i = \ell$

for $j = \ell$ to $r - 1$ do

 if $A[j] \leq \text{pivot}$ then

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

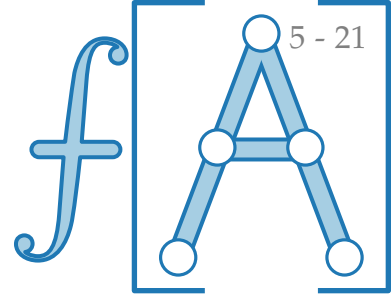
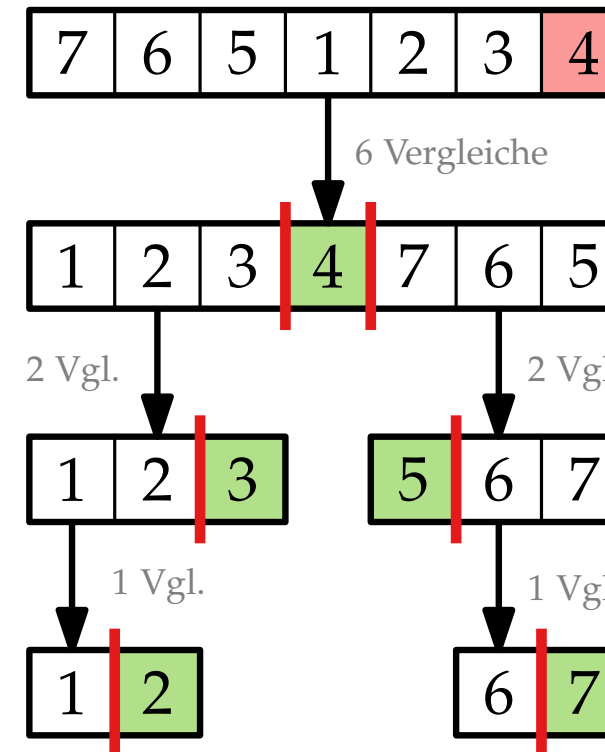
Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

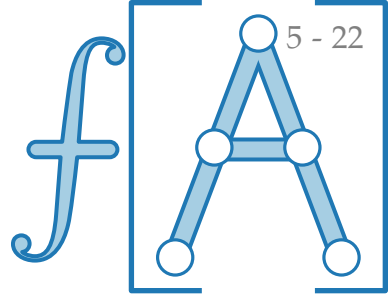
Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.

Definiere Indikator-Zufallsvariable:



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

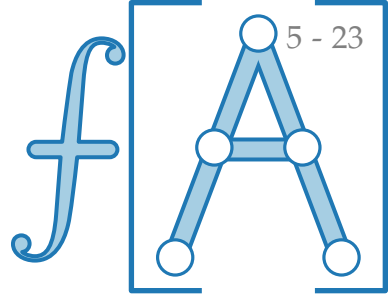
Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.

Definiere Indikator-Zufallsvariable:

$$X_{ij} = \begin{cases} 1, & \text{falls Alg. } z_i \text{ und } z_j \text{ vergleicht,} \\ 0 & \text{sonst.} \end{cases}$$



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

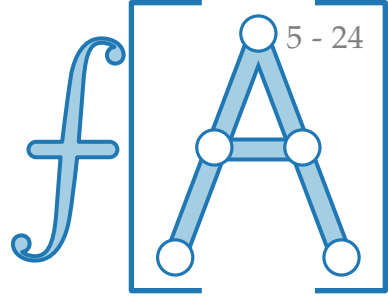
höchstens ein Mal:

wenn eins von beiden *pivot* ist.

Definiere Indikator-Zufallsvariable:

$$X_{ij} = \begin{cases} 1, & \text{falls Alg. } z_i \text{ und } z_j \text{ vergleicht,} \\ 0 & \text{sonst.} \end{cases}$$

Sei X ZV für Gesamtanz. von Vgl.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return PARTITION(A, ℓ, r)

int PARTITION(int[] A , int ℓ , int r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

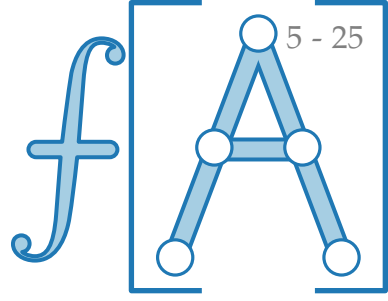
wenn eins von beiden *pivot* ist.

Definiere Indikator-Zufallsvariable:

$$X_{ij} = \begin{cases} 1, & \text{falls Alg. } z_i \text{ und } z_j \text{ vergleicht,} \\ 0 & \text{sonst.} \end{cases}$$

Sei X ZV für Gesamtanz. von Vgl.

Dann gilt $X =$



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

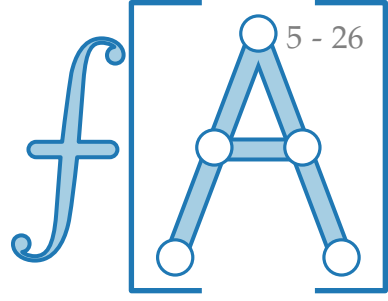
wenn eins von beiden *pivot* ist.

Definiere Indikator-Zufallsvariable:

$$X_{ij} = \begin{cases} 1, & \text{falls Alg. } z_i \text{ und } z_j \text{ vergleicht,} \\ 0 & \text{sonst.} \end{cases}$$

Sei X ZV für Gesamtanz. von Vgl.

Dann gilt $X = \sum_{1 \leq i < j \leq n} X_{ij}$.



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return **PARTITION**(A, ℓ, r)

int **PARTITION**(**int**[] A , **int** ℓ , **int** r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.

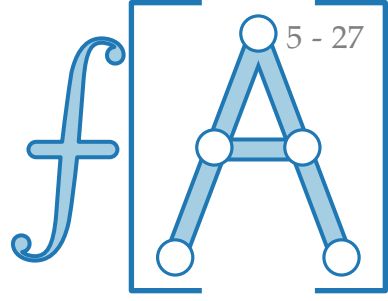
Definiere Indikator-Zufallsvariable:

$$X_{ij} = \begin{cases} 1, & \text{falls Alg. } z_i \text{ und } z_j \text{ vergleicht,} \\ 0 & \text{sonst.} \end{cases}$$

Sei X ZV für Gesamtanz. von Vgl.

Dann gilt $X = \sum_{1 \leq i < j \leq n} X_{ij}$.

$\Rightarrow E[X] =$



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return PARTITION(A, ℓ, r)

int PARTITION(int[] A , int ℓ , int r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.

Definiere Indikator-Zufallsvariable:

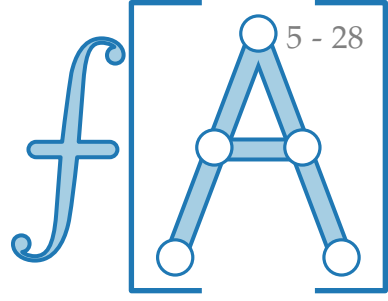
$$X_{ij} = \begin{cases} 1, & \text{falls Alg. } z_i \text{ und } z_j \text{ vergleicht,} \\ 0 & \text{sonst.} \end{cases}$$

Sei X ZV für Gesamtanz. von Vgl.

Dann gilt $X = \sum_{1 \leq i < j \leq n} X_{ij}$.

$\Rightarrow E[X] =$

Linearität des Erwartungswerts!



Pivot Auswahl

RANDOMIZEDPARTITION(A, ℓ, r)

$k = \text{RANDOM}(\ell, r)$ Liefert Zufallszahl
 $\in \{\ell, \dots, r\}$.

$A[k] \rightleftharpoons A[r]$

return PARTITION(A, ℓ, r)

int PARTITION(int[] A , int ℓ , int r)

$pivot = A[r]$

$i = \ell$

for $j = \ell$ **to** $r - 1$ **do**

if $A[j] \leq pivot$ **then**

$A[i] \rightleftharpoons A[j]$

$i = i + 1$

$A[i] \rightleftharpoons A[r]$

return i

Idee: Steck Zufall in den Algorithmus!

Seien $z_1, z_2, \dots, z_n$ die Elemente von A in sortierter Reihenfolge.

Wann vergleicht Alg. z_i und z_j ?

höchstens ein Mal:

wenn eins von beiden *pivot* ist.

Definiere Indikator-Zufallsvariable:

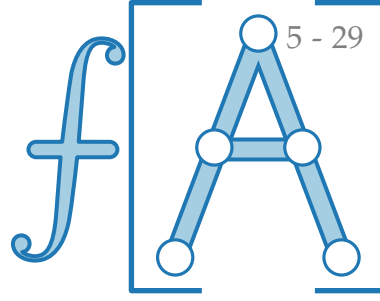
$$X_{ij} = \begin{cases} 1, & \text{falls Alg. } z_i \text{ und } z_j \text{ vergleicht,} \\ 0 & \text{sonst.} \end{cases}$$

Sei X ZV für Gesamtanz. von Vgl.

Dann gilt $X = \sum_{1 \leq i < j \leq n} X_{ij}$.

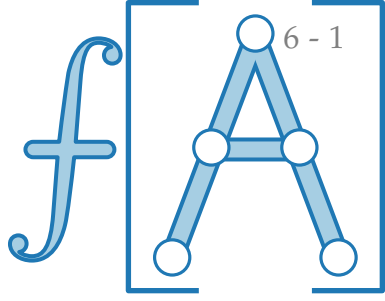
$$\Rightarrow E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}]$$

Linearität des Erwartungswerts!



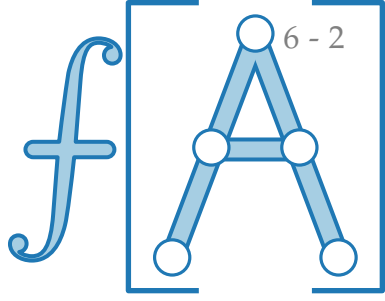
First come, first serve

$$E[X_{ij}] =$$



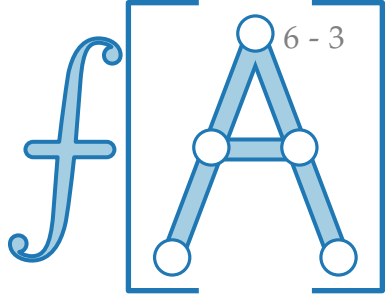
First come, first serve

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j]$$



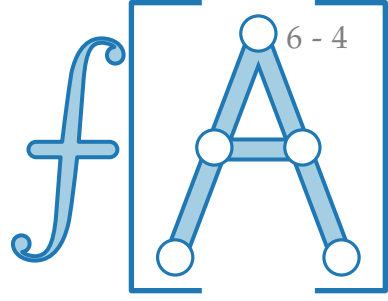
First come, first serve

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] =$$



First come, first serve

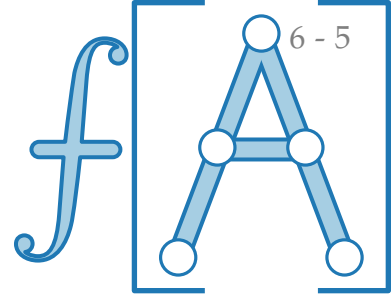
$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \boxed{?}$$



First come, first serve

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \boxed{?}$$

Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.



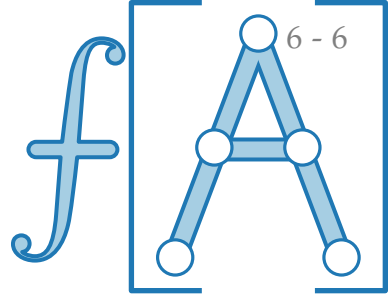
First come, first serve

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] =$$

?

Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.



First come, first serve

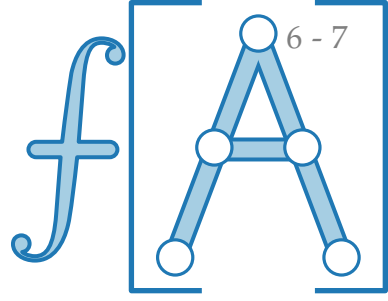
$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] =$$

?

Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.

Es gilt: Alg. vergleicht z_i und $z_j \iff$



First come, first serve

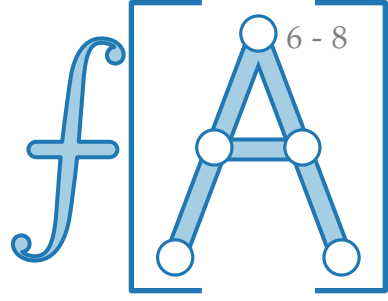
$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] =$$

?

Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.

Es gilt: Alg. vergleicht z_i und $z_j \iff z^* = z_i$ oder $z^* = z_j$.



First come, first serve

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] =$$

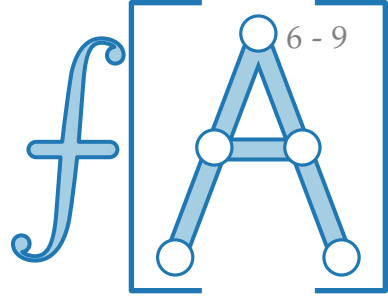
?

Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.

Es gilt: Alg. vergleicht z_i und $z_j \iff z^* = z_i$ oder $z^* = z_j$.

$$\Rightarrow \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] =$$



First come, first serve

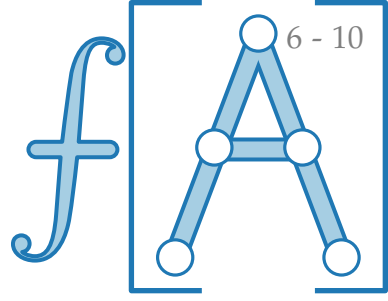
$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \boxed{?}$$

Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

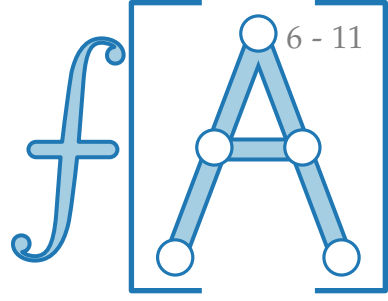
Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.

Es gilt: Alg. vergleicht z_i und $z_j \iff z^* = z_i$ oder $z^* = z_j$.

$$\Rightarrow \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \Pr[z^* = z_i \text{ oder } z^* = z_j]$$



First come, first serve



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \boxed{?}$$

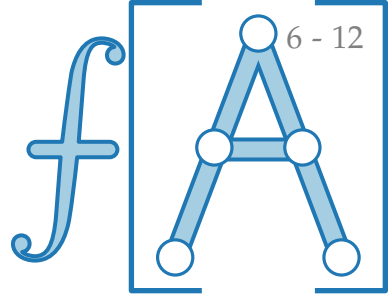
Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.

Es gilt: Alg. vergleicht z_i und $z_j \iff z^* = z_i$ oder $z^* = z_j$.

$$\begin{aligned} \Rightarrow \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] &= \Pr[z^* = z_i \text{ oder } z^* = z_j] \\ &\stackrel{i \neq j}{=} \Pr[z^* = z_i] + \Pr[z^* = z_j] \end{aligned}$$

First come, first serve



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \boxed{?}$$

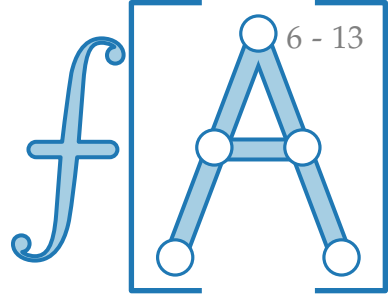
Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.

Es gilt: Alg. vergleicht z_i und $z_j \iff z^* = z_i$ oder $z^* = z_j$.

$$\begin{aligned} \Rightarrow \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] &= \Pr[z^* = z_i \text{ oder } z^* = z_j] \\ &\stackrel{i \neq j}{=} \Pr[z^* = z_i] + \Pr[z^* = z_j] \\ &= \frac{1}{|Z_{ij}|} + \frac{1}{|Z_{ij}|} \end{aligned}$$

First come, first serve



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \boxed{?}$$

Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.

Es gilt: Alg. vergleicht z_i und $z_j \iff z^* = z_i$ oder $z^* = z_j$.

$$\begin{aligned} \Rightarrow \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] &= \Pr[z^* = z_i \text{ oder } z^* = z_j] \\ &\stackrel{i \neq j}{=} \Pr[z^* = z_i] + \Pr[z^* = z_j] \\ &= \frac{1}{|Z_{ij}|} + \frac{1}{|Z_{ij}|} \\ &= \frac{2}{j - i + 1} \end{aligned}$$

First come, first serve

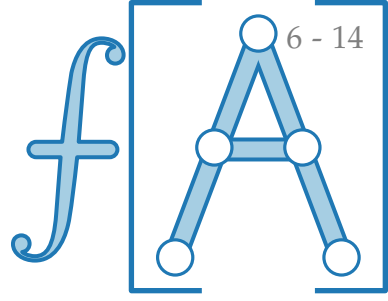
$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \boxed{\frac{2}{j-i+1}}$$

Betrachte die Menge $Z_{ij} := \{z_i, z_{i+1}, \dots, z_j\}$.

Sei z^* die erste Zahl in Z_{ij} , die *pivot* wird.

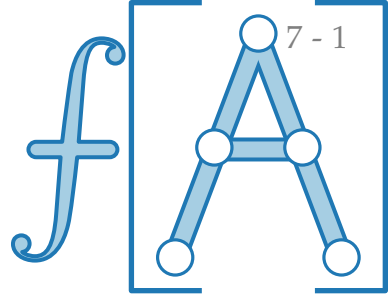
Es gilt: Alg. vergleicht z_i und $z_j \iff z^* = z_i$ oder $z^* = z_j$.

$$\begin{aligned} \Rightarrow \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] &= \Pr[z^* = z_i \text{ oder } z^* = z_j] \\ &\stackrel{i \neq j}{=} \Pr[z^* = z_i] + \Pr[z^* = z_j] \\ &= \frac{1}{|Z_{ij}|} + \frac{1}{|Z_{ij}|} \\ &= \frac{2}{j-i+1} \end{aligned}$$



Rechnereien

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

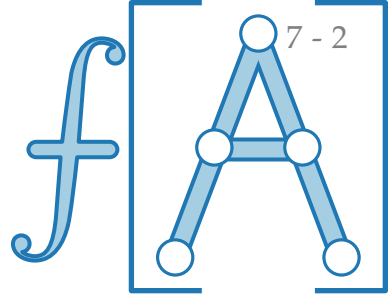


Rechnereien

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

$$E[X] =$$

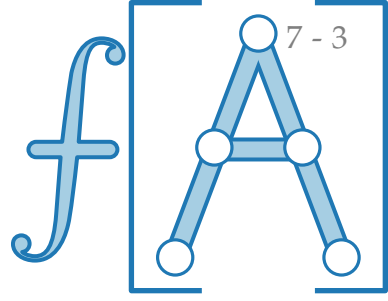


Rechnereien

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}]$$

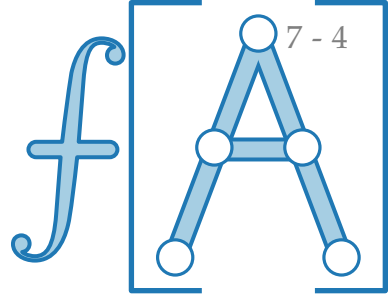


Rechnereien

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

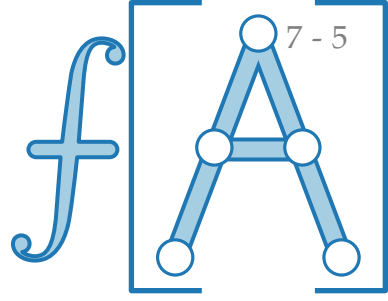


Rechnereien

$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

$$\begin{aligned} E[X] &= \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1} \\ &= \sum_{i=1}^{n-1} \end{aligned}$$

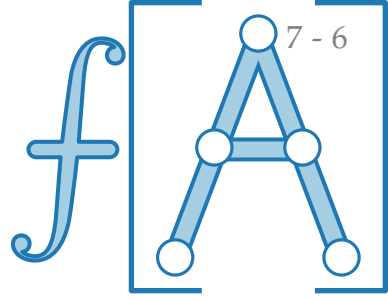


Rechnereien

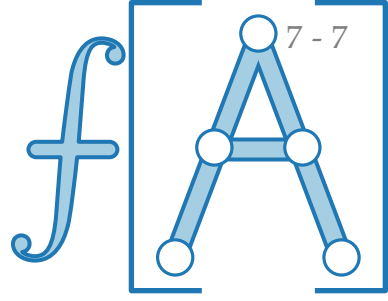
$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

$$\begin{aligned} E[X] &= \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \end{aligned}$$



Rechnereien

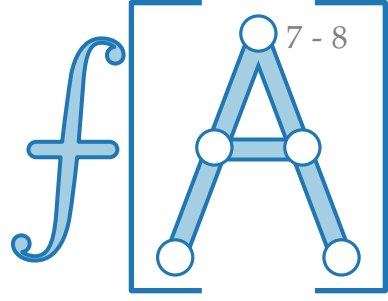


$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j-i+1}$$

Wir wissen:

$$\begin{aligned} E[X] &= \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \end{aligned}$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j-i+1}$$

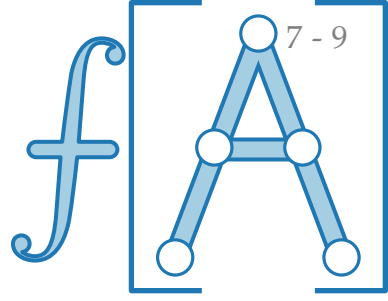
Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

Trick: ersetze $j-i$ durch k

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j-i+1}$$

Wir wissen:

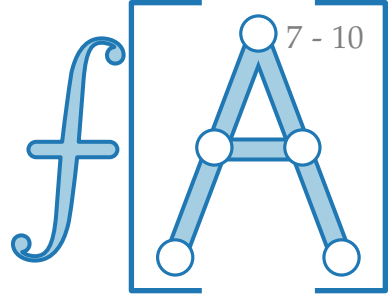
$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

Trick: ersetze $j-i$ durch k

$$= \sum_{i=1}^{n-1}$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

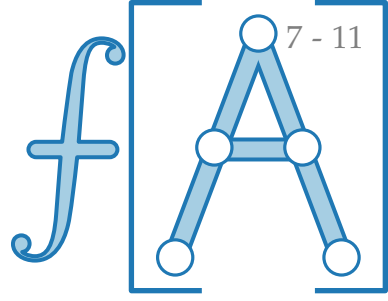
$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1}$$

Trick: ersetze $j - i$ durch k

$$= \sum_{i=1}^{n-1} \sum \frac{2}{k + 1}$$

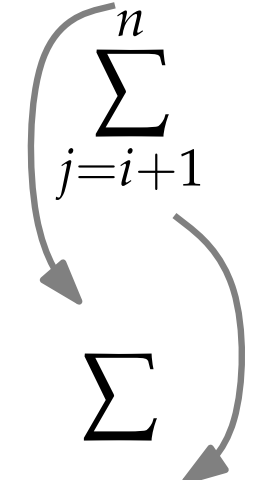
Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

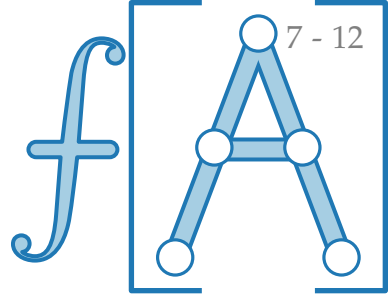
Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1}$$
$$= \sum_{i=1}^{n-1} \sum \frac{2}{k + 1}$$


Trick: ersetze $j - i$ durch k

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

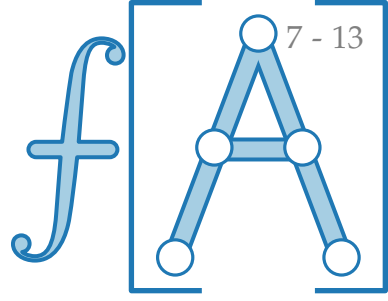
$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \left(\sum_{j=i+1}^n \frac{2}{j - i + 1} \right)$$

Trick: ersetze $j - i$ durch k

$$= \sum_{i=1}^{n-1} \left(\sum_{k=0}^{n-i} \frac{2}{k + 1} \right)$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

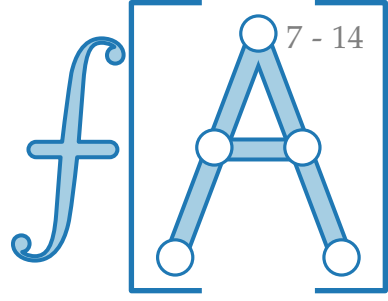
$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1}$$

Trick: ersetze $j - i$ durch k

$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k + 1}$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j-i+1}$$

Wir wissen:

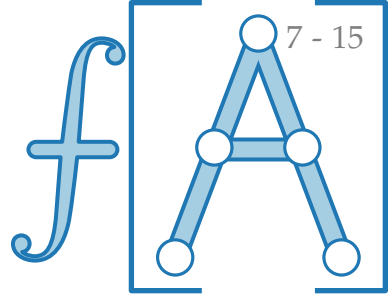
$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

Trick: ersetze $j-i$ durch k

$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

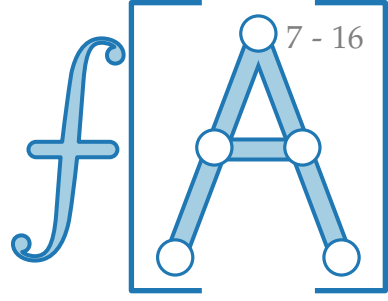
$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1}$$

Trick: ersetze $j - i$ durch k

$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

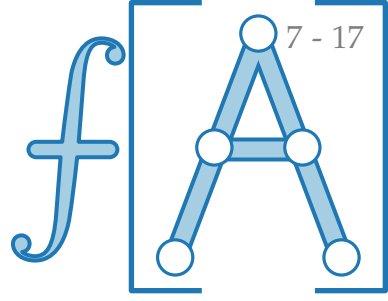
Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} \quad \text{Trick: ersetze } j - i \text{ durch } k$$

$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} <$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

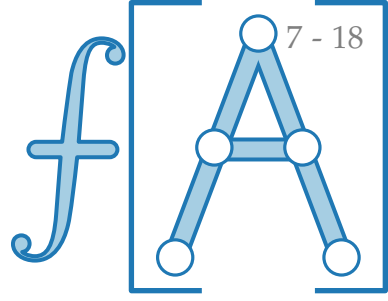
Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} \quad \text{Trick: ersetze } j - i \text{ durch } k$$

$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k}$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j-i+1}$$

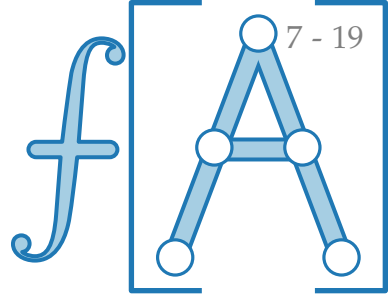
Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \quad \text{Trick: ersetze } j-i \text{ durch } k$$

$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \in \mathcal{O}(\quad)$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1}$$

Trick: ersetze $j - i$ durch k

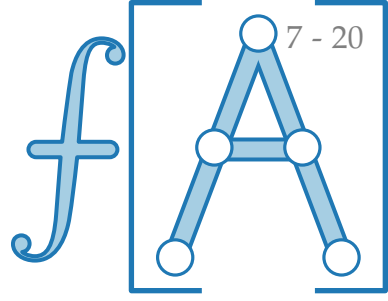
$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$$

$$< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \in \mathcal{O}()$$

harmonische Reihe

$$\sum_{i=1}^n \frac{1}{i} \approx \ln n$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j - i + 1}$$

Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1}$$

Trick: ersetze $j - i$ durch k

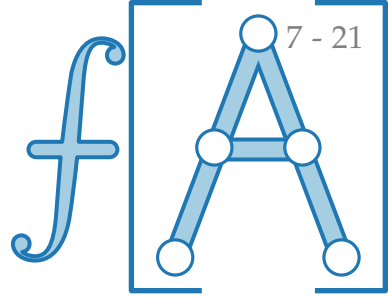
$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$$

$$< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \in \mathcal{O}(n \log n)$$

harmonische Reihe

$$\sum_{i=1}^n \frac{1}{i} \approx \ln n$$

Rechnereien



$$E[X_{ij}] = \Pr[\text{Alg. vergleicht } z_i \text{ und } z_j] = \frac{2}{j-i+1}$$

Wir wissen:

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1}$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

Trick: ersetze $j-i$ durch k

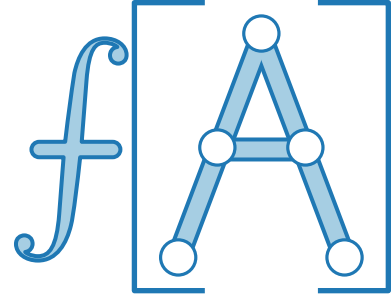
$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$$

harmonische Reihe

$$\sum_{i=1}^n \frac{1}{i} \approx \ln n$$

$$< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \in \mathcal{O}(n \log n)$$

Satz. RANDOMIZEDQUICKSORT sortiert n Zahlen in $\mathcal{O}(n \log n)$ erwarteter Zeit.



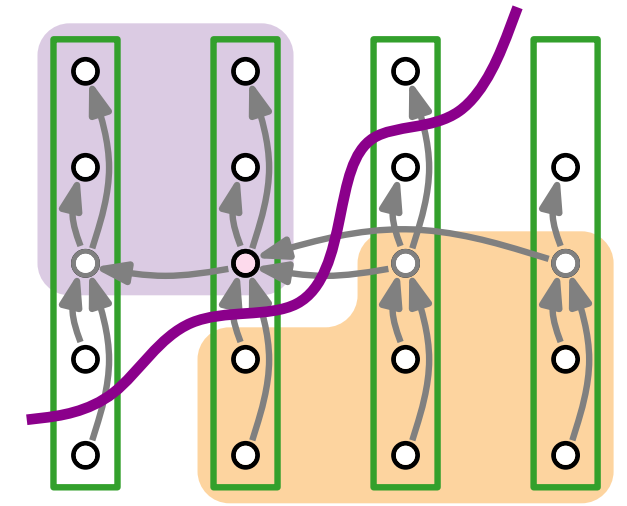
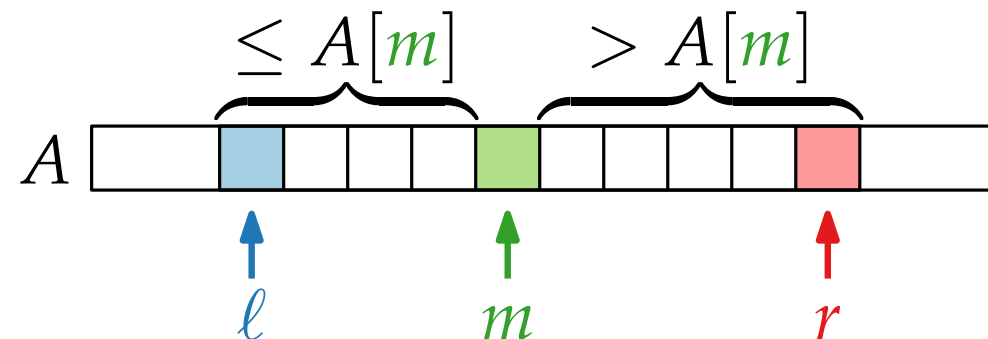
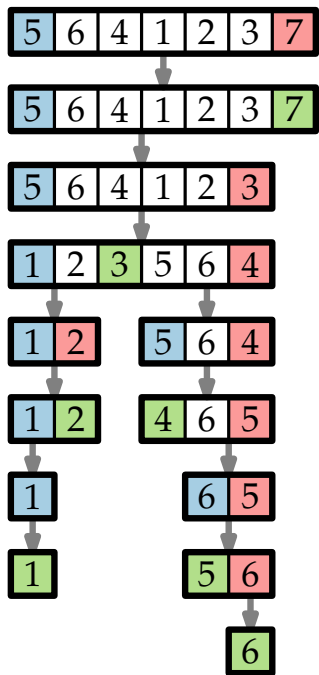
Fortgeschrittene Algorithmen

4. Vorlesung

Randomisierte Algorithmen II: Das Auswahlproblem

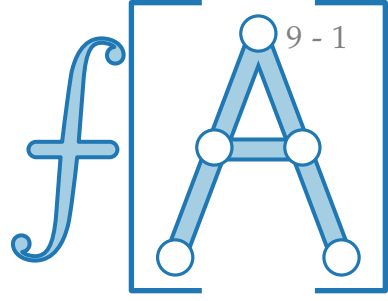
Teil II:

Das Auswahlproblem:
randomisierter Algorithmus



Das Auswahlproblem

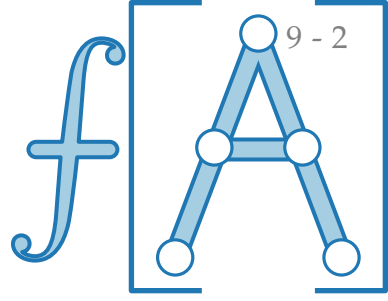
Problem. AUSWAHLPROBLEM



Das Auswahlproblem

Problem. AUSWAHLPROBLEM

Geg. Eine Reihe von n Messwerten $A[1 \dots n]$

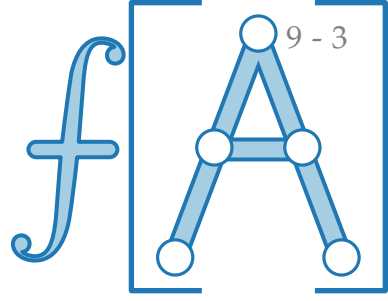


Das Auswahlproblem

Problem. AUSWAHLPROBLEM

Geg. Eine Reihe von n Messwerten $A[1 \dots n]$

Ges. Das i -kleinste Element



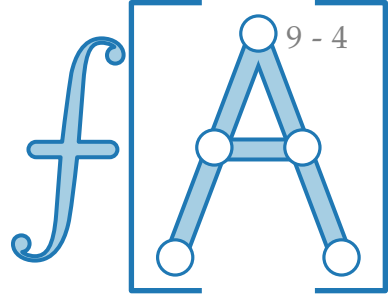
Das Auswahlproblem

Problem. AUSWAHLPROBLEM

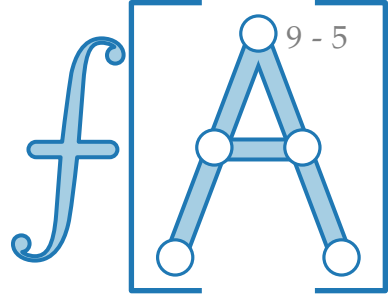
Geg. Eine Reihe von n Messwerten $A[1 \dots n]$

Ges. Das i -kleinste Element

Lösung.



Das Auswahlproblem



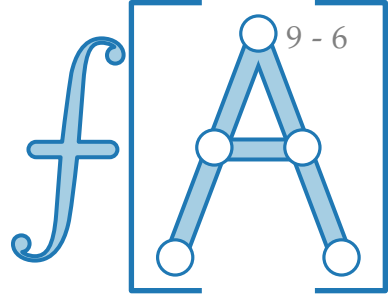
Problem. AUSWAHLPROBLEM

Geg. Eine Reihe von n Messwerten $A[1 \dots n]$

Ges. Das i -kleinste Element

Lösung. Sortiere und gib $A[i]$ zurück!

Das Auswahlproblem



Problem. AUSWAHLPROBLEM

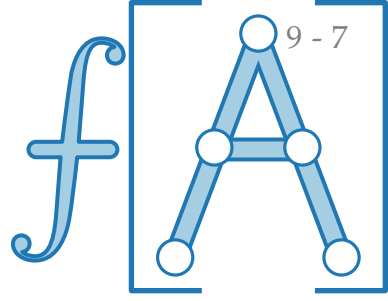
Geg. Eine Reihe von n Messwerten $A[1 \dots n]$

Ges. Das i -kleinste Element

Lösung. Sortiere und gib $A[i]$ zurück!

Worst-Case-Laufzeit:

Das Auswahlproblem



Problem. AUSWAHLPROBLEM

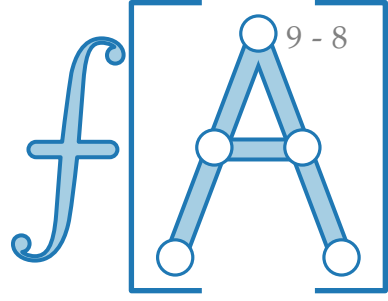
Geg. Eine Reihe von n Messwerten $A[1 \dots n]$

Ges. Das i -kleinste Element

Lösung. Sortiere und gib $A[i]$ zurück!

Worst-Case-Laufzeit: $\Theta(n \log n)$

Das Auswahlproblem



Problem. AUSWAHLPROBLEM

Geg. Eine Reihe von n Messwerten $A[1 \dots n]$

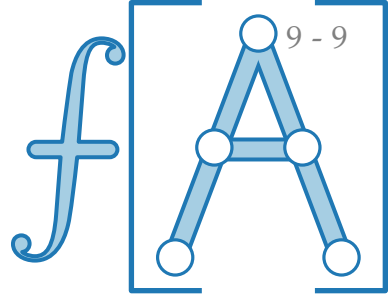
Ges. Das i -kleinste Element

Lösung. Sortiere und gib $A[i]$ zurück!

Worst-Case-Laufzeit: $\Theta(n \log n)$

wenn man nichts über die
Verteilung der Zahlen weiß

Das Auswahlproblem



Problem. AUSWAHLPROBLEM

Geg. Eine Reihe von n Messwerten $A[1 \dots n]$

Ges. Das i -kleinste Element

Lösung. Sortiere und gib $A[i]$ zurück!

Worst-Case-Laufzeit: $\Theta(n \log n)$

wenn man nichts über die
Verteilung der Zahlen weiß

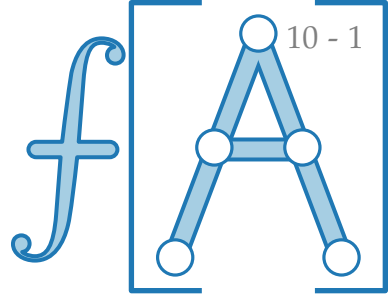
Geht das besser?

Spezialfälle

$$i = \lfloor \frac{n+1}{2} \rfloor:$$

$$i = 1:$$

$$i = n:$$

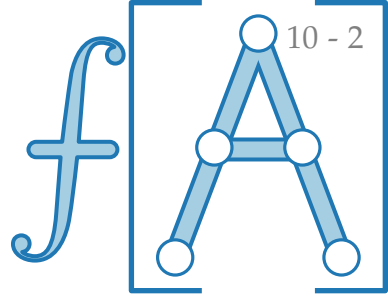


Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum



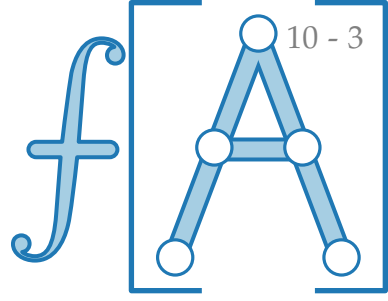
Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

```
FINDMIN(int[ ] A)
```



Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

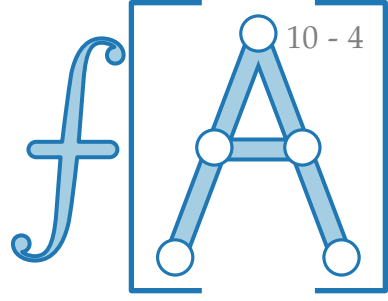
$i = 1$: Minimum

$i = n$: Maximum

```
FINDMIN(int[ ] A)
```

```
     $m = A[1]$ 
```

```
    return  $m$ 
```



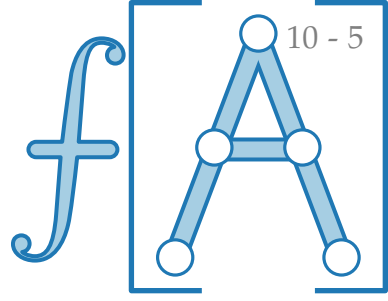
Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

```
FINDMIN(int[ ] A)
   $m = A[1]$ 
  for  $i = 2, \dots, n$  do
    |
  return  $m$ 
```



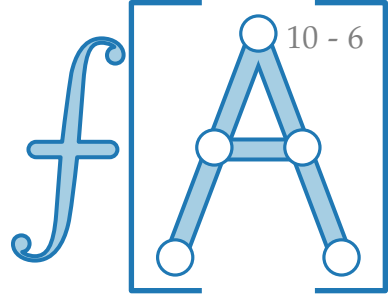
Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

```
FINDMIN(int[ ] A)
   $m = A[1]$ 
  for  $i = 2, \dots, n$  do
    if  $A[i] < m$  then  $m = A[i]$ 
  return  $m$ 
```



Spezialfälle

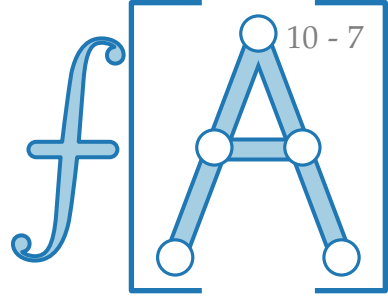
$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

```
FINDMIN(int[ ] A)
   $m = A[1]$ 
  for  $i = 2, \dots, n$  do
    if  $A[i] < m$  then  $m = A[i]$ 
  return  $m$ 
```

Anzahl Vergleiche =



Spezialfälle

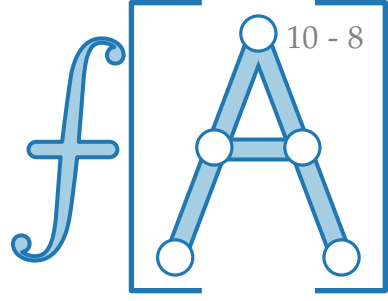
$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

```
FINDMIN(int[ ] A)
   $m = A[1]$ 
  for  $i = 2, \dots, n$  do
    if  $A[i] < m$  then  $m = A[i]$ 
  return  $m$ 
```

Anzahl Vergleiche = $n - 1$



Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

} Laufzeit $\Theta(n)$

```
FINDMIN(int[ ] A)
```

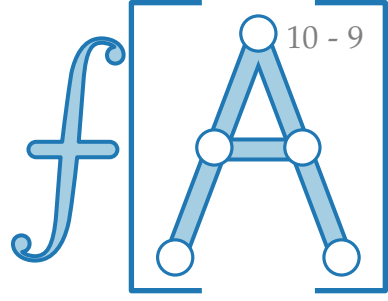
```
   $m = A[1]$ 
```

```
  for  $i = 2, \dots, n$  do
```

```
    if  $A[i] < m$  then  $m = A[i]$ 
```

```
  return  $m$ 
```

Anzahl Vergleiche = $n - 1$



Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

} Laufzeit $\Theta(n)$

```
FINDMIN(int[ ] A)
```

```
   $m = A[1]$ 
```

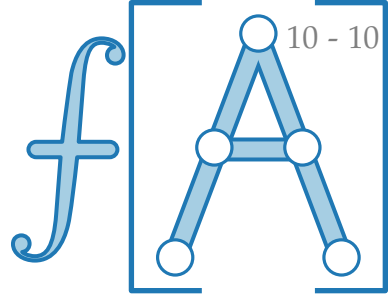
```
  for  $i = 2, \dots, n$  do
```

```
    if  $A[i] < m$  then  $m = A[i]$ 
```

```
  return  $m$ 
```

Ist das optimal?

Anzahl Vergleiche = $n - 1$



Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

} Laufzeit $\Theta(n)$

```
FINDMIN(int[ ] A)
```

```
   $m = A[1]$ 
```

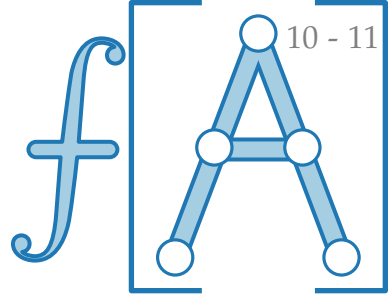
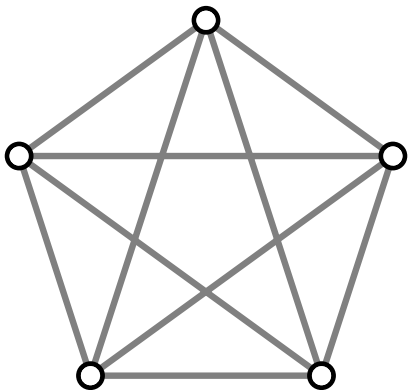
```
  for  $i = 2, \dots, n$  do
```

```
    if  $A[i] < m$  then  $m = A[i]$ 
```

```
  return  $m$ 
```

Anzahl Vergleiche = $n - 1$

Ist das optimal? Betrachte ein K.O.-Turnier.



Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

} Laufzeit $\Theta(n)$

```
FINDMIN(int[ ] A)
```

```
   $m = A[1]$ 
```

```
  for  $i = 2, \dots, n$  do
```

```
    if  $A[i] < m$  then  $m = A[i]$ 
```

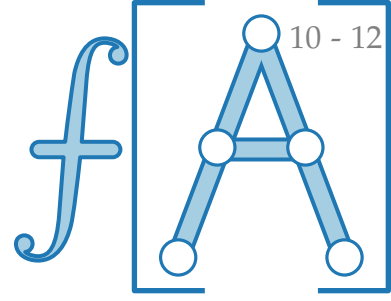
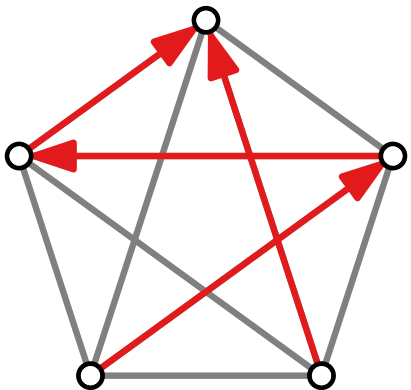
```
  return  $m$ 
```

Anzahl Vergleiche = $n - 1$

Ist das optimal?

Betrachte ein K.O.-Turnier.

Bis ein Gewinner feststeht, muss *jeder* – außer dem Gewinner – mindestens einmal verlieren.



Spezialfälle

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum

$i = n$: Maximum

} Laufzeit $\Theta(n)$

```
FINDMIN(int[ ] A)
```

```
   $m = A[1]$ 
```

```
  for  $i = 2, \dots, n$  do
```

```
    if  $A[i] < m$  then  $m = A[i]$ 
```

```
  return  $m$ 
```

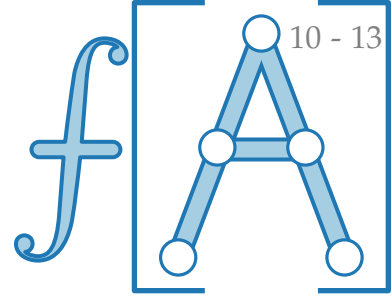
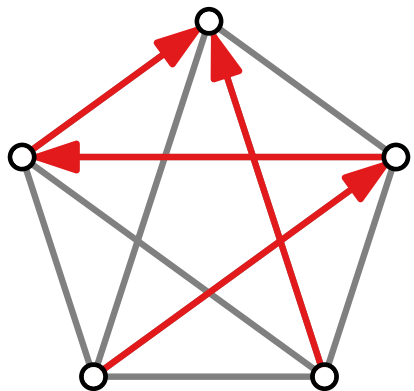
Anzahl Vergleiche = $n - 1$

Ist das optimal?

Betrachte ein K.O.-Turnier.

Bis ein Gewinner feststeht, muss *jeder* – außer dem Gewinner – mindestens einmal verlieren.

Also sind $n - 1$ Vergleiche optimal.



Spezialfälle

Geht das auch in linearer Zeit?

$i = \lfloor \frac{n+1}{2} \rfloor$: Median

$i = 1$: Minimum
 $i = n$: Maximum

Laufzeit $\Theta(n)$

```
FINDMIN(int[ ] A)
```

```
  m = A[1]
```

```
  for i = 2, ..., n do
```

```
    if A[i] < m then m = A[i]
```

```
  return m
```

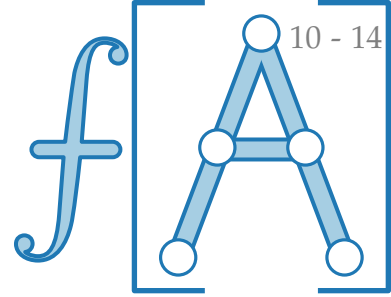
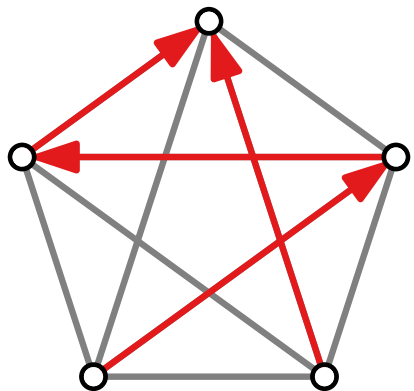
Anzahl Vergleiche = $n - 1$

Ist das optimal?

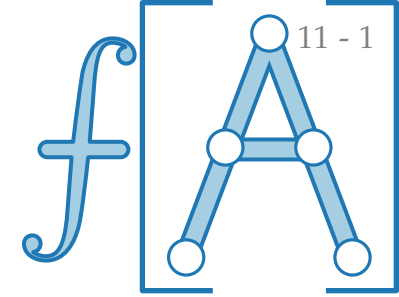
Betrachte ein K.O.-Turnier.

Bis ein Gewinner feststeht, muss *jeder* – außer dem Gewinner – mindestens einmal verlieren.

Also sind $n - 1$ Vergleiche optimal.



Auswahl per Teile & Herrsche



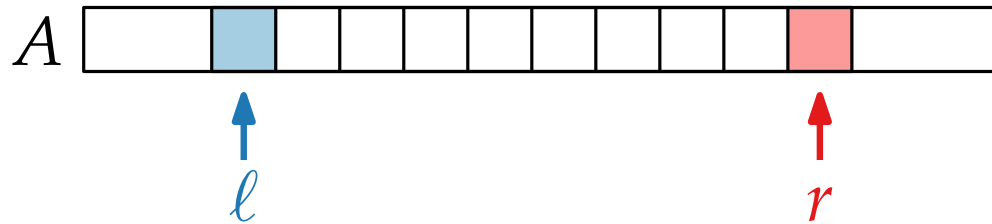
```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

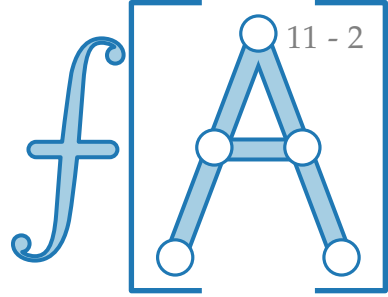
```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```



Auswahl per Teile & Herrsche



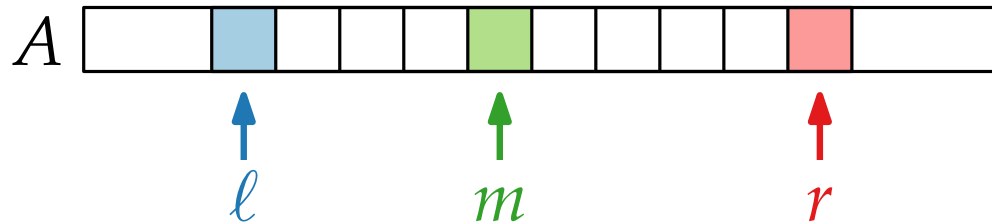
```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

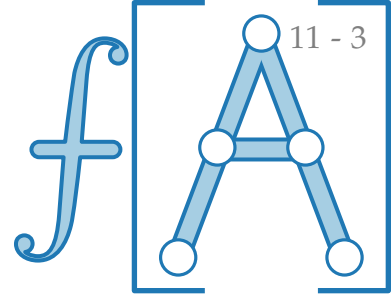
```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```



Auswahl per Teile & Herrsche



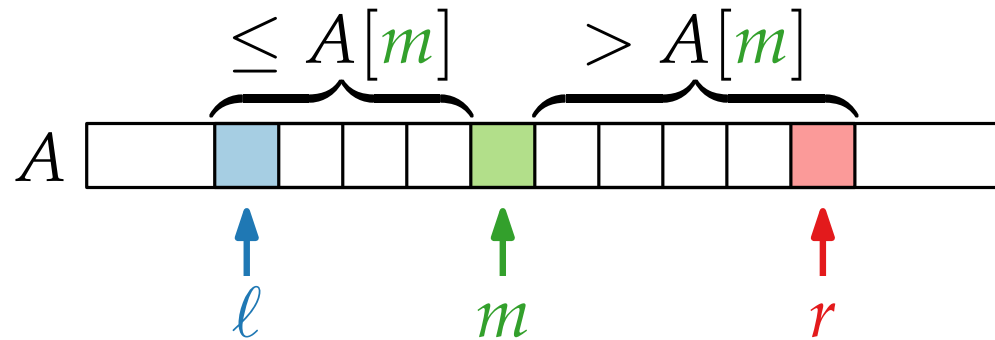
```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

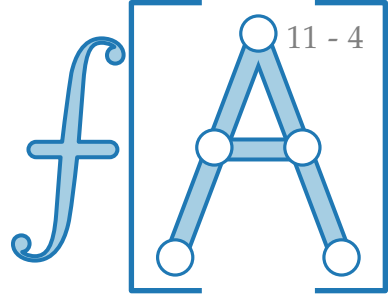
```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```



Auswahl per Teile & Herrsche



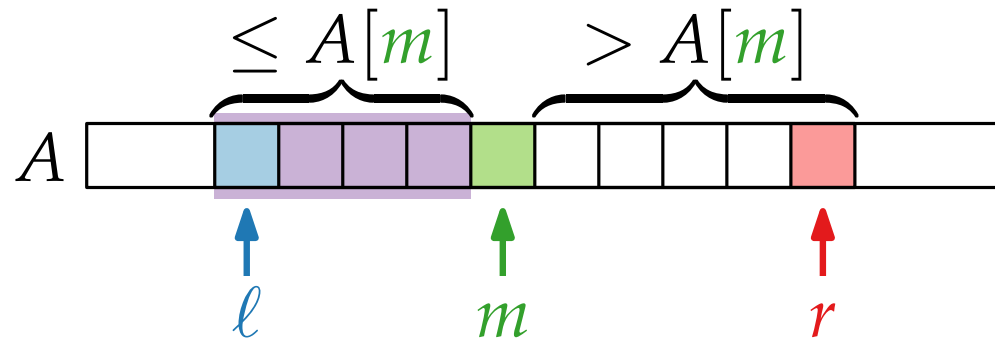
```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

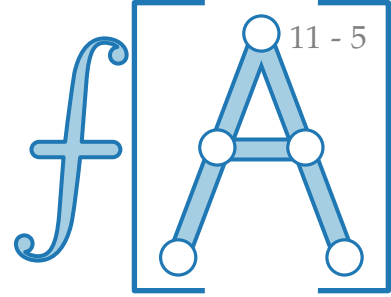
```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```



Auswahl per Teile & Herrsche



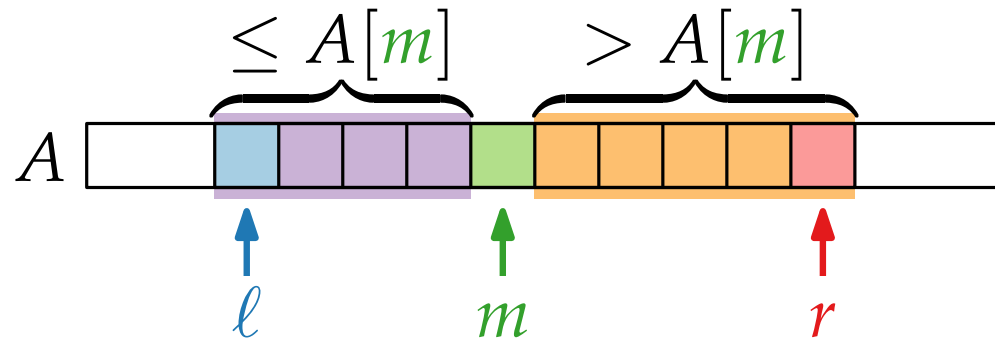
```
QUICKSORT( $A, \ell = 1, r = A.length$ )
```

```
  if  $\ell < r$  then
```

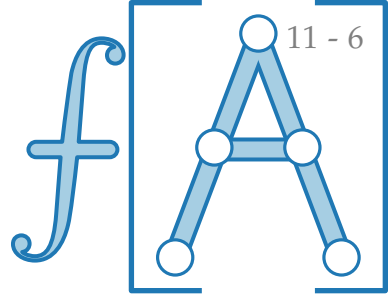
```
     $m = \text{PARTITION}(A, \ell, r)$ 
```

```
    QUICKSORT( $A, \ell, m - 1$ )
```

```
    QUICKSORT( $A, m + 1, r$ )
```



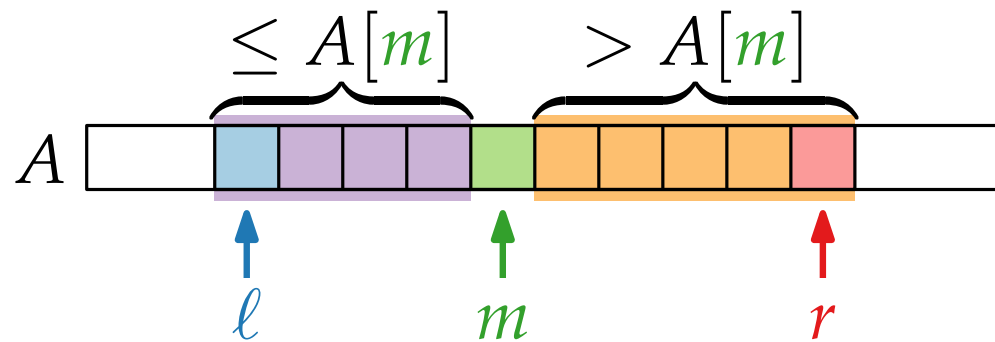
Auswahl per Teile & Herrsche



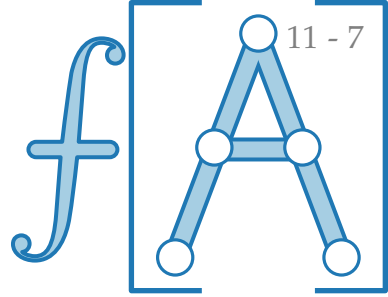
RANDOMIZED
QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ **then**

$m = \text{PARTITION}(A, \ell, r)$
 QUICKSORT($A, \ell, m - 1$)
 QUICKSORT($A, m + 1, r$)



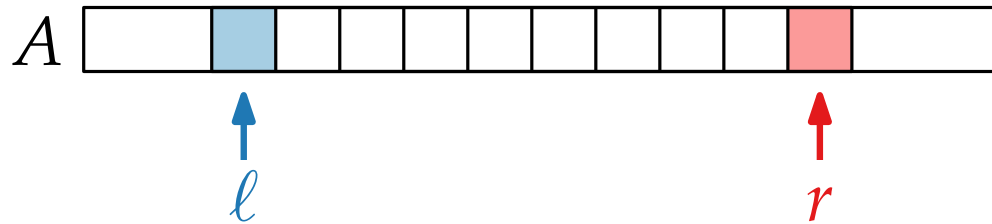
Auswahl per Teile & Herrsche



RANDOMIZED
QUICKSORT($A, \ell = 1, r = A.length$)

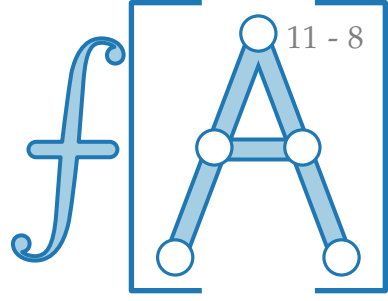
if $\ell < r$ then

RANDOMIZED
 $m = \text{PARTITION}(A, \ell, r)$
 QUICKSORT($A, \ell, m - 1$)
 QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

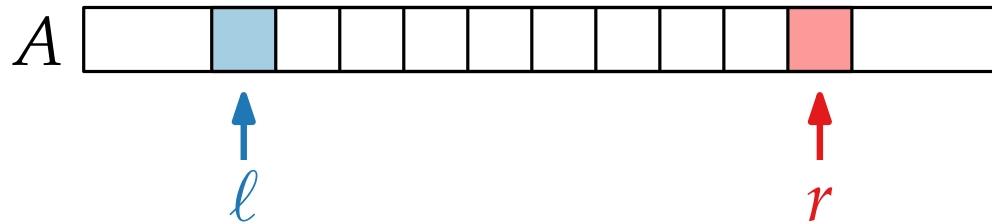
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

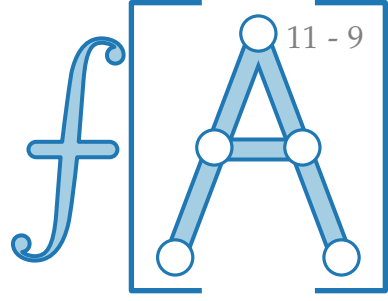
QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

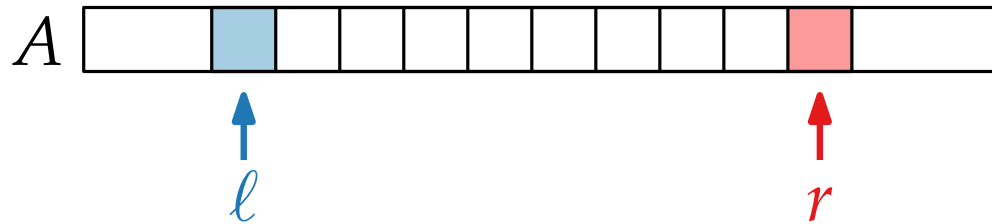
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

QUICKSORT($A, m + 1, r$)

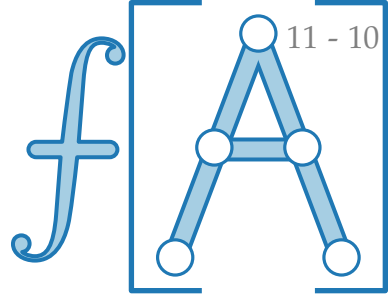


Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

Auswahl per Teile & Herrsche

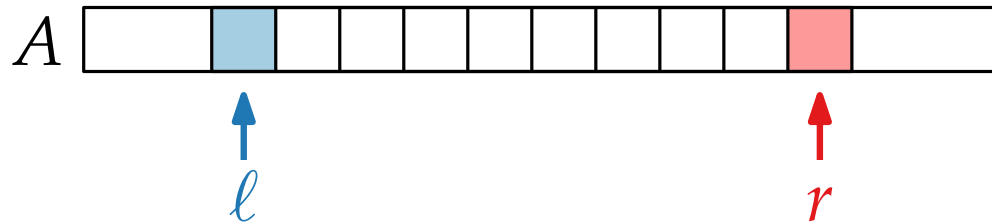


RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$
 QUICKSORT($A, \ell, m - 1$)
 QUICKSORT($A, m + 1, r$)



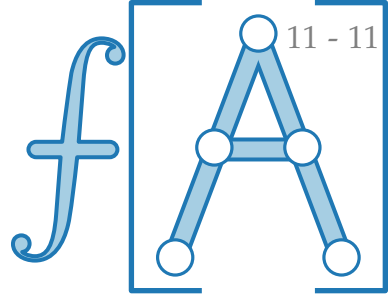
Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

Auswahl per Teile & Herrsche

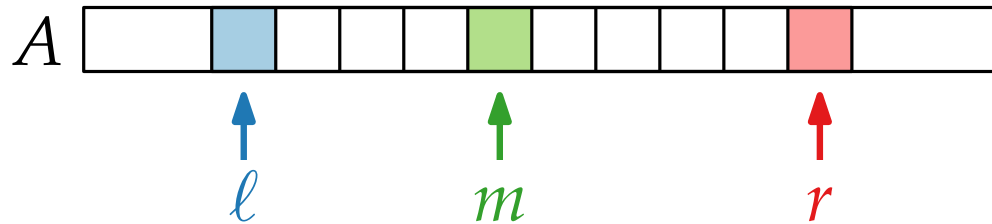


RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$
 QUICKSORT($A, \ell, m - 1$)
 QUICKSORT($A, m + 1, r$)



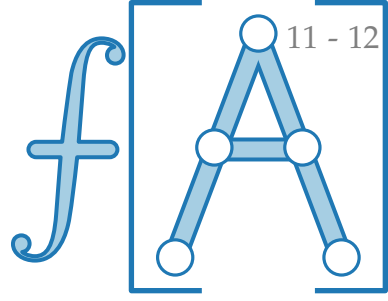
Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

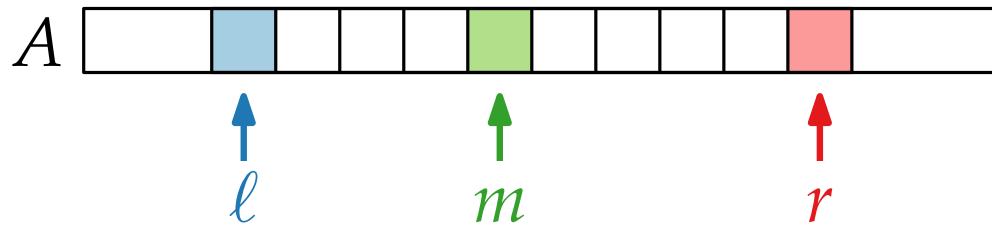
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

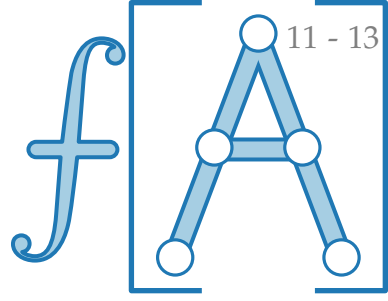
RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

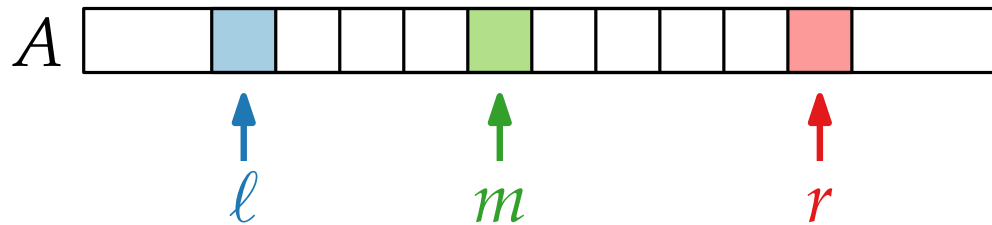
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

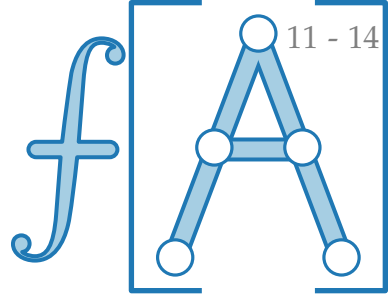
RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

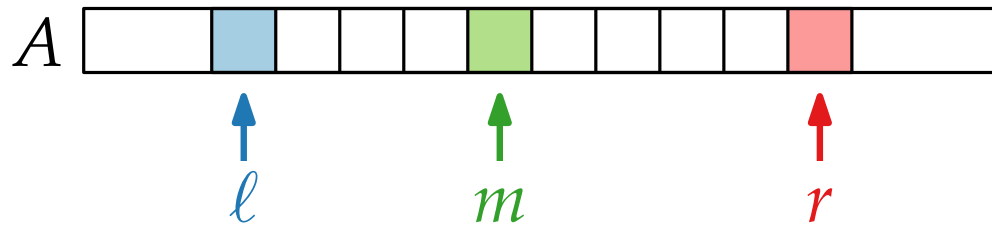
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

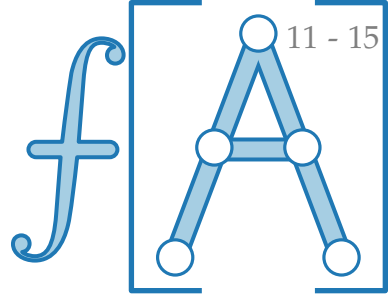
RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

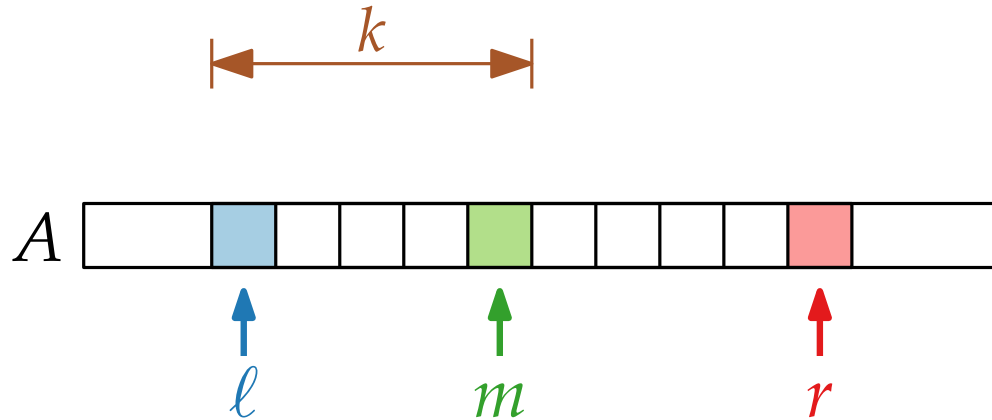
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

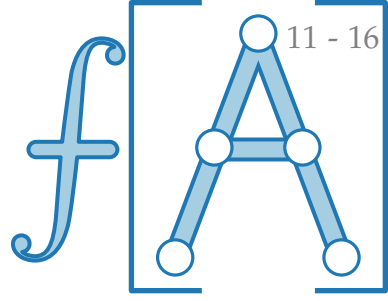
$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

if $i = k$ then

else

Auswahl per Teile & Herrsche



RANDOMIZED

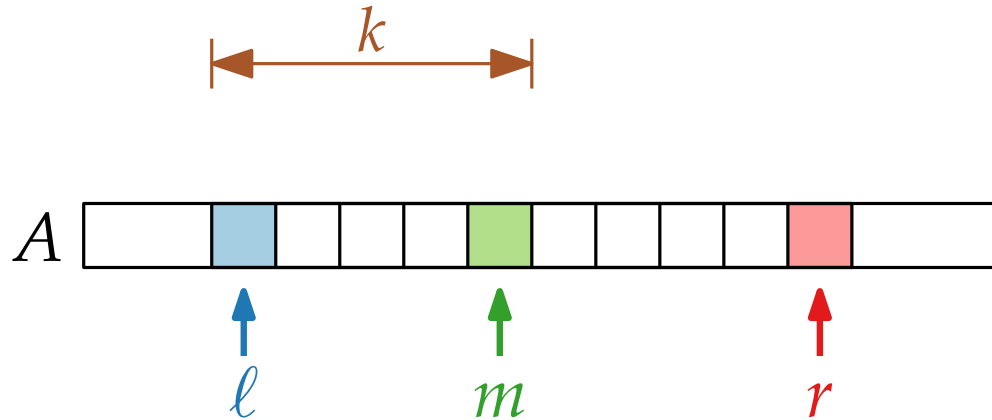
QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

 QUICKSORT($A, \ell, m - 1$)

 QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

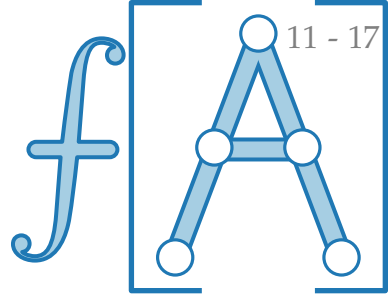
if $i = k$ then

else

 if $i < k$ then

 else

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

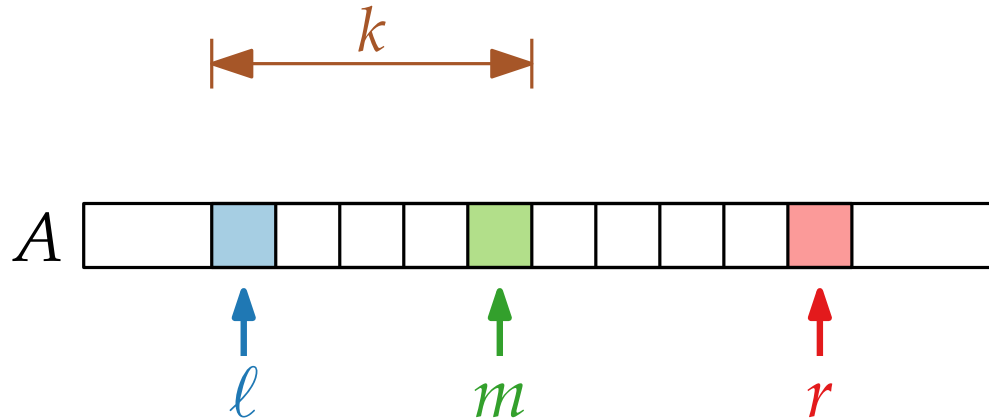
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

if $i = k$ then

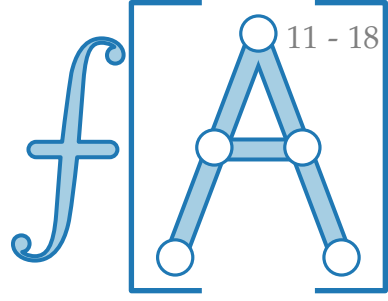
 return $A[m]$

else

 if $i < k$ then

 else

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

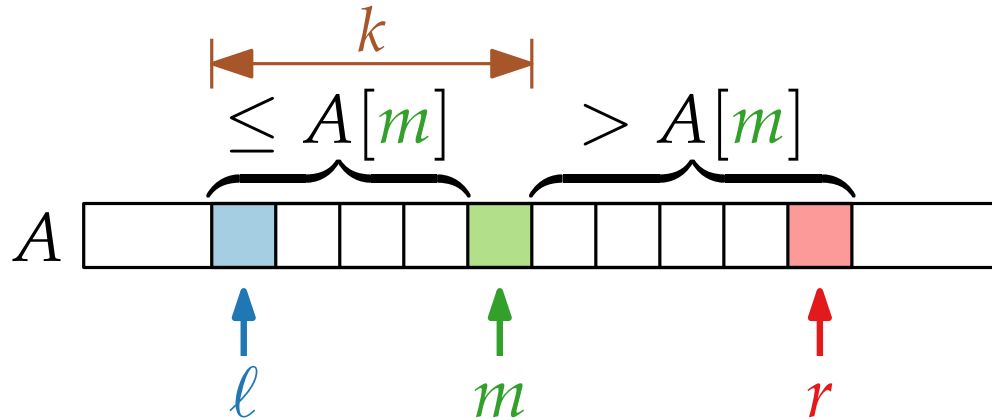
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

if $i = k$ then

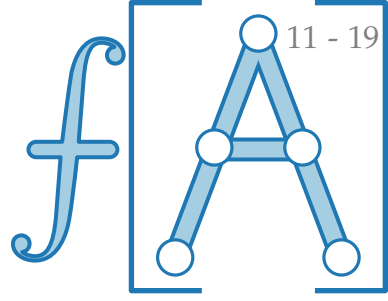
 return $A[m]$

else

 if $i < k$ then

 else

Auswahl per Teile & Herrsche



RANDOMIZED

QUICKSORT($A, \ell = 1, r = A.length$)

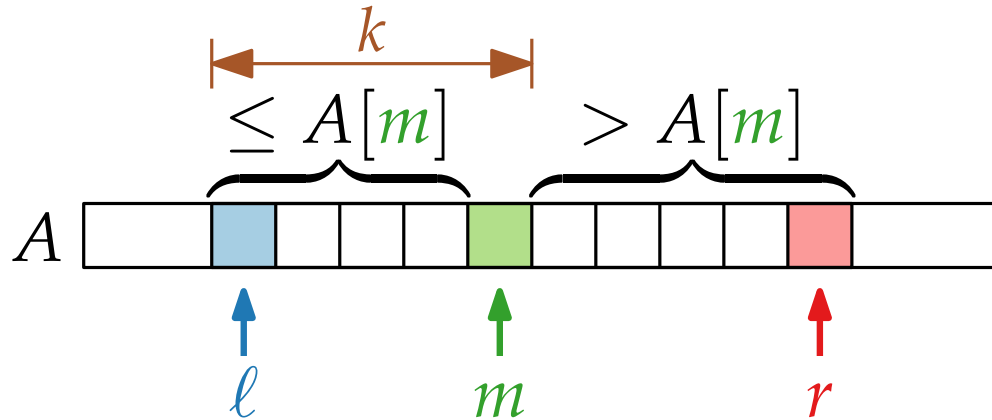
if $\ell < r$ then

RANDOMIZED

$m = \text{PARTITION}(A, \ell, r)$

QUICKSORT($A, \ell, m - 1$)

QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

if $i = k$ then

 return $A[m]$

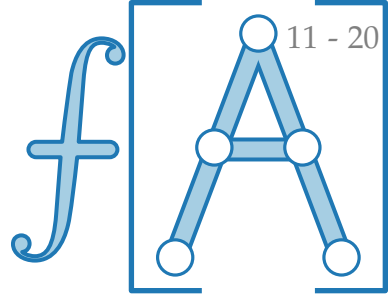
else

 if $i < k$ then

 return RSELECT($A, \ell, m - 1, i$)

 else

Auswahl per Teile & Herrsche



RANDOMIZED

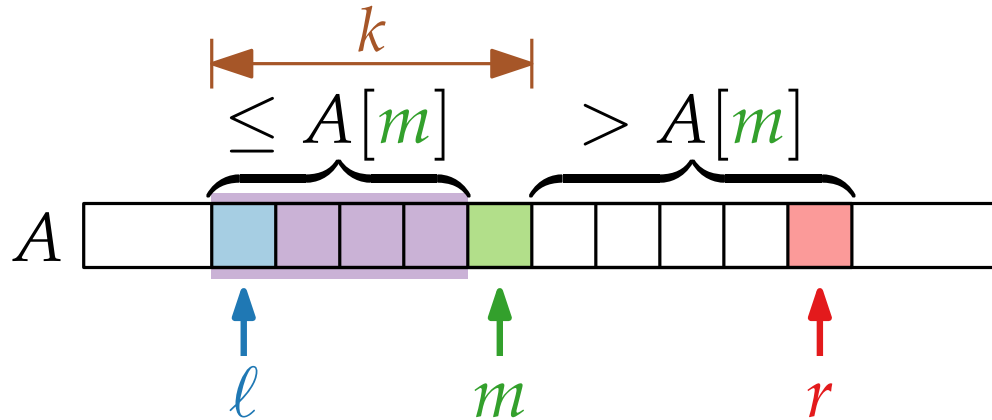
QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

 QUICKSORT($A, \ell, m - 1$)

 QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

if $i = k$ then

 return $A[m]$

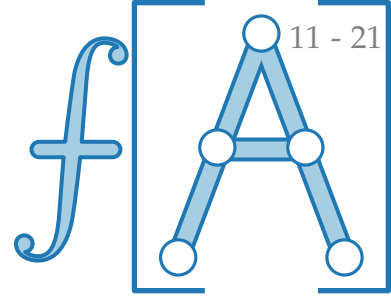
else

 if $i < k$ then

 return RSELECT($A, \ell, m - 1, i$)

 else

Auswahl per Teile & Herrsche



RANDOMIZED

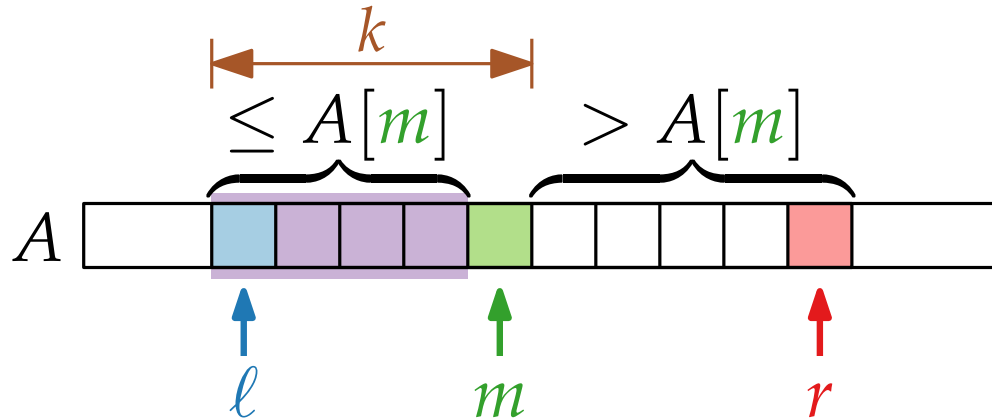
QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

 QUICKSORT($A, \ell, m - 1$)

 QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

if $i = k$ then

 return $A[m]$

else

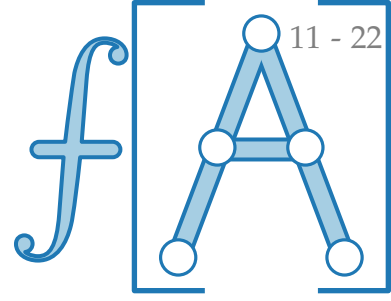
 if $i < k$ then

 return RSELECT($A, \ell, m - 1, i$)

 else

 return RSELECT($A, m + 1, r, i - k$)

Auswahl per Teile & Herrsche



RANDOMIZED

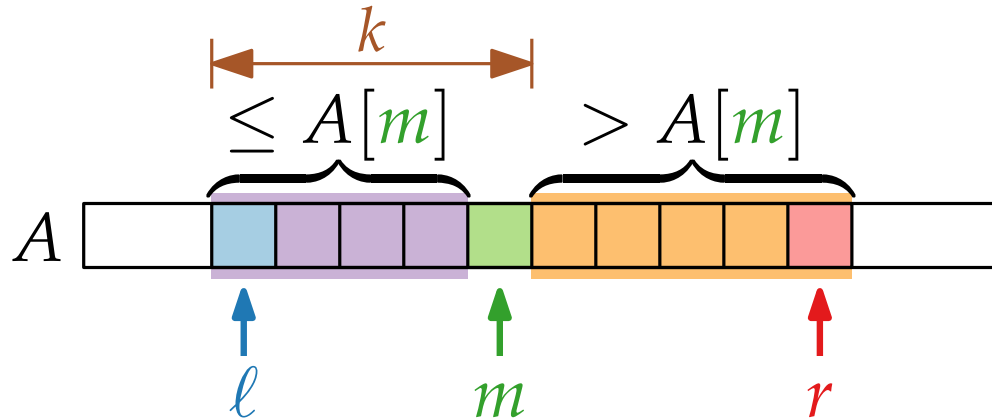
QUICKSORT($A, \ell = 1, r = A.length$)

if $\ell < r$ then

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

 QUICKSORT($A, \ell, m - 1$)

 QUICKSORT($A, m + 1, r$)



Finde i -kleinstes Element in $A[\ell \dots r]$!

RANDOMIZEDSELECT(int[] A , int ℓ, r, i)

if $\ell = r$ then return $A[\ell]$

$m = \text{RANDOMIZEDPARTITION}(A, \ell, r)$

$k = m - \ell + 1$ $\text{Rang von } A[m] \text{ in } A[\ell \dots r]$

if $i = k$ then

 return $A[m]$

else

 if $i < k$ then

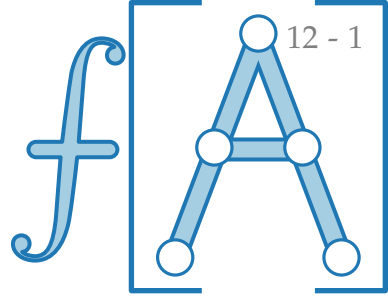
 return RSELECT($A, \ell, m - 1, i$)

 else

 return RSELECT($A, m + 1, r, i - k$)

Laufzeitanalyse

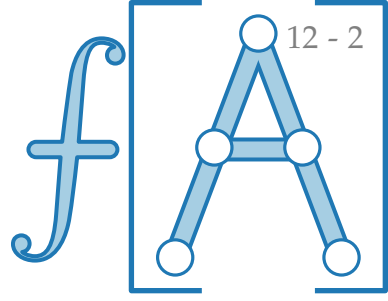
Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.



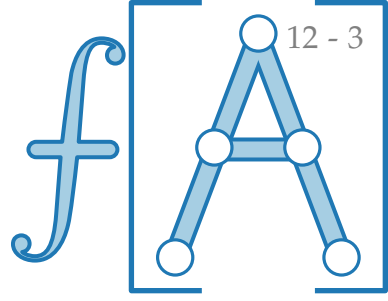
Laufzeitanalyse

Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



Laufzeitanalyse

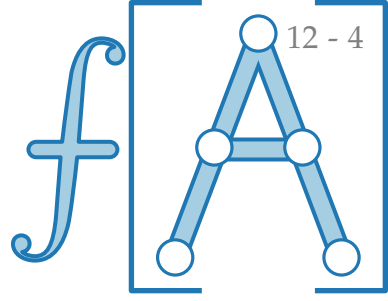


Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



Laufzeitanalyse



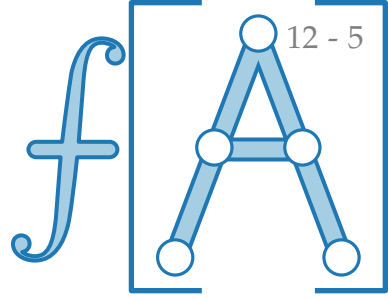
Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

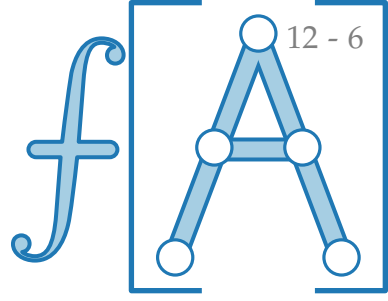
Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

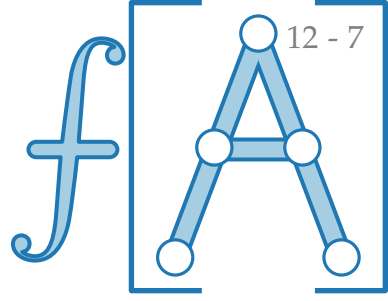
Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

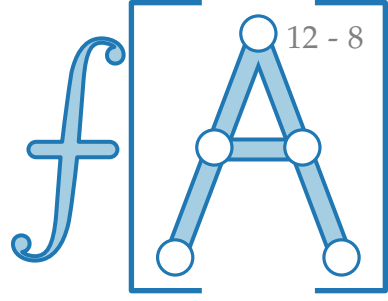


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$X(n) =$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

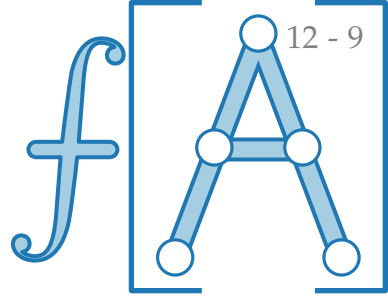


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = X_{\text{Part}}(n) +$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

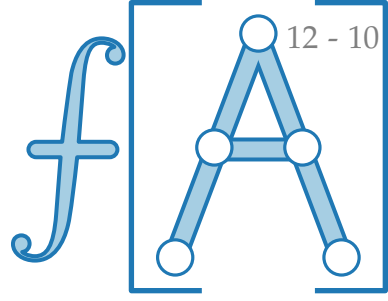


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = X_{\text{Part}}(n) + \begin{cases} X(n-1) & \text{falls } m = 1 \end{cases}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

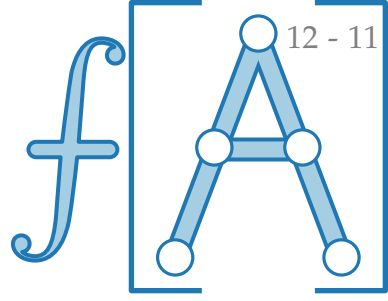


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = X_{\text{Part}}(n) + \begin{cases} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \end{cases}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

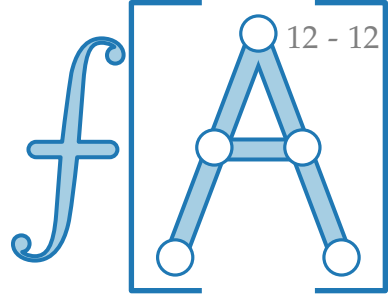


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = X_{\text{Part}}(n) + \begin{cases} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \end{cases}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

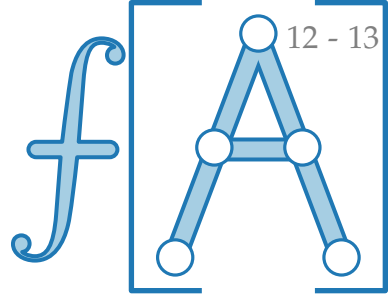


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = X_{\text{Part}}(n) + \begin{cases} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots \\ X(n-2) & \text{falls } m = n-1 \end{cases}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

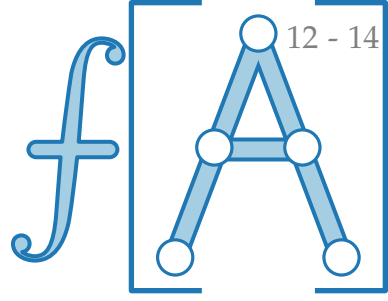


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = X_{\text{Part}}(n) + \begin{cases} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{cases}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

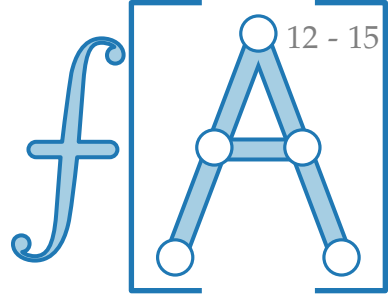


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)} + \begin{cases} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{cases}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

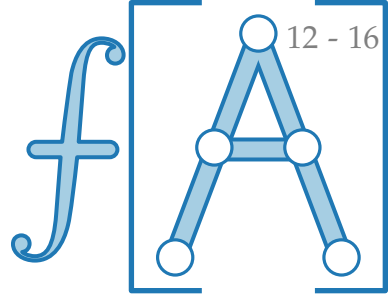


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \begin{cases} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{cases}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.

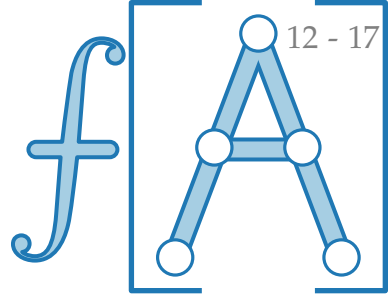


→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \left\{ \begin{array}{ll} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots & \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots & \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{array} \right\} \begin{array}{l} \text{Alle Fälle gleich} \\ \text{wahrscheinlich!} \end{array}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



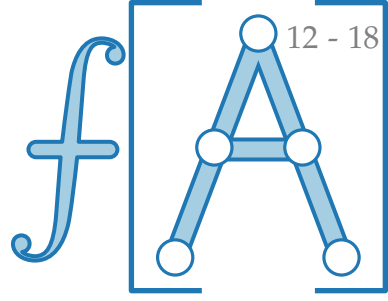
→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \left\{ \begin{array}{ll} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots & \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots & \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{array} \right\} \text{Alle Fälle gleich wahrscheinlich!}$$

vorausgesetzt alle Elemente sind verschieden!

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



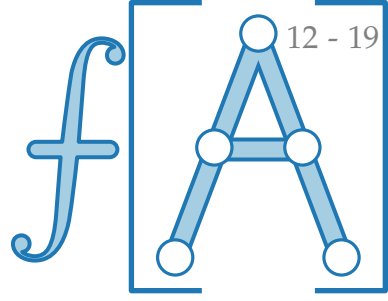
→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \left\{ \begin{array}{ll} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots & \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots & \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{array} \right\} \text{ Alle Fälle gleich wahrscheinlich!}$$

vorausgesetzt alle Elemente sind verschieden!

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

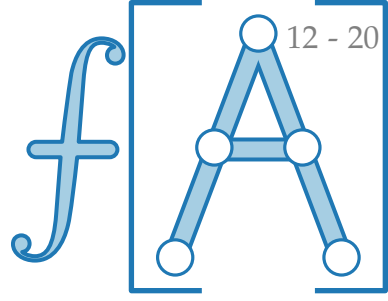
$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \left\{ \begin{array}{ll} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots & \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots & \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{array} \right\}$$

Alle Fälle gleich
wahrscheinlich!

vorausgesetzt alle
Elemente sind
verschieden!

$$\Rightarrow E[X(n)] \leq$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

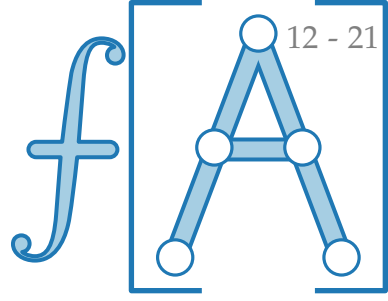
- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \left\{ \begin{array}{ll} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots & \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots & \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{array} \right\} \text{ Alle Fälle gleich wahrscheinlich!}$$

vorausgesetzt alle Elemente sind verschieden!

$$\Rightarrow E[X(n)] \leq n - 1 +$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

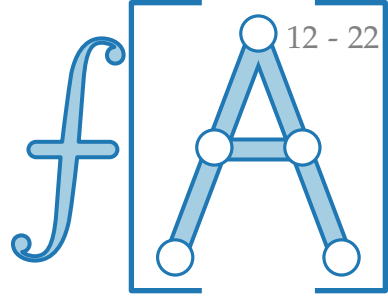
$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \left\{ \begin{array}{ll} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots & \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots & \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{array} \right\}$$

Alle Fälle gleich
wahrscheinlich!

vorausgesetzt alle
Elemente sind
verschieden!

$$\Rightarrow E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

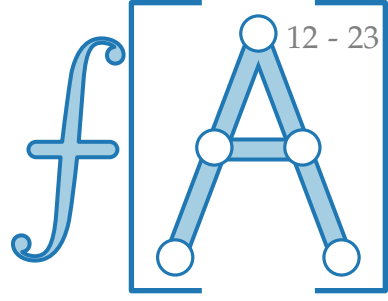
- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \left\{ \begin{array}{ll} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots & \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots & \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{array} \right\} \text{ Alle Fälle gleich wahrscheinlich!}$$

vorausgesetzt alle Elemente sind verschieden!

$$\Rightarrow E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1}$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

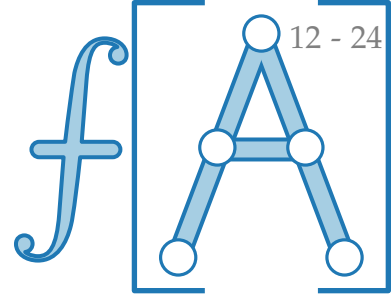
- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n - 1} + \left\{ \begin{array}{l} X(n-1) \text{ falls } m = 1 \\ X(n-2) \text{ falls } m = 2 \\ \dots \\ X(\lfloor \frac{n}{2} \rfloor) \text{ falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots \\ X(n-2) \text{ falls } m = n-1 \\ X(n-1) \text{ falls } m = n \end{array} \right\} \text{ Alle Fälle gleich wahrscheinlich!}$$

vorausgesetzt alle Elemente sind verschieden!

$$\Rightarrow E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Laufzeitanalyse



Anzahl Vergleiche von RANDOMIZEDSELECT ist ZV; hängt von n und i ab.

Trick. Geh davon aus, dass das gesuchte i . Element immer im **größeren** Teilfeld liegt.



→ resultierende Zufallsvariable $X(n)$ ist

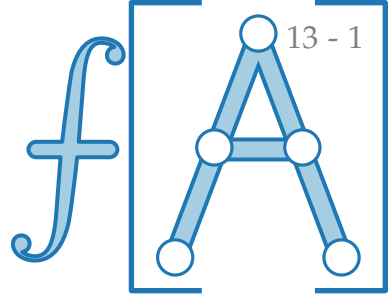
- obere Schranke für tatsächliche Anzahl von Vergleichen
- unabhängig von i

$$X(n) = \underbrace{X_{\text{Part}}(n)}_{= n-1} + \left\{ \begin{array}{ll} X(n-1) & \text{falls } m = 1 \\ X(n-2) & \text{falls } m = 2 \\ \dots & \dots \\ X(\lfloor \frac{n}{2} \rfloor) & \text{falls } m = \lfloor \frac{n}{2} \rfloor + 1 \\ \dots & \dots \\ X(n-2) & \text{falls } m = n-1 \\ X(n-1) & \text{falls } m = n \end{array} \right\} \begin{array}{l} \text{Alle Fälle gleich} \\ \text{wahrscheinlich!} \end{array}$$

vorausgesetzt alle Elemente sind verschieden!

$$\Rightarrow E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)] \leq \overset{?}{c} \cdot n \quad \left(\begin{array}{l} \text{für ein} \\ c > 0 \end{array} \right)$$

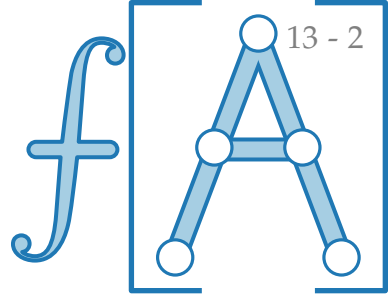
Substitutionsmethode



$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode

Wir schreiben $f(n)$ für $E[X(n)]$.

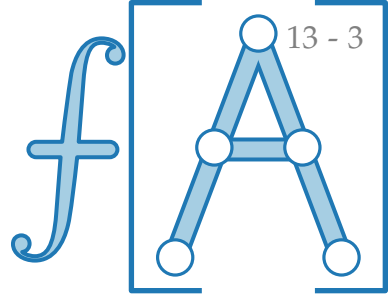


$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode

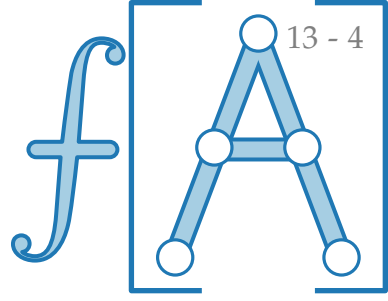
Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$



$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



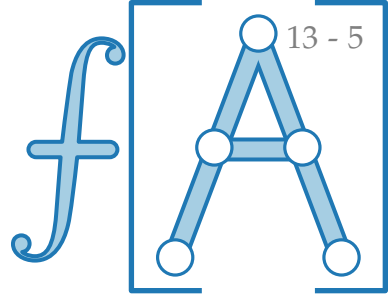
Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



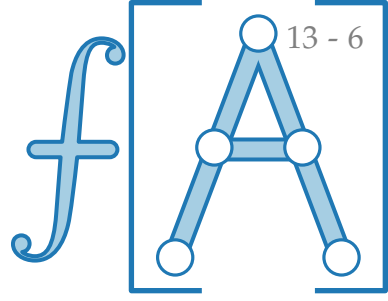
Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

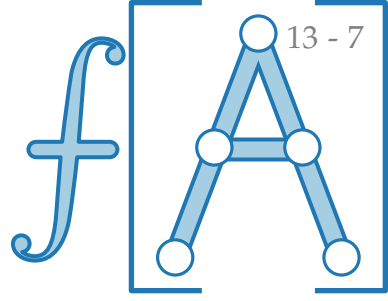
Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

Also: $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k$ laut Annahme

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

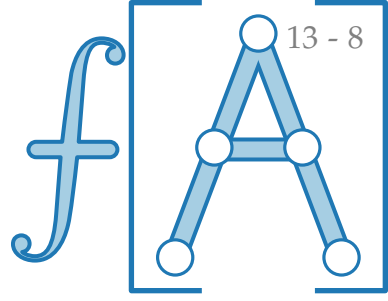
Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\begin{aligned} \text{Also: } f(n) &\leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k && \text{laut Annahme} \\ &= n + \frac{2c}{n} \left(\right) \end{aligned}$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

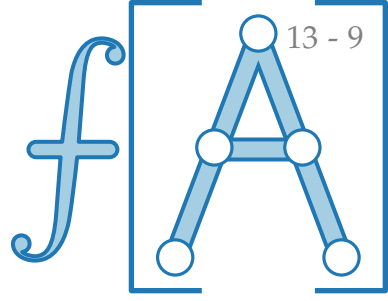
Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

Also: $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k$ laut Annahme

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \right)$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

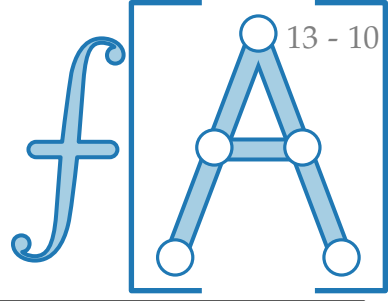
Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

Also: $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k$ laut Annahme

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

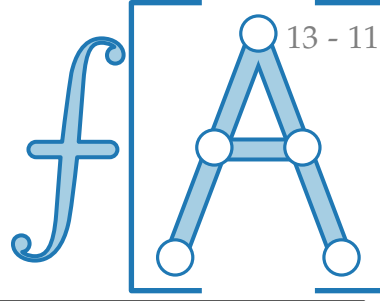
Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

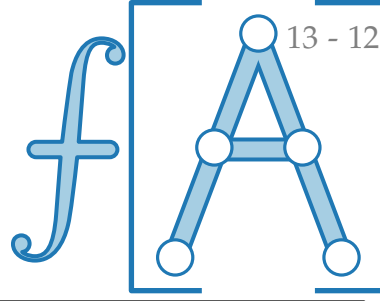
$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

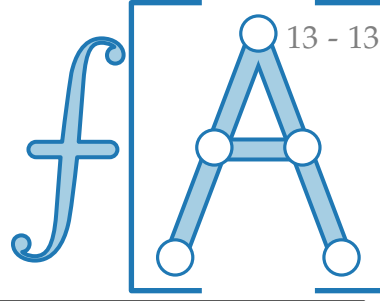
$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

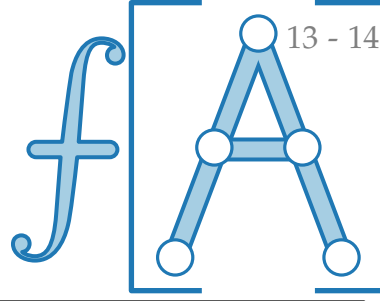
$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

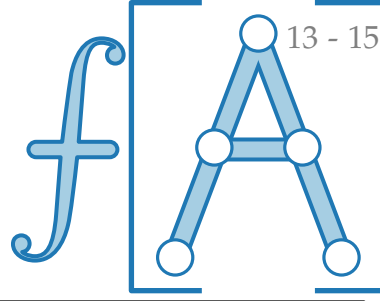
$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

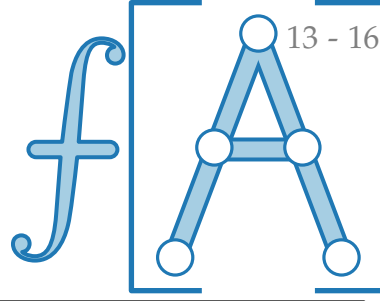
Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$\begin{aligned} &= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right) \\ &= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right) \\ &\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2)) \\ &= n + c(n - 1 - n/4 + 3/2 - 2/n) \end{aligned}$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

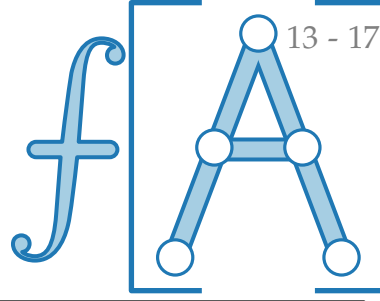
Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$\begin{aligned} &= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right) \\ &= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right) \\ &\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2)) \\ &= n + c(n - 1 - n/4 + 3/2 - 2/n) \end{aligned}$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

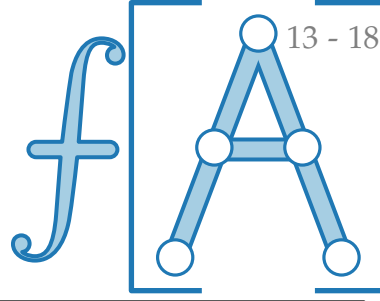
$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c(n - 1 - n/4 + 3/2 - 2/n) \leq n + c.$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

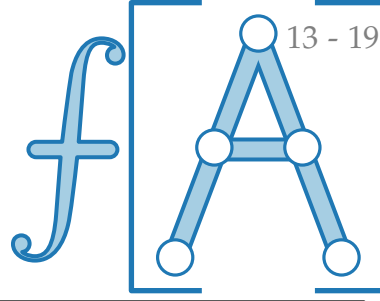
$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

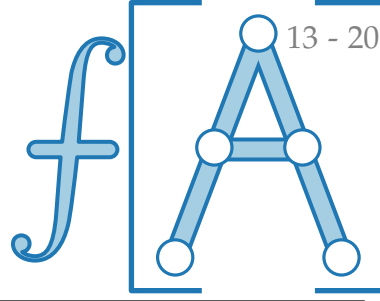
$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n}{4}$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

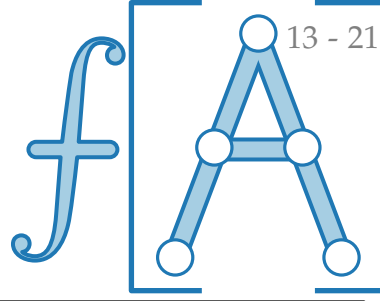
$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n}{4}$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

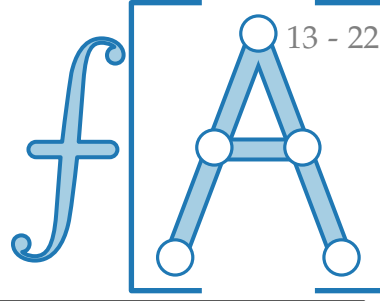
$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

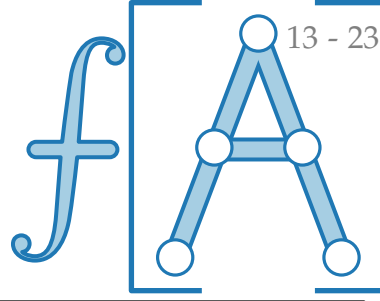
$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \left(c \cdot \frac{n-2}{4} - n \right)$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

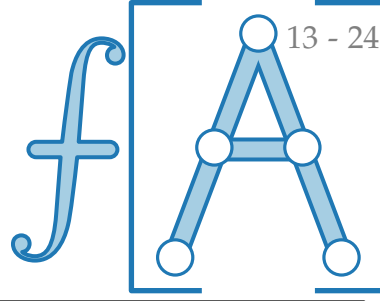
$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \left(c \cdot \frac{n-2}{4} - n \right) \leq cn$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

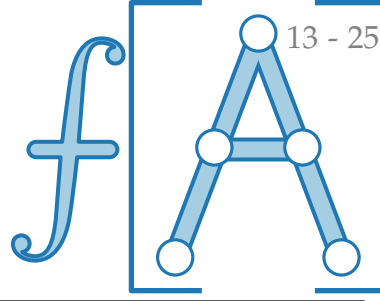
$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \left(c \cdot \frac{n-2}{4} - n \right) \leq cn$$

$\geq 0?! \rightarrow$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

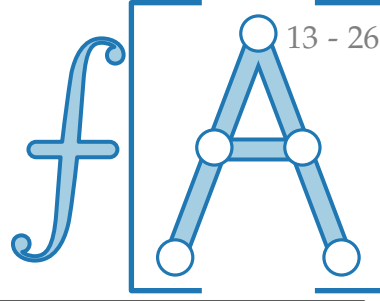
$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \left(c \cdot \frac{n-2}{4} - n \right) \leq cn \quad \text{falls } c \geq \frac{4n}{n-2} =$$

$\geq 0?! \rightarrow$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

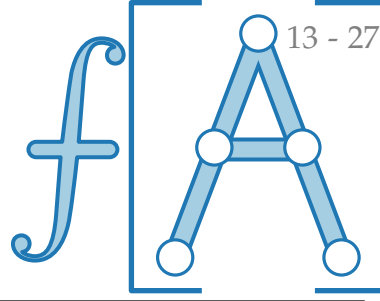
$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \underbrace{\left(c \cdot \frac{n-2}{4} - n \right)}_{\geq 0?!} \leq cn \quad \text{falls } c \geq \frac{4n}{n-2} = \frac{4}{1-2/n} \xrightarrow{n \rightarrow \infty}$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

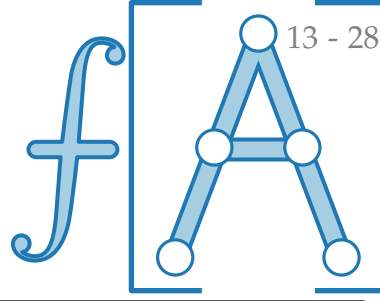
$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \underbrace{\left(c \cdot \frac{n-2}{4} - n \right)}_{\geq 0?!} \leq cn \quad \text{falls } c \geq \frac{4n}{n-2} = \frac{4}{1-2/n} \xrightarrow{n \rightarrow \infty} 4^+$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)]$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

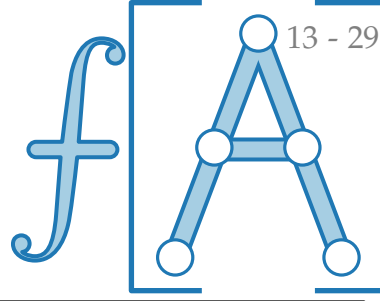
$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \underbrace{\left(c \cdot \frac{n-2}{4} - n \right)}_{\geq 0?!} \leq cn \quad \text{falls } c \geq \frac{4n}{n-2} = \frac{4}{1-2/n} \xrightarrow{n \rightarrow \infty} 4^+$$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)] \leq \overbrace{(4 + \varepsilon)}^{c:=} n$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

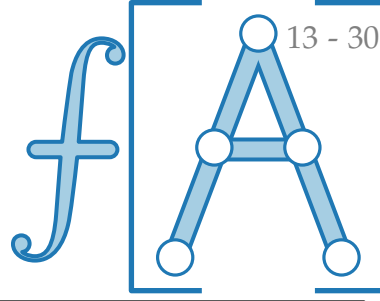
$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \left(c \cdot \frac{n-2}{4} - n \right) \leq cn \quad \text{falls } c \geq \frac{4n}{n-2} = \frac{4}{1-2/n} \xrightarrow{n \rightarrow \infty} 4^+$$

$\geq 0?! \rightarrow$

$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)] \leq \overbrace{(4 + \varepsilon)}^{c :=} n \quad \text{falls } n \geq \frac{8}{\varepsilon} + 2$$

Substitutionsmethode



Wir schreiben $f(n)$ für $E[X(n)]$.

Dann gilt $f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} f(k)$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \text{arithmetische Reihe}$$

Wir wollen prüfen, ob es ein $c > 0$ gibt, so dass $f(n) \leq cn$.

$$\text{Also: } f(n) \leq n + \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} c \cdot k \quad \text{laut Annahme}$$

$$= n + \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right)$$

$$= n + \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{c}{n} (n(n-1) - (n/2 - 1)(n/2 - 2))$$

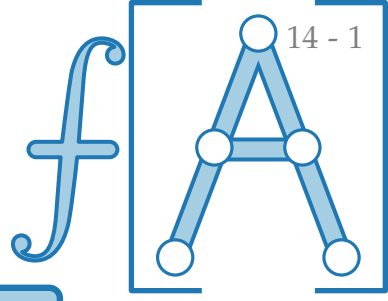
$$= n + c \left(n - 1 - n/4 + 3/2 - 2/n \right) \leq n + c \cdot \frac{3n+2}{4}$$

$$= cn - \underbrace{\left(c \cdot \frac{n-2}{4} - n \right)}_{\geq 0?!} \leq cn \quad \text{falls } c \geq \frac{4n}{n-2} = \frac{4}{1-2/n} \xrightarrow{n \rightarrow \infty} 4^+$$

Für jedes $\varepsilon > 0$ gilt:

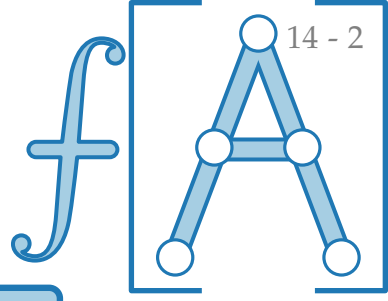
$$E[X(n)] \leq n - 1 + 2 \cdot \frac{1}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[X(k)] \leq \overbrace{(4 + \varepsilon)}^{c :=} n \quad \text{falls } n \geq \frac{8}{\varepsilon} + 2$$

Ergebnis und Diskussion



Satz. Das Auswahlproblem kann in erwarteter linearer Zeit gelöst werden.

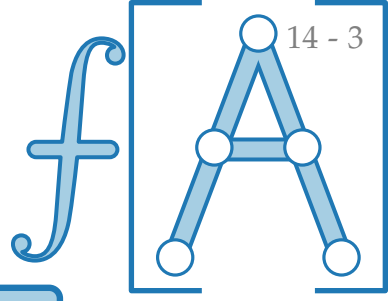
Ergebnis und Diskussion



Satz. Das Auswahlproblem kann in erwarteter linearer Zeit gelöst werden.

Genauer.

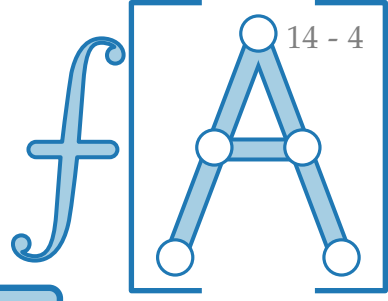
Ergebnis und Diskussion



Satz. Das Auswahlproblem kann in erwarteter linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq \frac{8}{\varepsilon} + 2$ Zahlen die i -kleinste Zahl ($1 \leq i \leq n$) mit erwartet $(4 + \varepsilon)n$ Vergleichen finden kann.

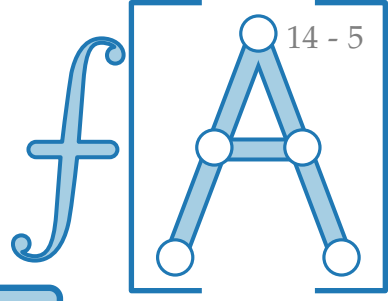
Ergebnis und Diskussion



Satz. Das Auswahlproblem kann in erwarteter linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq \frac{8}{\varepsilon} + 2$ Zahlen die i -kleinste Zahl ($1 \leq i \leq n$) mit **erwartet** $(4 + \varepsilon)n$ Vergleichen finden kann.

Ergebnis und Diskussion

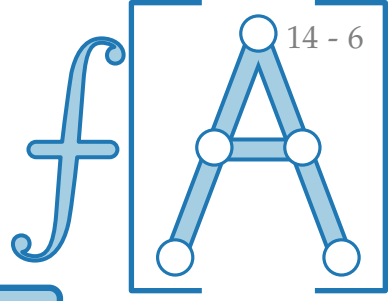


Satz. Das Auswahlproblem kann in erwarteter linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq \frac{8}{\varepsilon} + 2$ Zahlen die i -kleinste Zahl ($1 \leq i \leq n$) mit **erwartet** $(4 + \varepsilon)n$ Vergleichen finden kann.

Frage.

Ergebnis und Diskussion

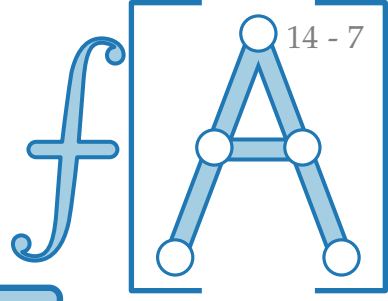


Satz. Das Auswahlproblem kann in erwarteter linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq \frac{8}{\varepsilon} + 2$ Zahlen die i -kleinste Zahl ($1 \leq i \leq n$) mit **erwartet** $(4 + \varepsilon)n$ Vergleichen finden kann.

Frage. Geht das auch **deterministisch**, d.h. ohne Zufall?

Ergebnis und Diskussion

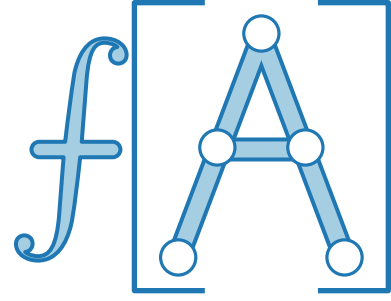


Satz. Das Auswahlproblem kann in erwarteter linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq \frac{8}{\varepsilon} + 2$ Zahlen die i -kleinste Zahl ($1 \leq i \leq n$) mit **erwartet** $(4 + \varepsilon)n$ Vergleichen finden kann.

Frage. Geht das auch **deterministisch**, d.h. ohne Zufall?

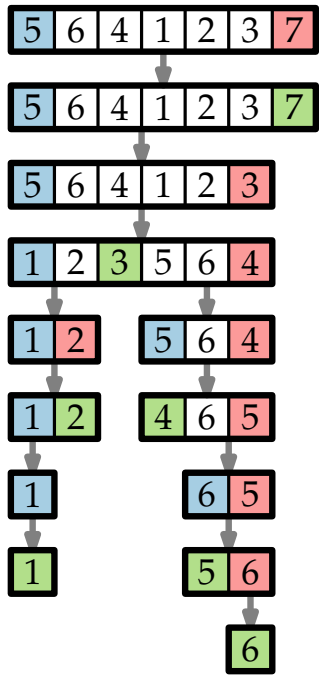
M.a.W. Kann man das Auswahlproblem auch im **schlechtesten Fall** in linearer Zeit lösen?



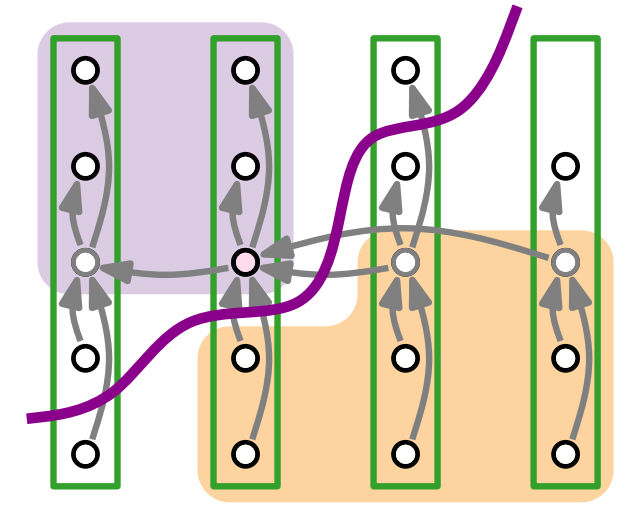
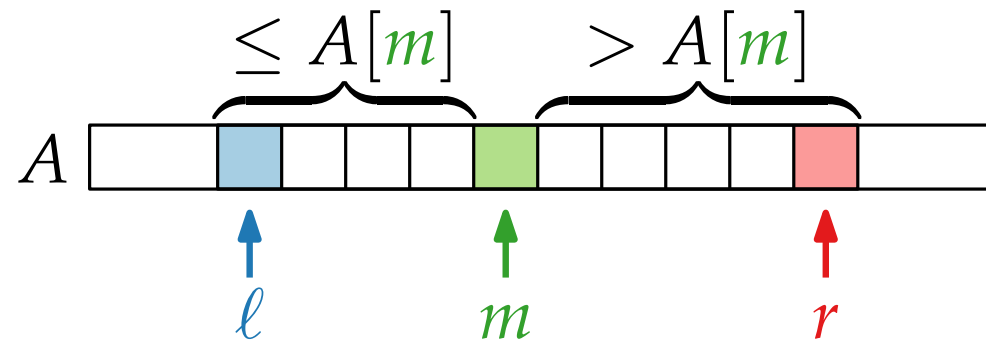
Fortgeschrittene Algorithmen

4. Vorlesung

Randomisierte Algorithmen II: Das Auswahlproblem

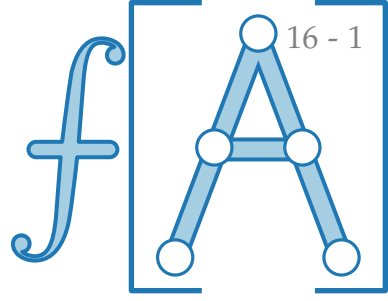


Teil III:
Das Auswahlproblem:
deterministischer Algorithmus



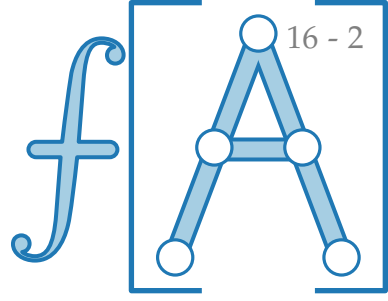
Vorbereitung

Wir verwenden wieder Divide & Conquer –



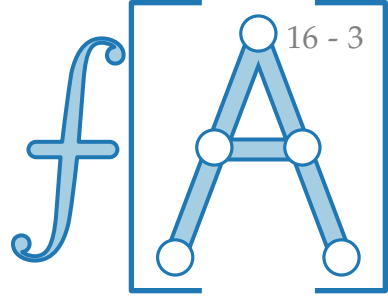
Vorbereitung

Wir verwenden wieder Divide & Conquer –
aber diesmal mit einer garantiert **guten** Aufteilung in Teilfelder.



Vorbereitung

Wir verwenden wieder Divide & Conquer –
aber diesmal mit einer garantiert **guten** Aufteilung in Teilfelder.
d.h. **balanciert**:

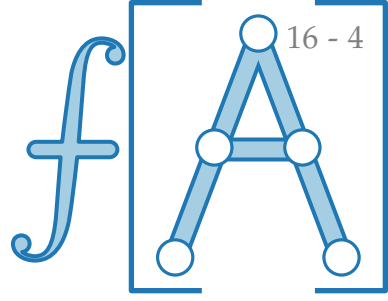


Vorbereitung

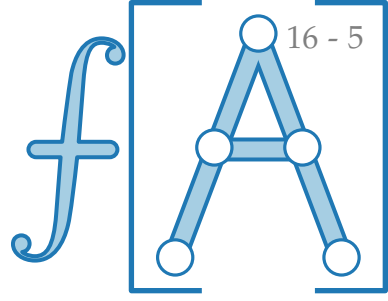
Wir verwenden wieder Divide & Conquer –
aber diesmal mit einer garantiert **guten** Aufteilung in Teilfelder.

d.h. **balanciert**:

jede Seite sollte $\geq \gamma n$ Elem. enthalten, für ein festes $0 < \gamma \leq \frac{1}{2}$.



Vorbereitung



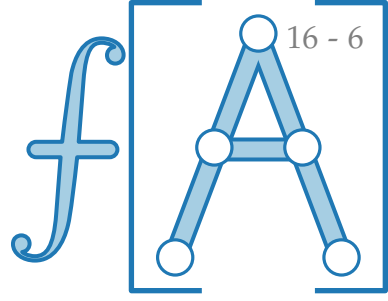
Wir verwenden wieder Divide & Conquer –
aber diesmal mit einer garantiert **guten** Aufteilung in Teilfelder.

d.h. **balanciert**:

jede Seite sollte $\geq \gamma n$ Elem. enthalten, für ein festes $0 < \gamma \leq \frac{1}{2}$.

```
PARTITION ( $A, \ell, r$ )  
   $pivot = A[r]$   
   $i = \ell$   
  for  $j = \ell$  to  $r - 1$  do  
    if  $A[j] \leq pivot$  then  
       $A[i] \rightleftharpoons A[j]$   
       $i = i + 1$   
   $A[i] \rightleftharpoons A[r]$   
  return  $i$ 
```

Vorbereitung



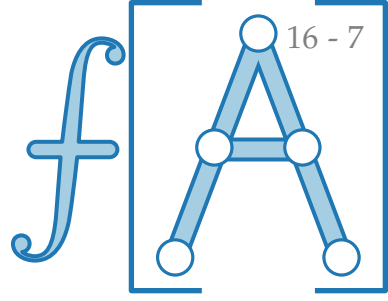
Wir verwenden wieder Divide & Conquer –
aber diesmal mit einer garantiert **guten** Aufteilung in Teilfelder.

d.h. **balanciert**:

jede Seite sollte $\geq \gamma n$ Elem. enthalten, für ein festes $0 < \gamma \leq \frac{1}{2}$.

```
PARTITION'(A,  $\ell$ ,  $r$ )  
  pivot = A[ $r$ ]  
   $i = \ell$   
  for  $j = \ell$  to  $r - 1$  do  
    if A[ $j$ ]  $\leq$  pivot then  
      A[ $i$ ]  $\rightleftharpoons$  A[ $j$ ]  
       $i = i + 1$   
  A[ $i$ ]  $\rightleftharpoons$  A[ $r$ ]  
  return  $i$ 
```

Vorbereitung



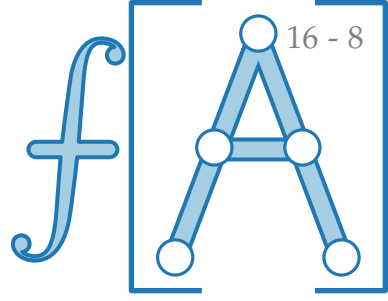
Wir verwenden wieder Divide & Conquer –
aber diesmal mit einer garantiert **guten** Aufteilung in Teilfelder.

d.h. **balanciert**:

jede Seite sollte $\geq \gamma n$ Elem. enthalten, für ein festes $0 < \gamma \leq \frac{1}{2}$.

```
PARTITION'(A,  $\ell$ ,  $r$ , pivot)
  pivot = A[ $r$ ]
   $i = \ell$ 
  for  $j = \ell$  to  $r - 1$  do
    if A[ $j$ ]  $\leq$  pivot then
      A[ $i$ ]  $\rightleftharpoons$  A[ $j$ ]
       $i = i + 1$ 
  A[ $i$ ]  $\rightleftharpoons$  A[ $r$ ]
  return  $i$ 
```

Vorbereitung



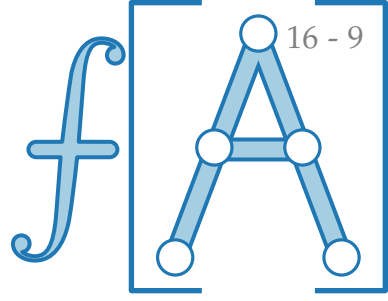
Wir verwenden wieder Divide & Conquer –
aber diesmal mit einer garantiert **guten** Aufteilung in Teilfelder.

d.h. **balanciert**:

jede Seite sollte $\geq \gamma n$ Elem. enthalten, für ein festes $0 < \gamma \leq \frac{1}{2}$.

```
PARTITION'(A,  $\ell$ ,  $r$ , pivot)
pivot = A[r]
 $i = \ell$ 
for  $j = \ell$  to  $r - 1$  do
    if  $A[j] \leq \text{pivot}$  then
         $A[i] \rightleftharpoons A[j]$ 
         $i = i + 1$ 
 $A[i] \rightleftharpoons A[r]$ 
return  $i$ 
```

Vorbereitung



Wir verwenden wieder Divide & Conquer –
aber diesmal mit einer garantiert **guten** Aufteilung in Teilfelder.

d.h. **balanciert**:

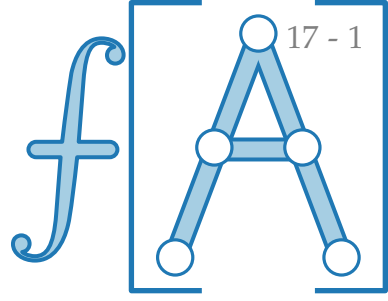
jede Seite sollte $\geq \gamma n$ Elem. enthalten, für ein festes $0 < \gamma \leq \frac{1}{2}$.

```
PARTITION'(A,  $\ell$ ,  $r$ , pivot)
pivot = A[r]
 $i = \ell$ 
for  $j = \ell$  to  $r - 1$  do
    if  $A[j] \leq \text{pivot}$  then
         $A[i] \rightleftharpoons A[j]$ 
         $i = i + 1$ 
 $A[i] \rightleftharpoons A[r]$ 
return  $i$ 
```

Wir gehen für die Analyse
wieder davon aus, dass alle
Elemente verschieden sind.

SELECT: deterministisch

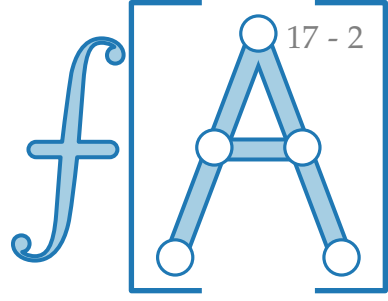
SELECT(A, ℓ, r, i)



SELECT: deterministisch

SELECT(A, ℓ, r, i)

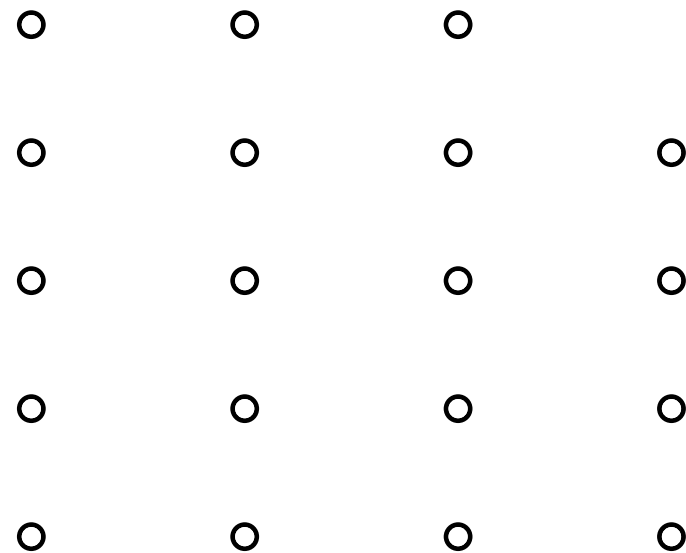
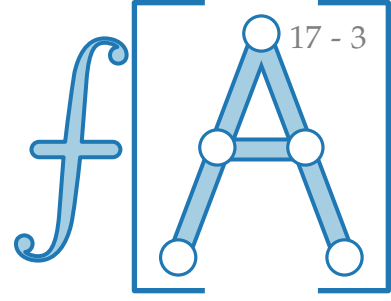
1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.



SELECT: deterministisch

SELECT(A, ℓ, r, i)

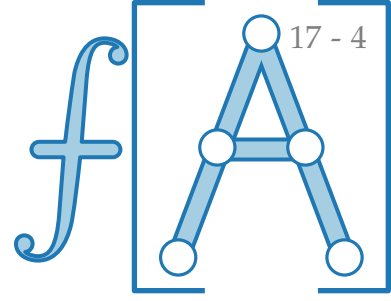
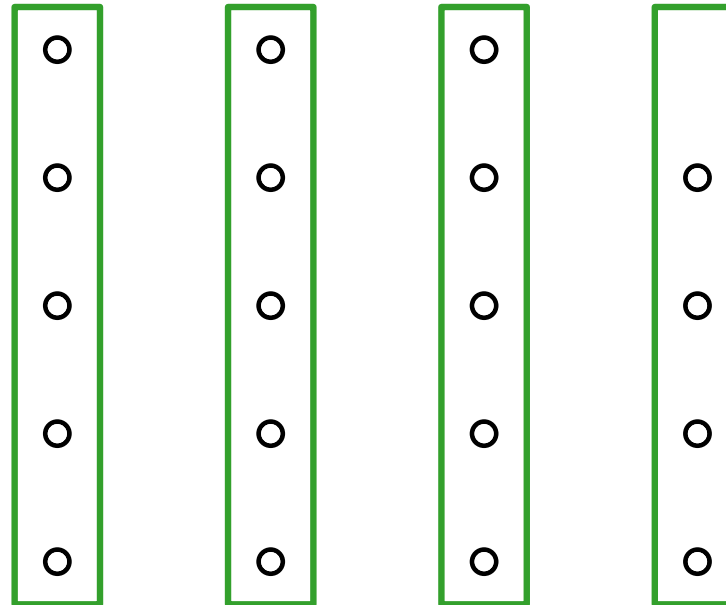
1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.



SELECT: deterministisch

SELECT(A, ℓ, r, i)

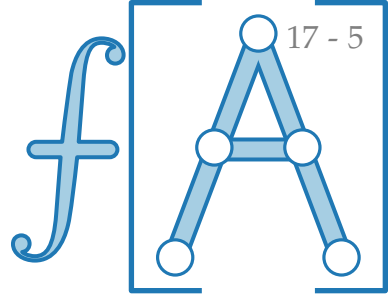
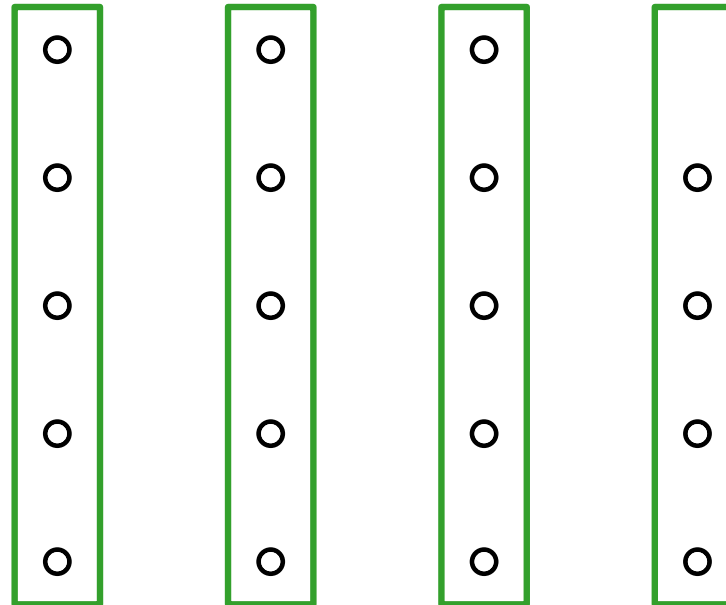
1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.



SELECT: deterministisch

SELECT(A, ℓ, r, i)

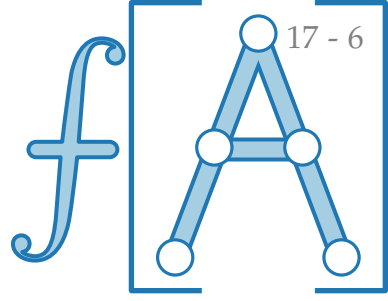
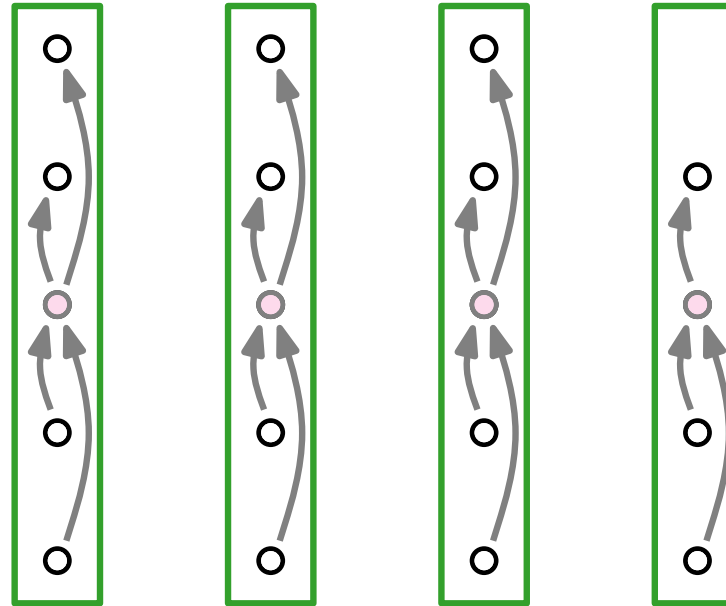
1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lceil n/5 \rceil$ Gruppen und bestimme ihren Median.



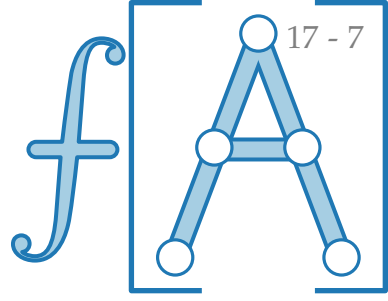
SELECT: deterministisch

SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.

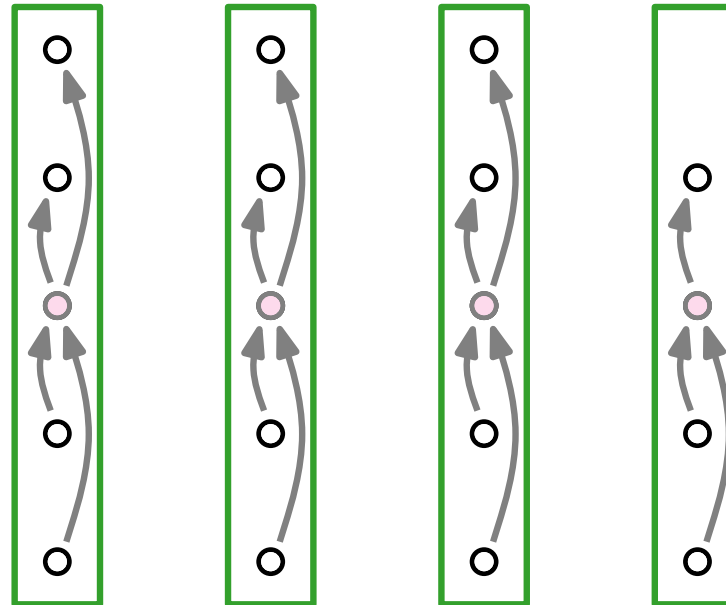


SELECT: deterministisch



SELECT(A, ℓ, r, i)

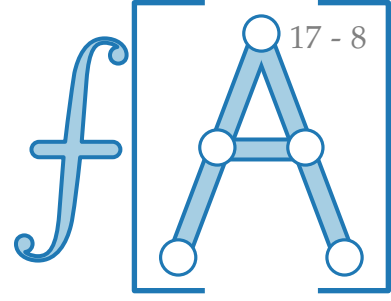
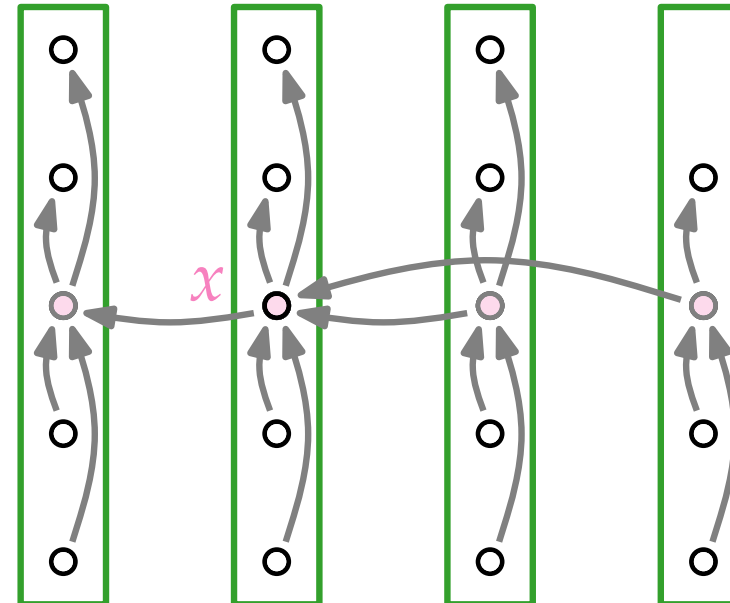
1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.



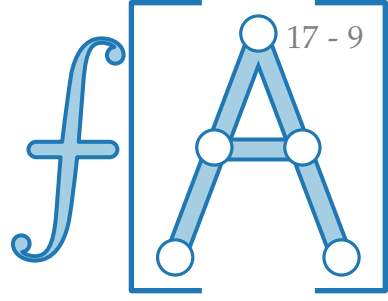
SELECT: deterministisch

SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.

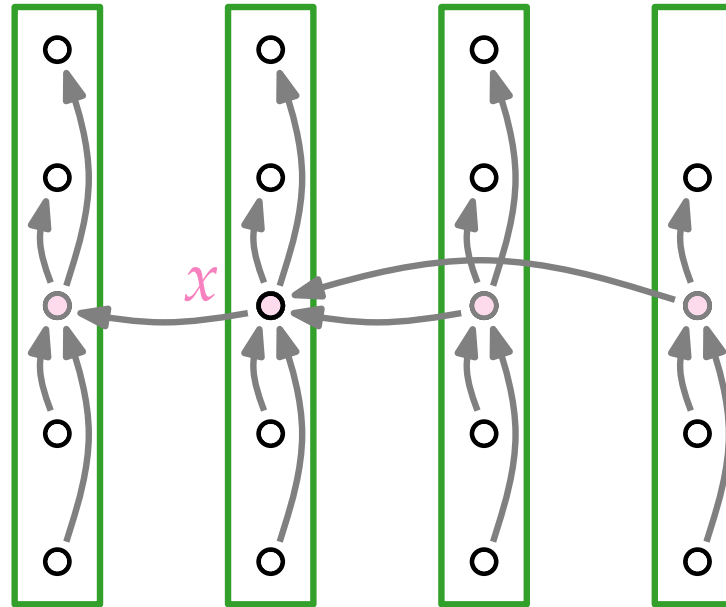


SELECT: deterministisch



SELECT(A, ℓ, r, i)

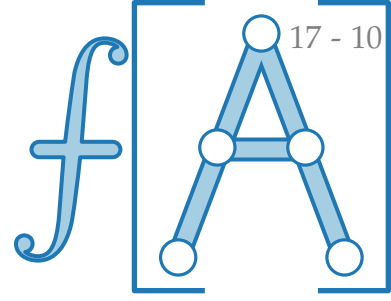
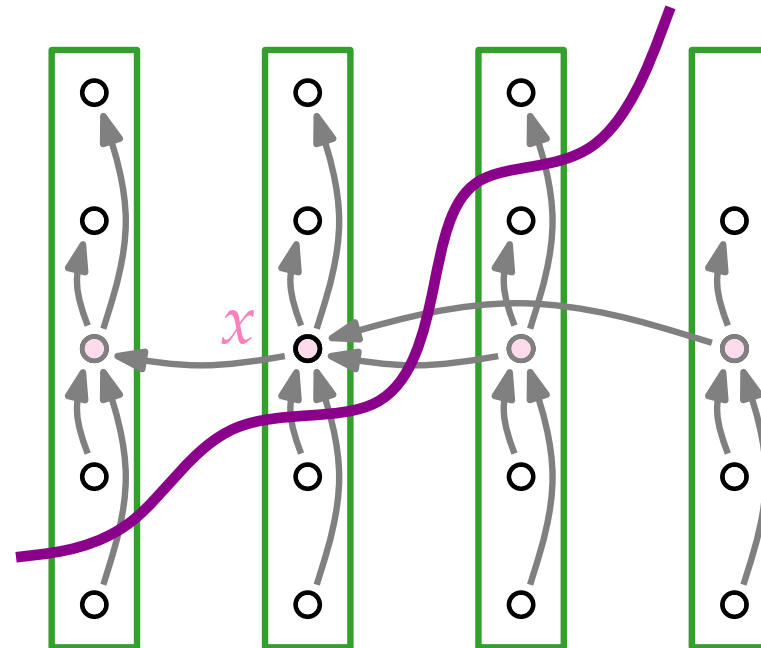
1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x)$



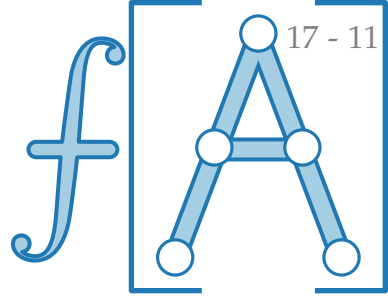
SELECT: deterministisch

SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x)$

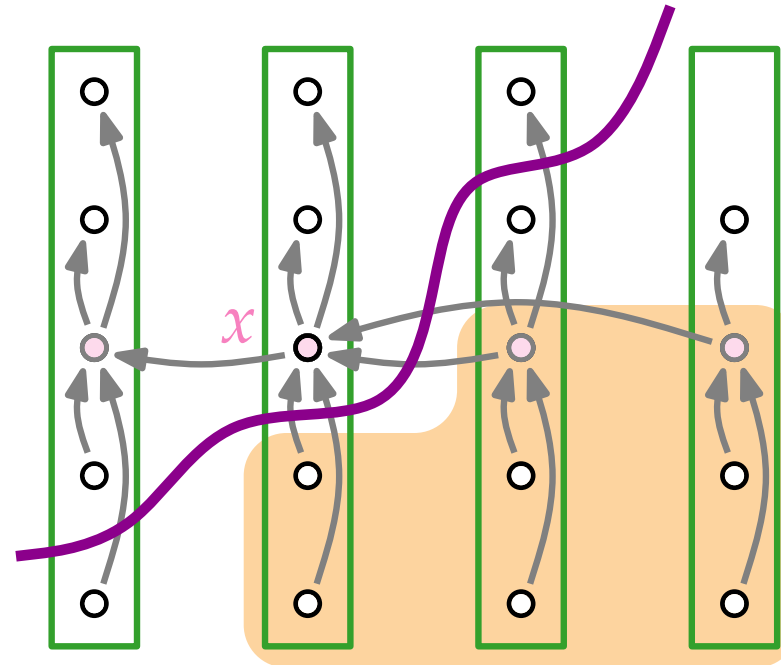


SELECT: deterministisch

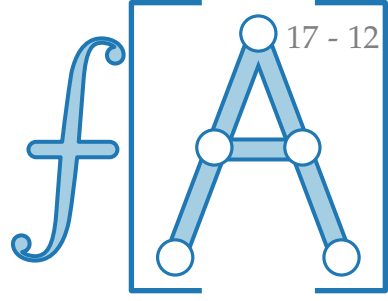


SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x)$

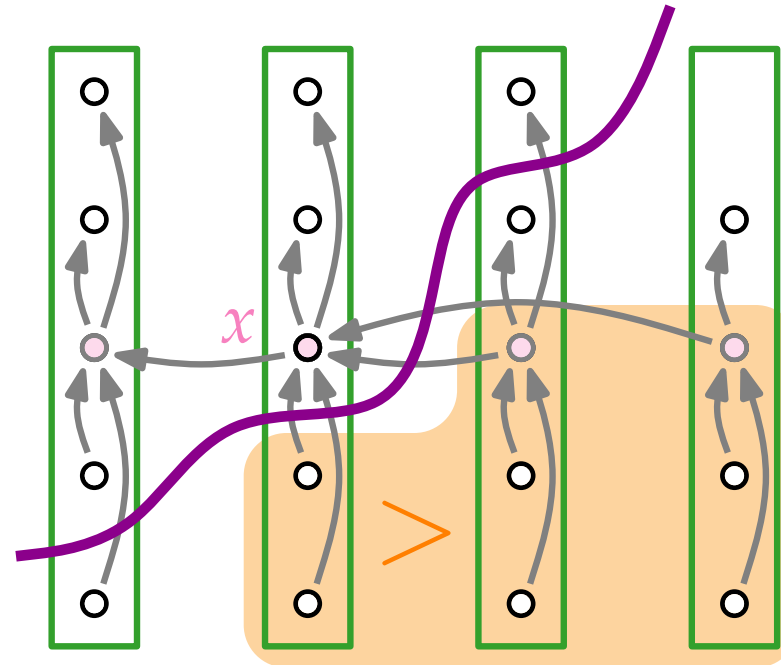


SELECT: deterministisch

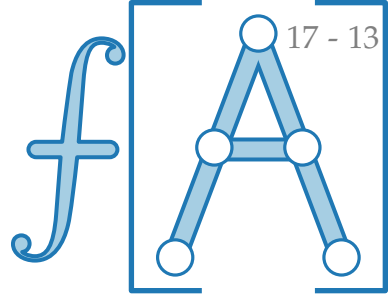


SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x)$

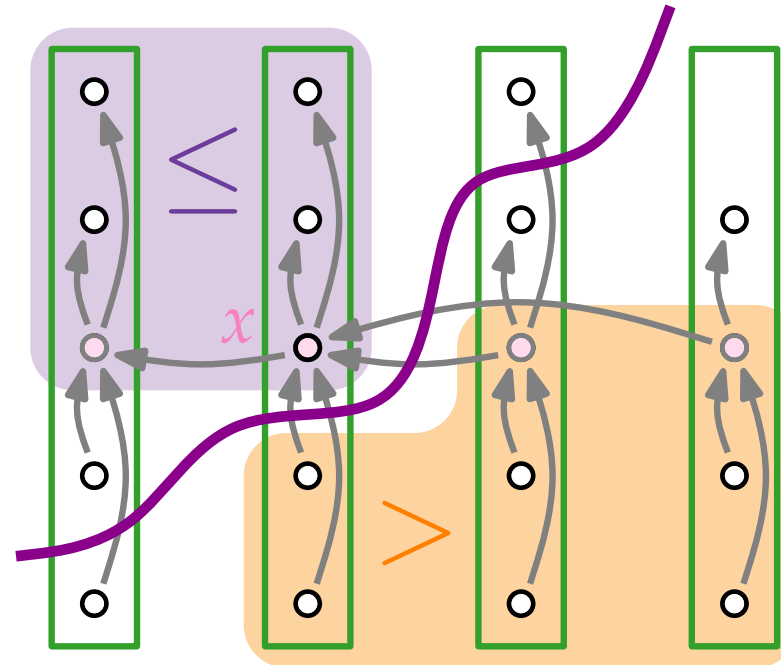


SELECT: deterministisch

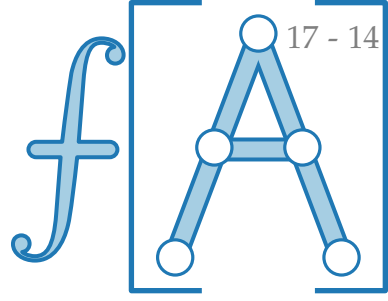


SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x)$

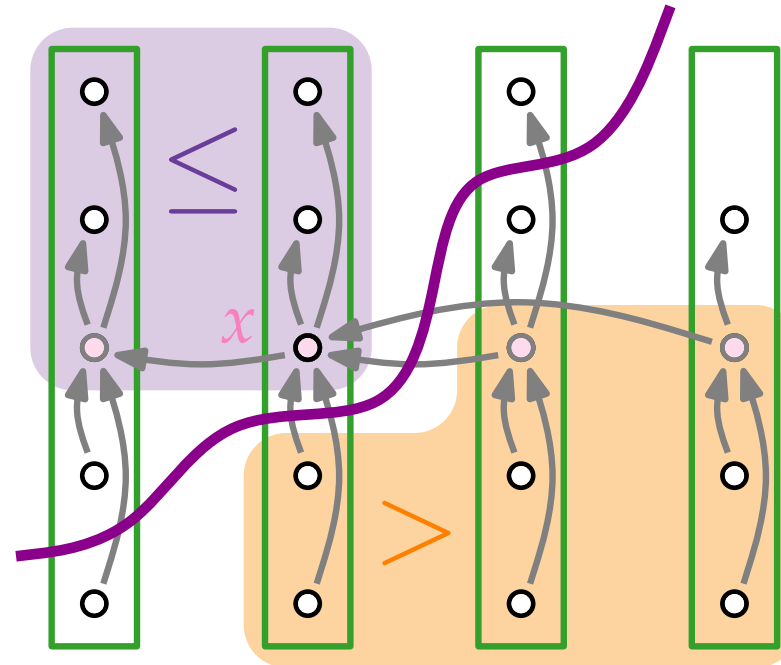


SELECT: deterministisch

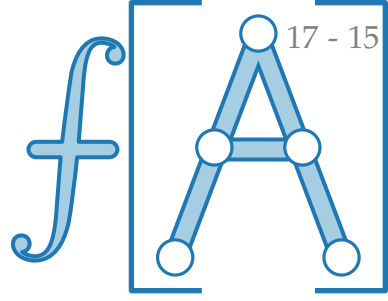


SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$

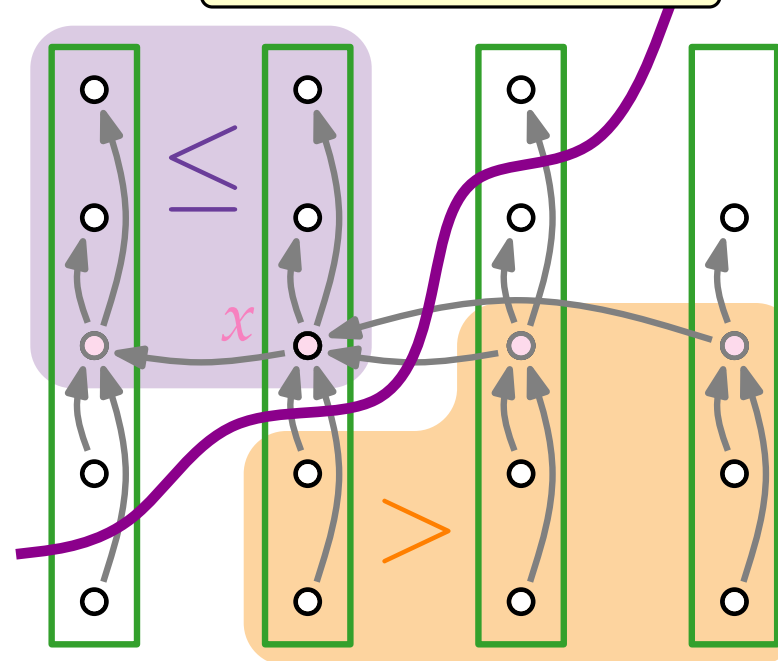


SELECT: deterministisch

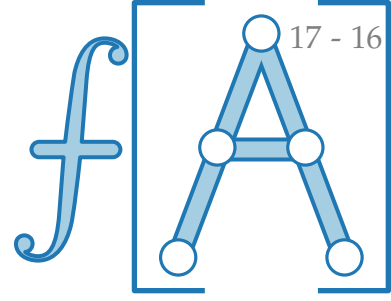


SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen $(n \bmod 5)$ Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$ A[m] k -kleinstes Element



SELECT: deterministisch

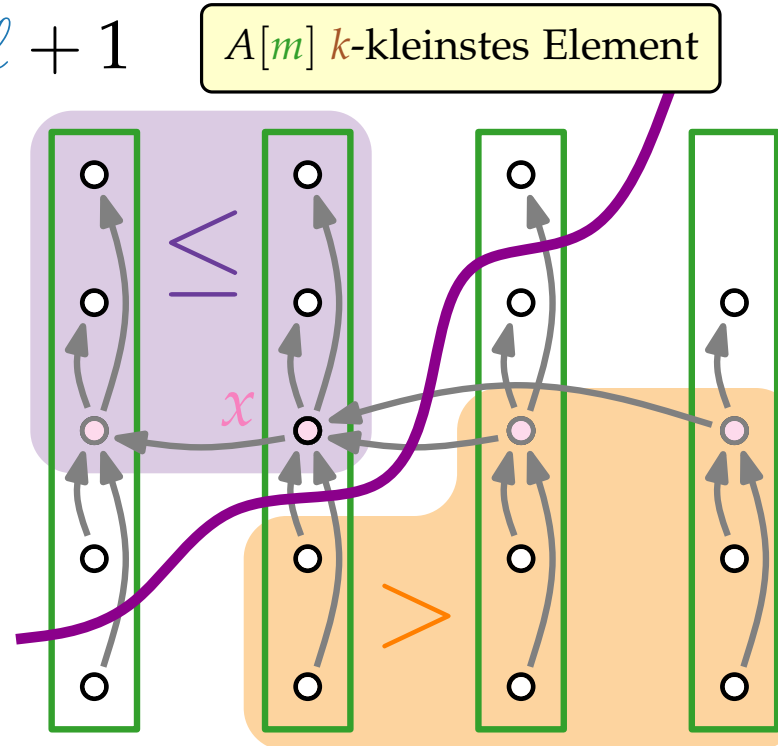


SELECT(A, ℓ, r, i)

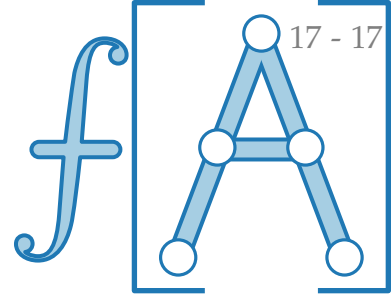
1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lceil n/5 \rceil$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$

5.

```
if  $i == k$  then return  $A[m]$ 
else
  if  $i < k$  then
    return SELECT( $A, \ell, m - 1, i$ )
  else
    return SELECT( $A, m + 1, r, i - k$ )
```



SELECT: deterministisch



SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$

5.

if $i == k$ then return $A[m]$

else

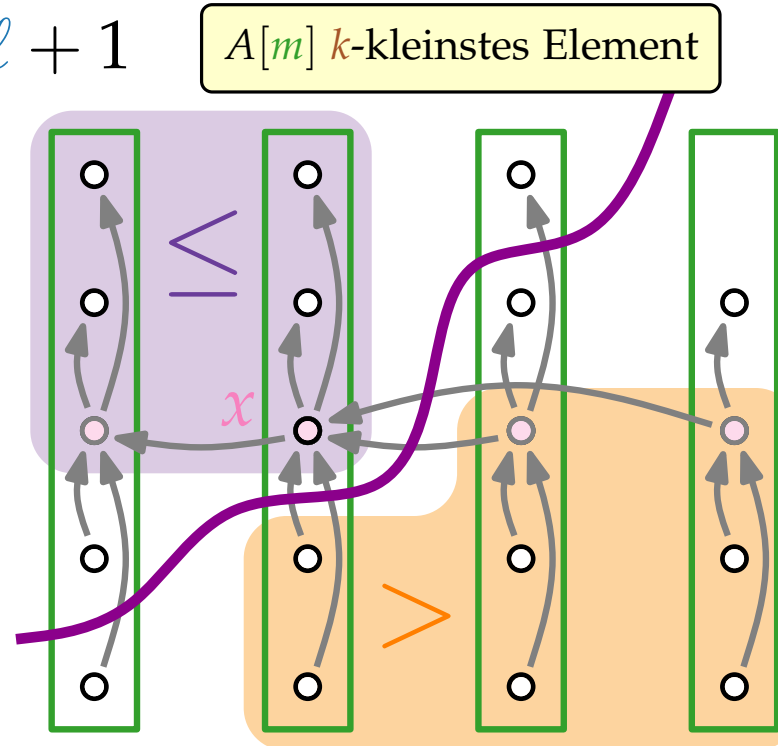
if $i < k$ then

return SELECT($A, \ell, m - 1, i$)

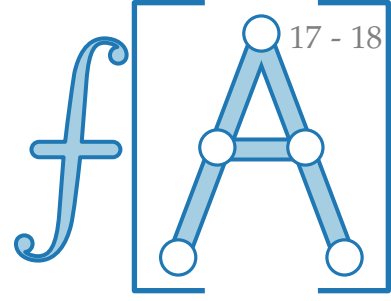
else

return SELECT($A, m + 1, r, i - k$)

x



SELECT: deterministisch



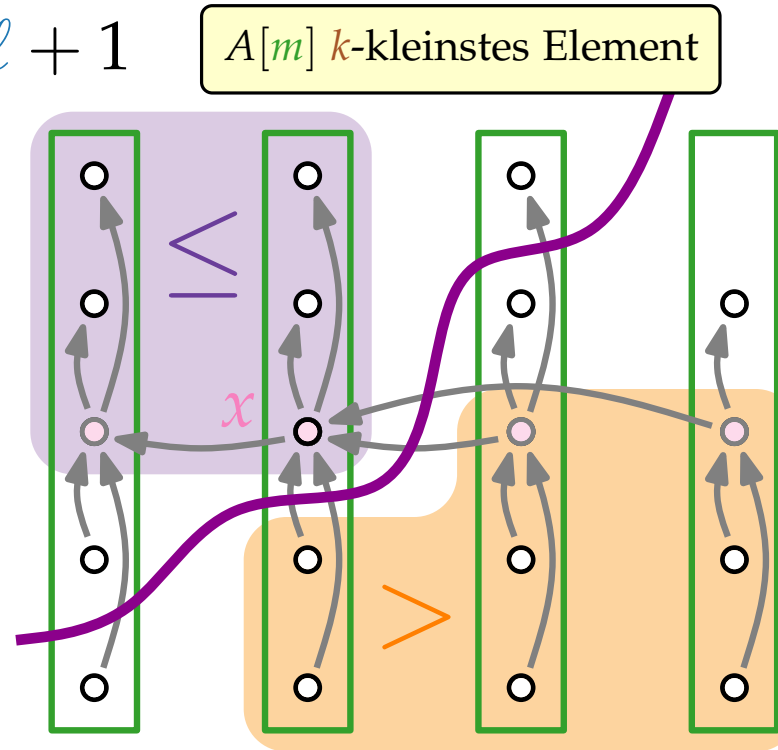
SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lfloor n/5 \rfloor$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$

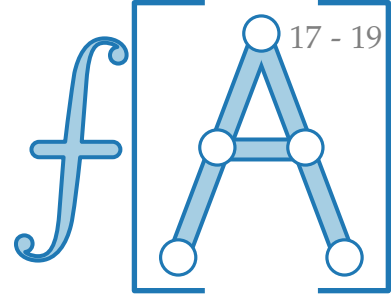
5.

```
if  $i == k$  then return  $A[m]$ 
else
  if  $i < k$  then
    return SELECT( $A, \ell, m - 1, i$ )
  else
    return SELECT( $A, m + 1, r, i - k$ )
```

Anzahl() $\geq$



SELECT: deterministisch



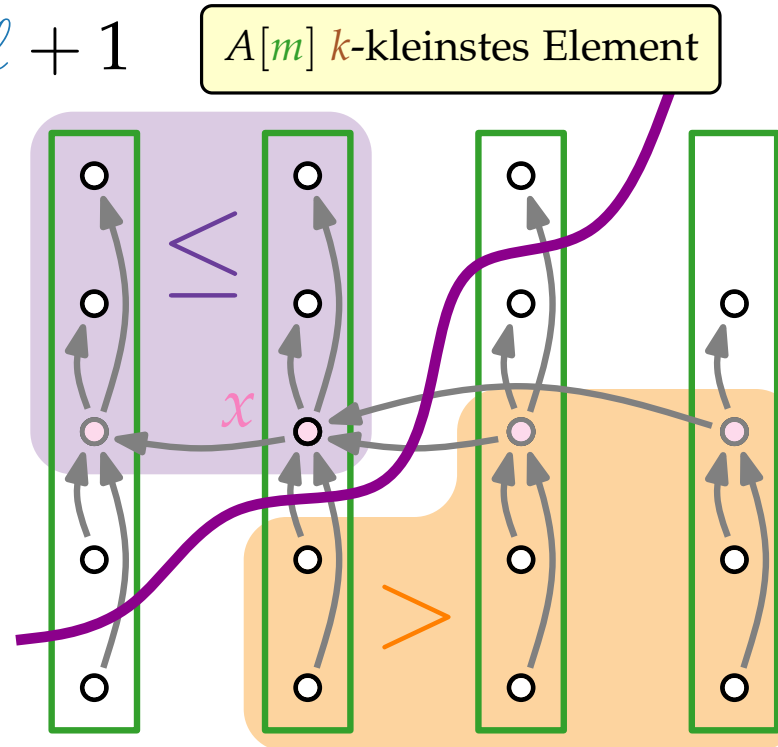
SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lceil n/5 \rceil$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$

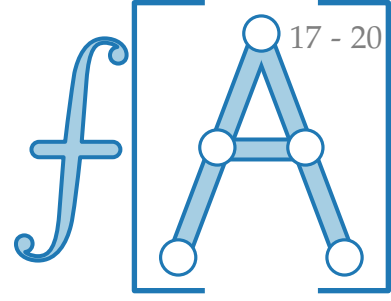
5.

```
if  $i == k$  then return  $A[m]$ 
else
  if  $i < k$  then
    return SELECT( $A, \ell, m - 1, i$ )
  else
    return SELECT( $A, m + 1, r, i - k$ )
```

Anzahl() $\geq 3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right)$



SELECT: deterministisch



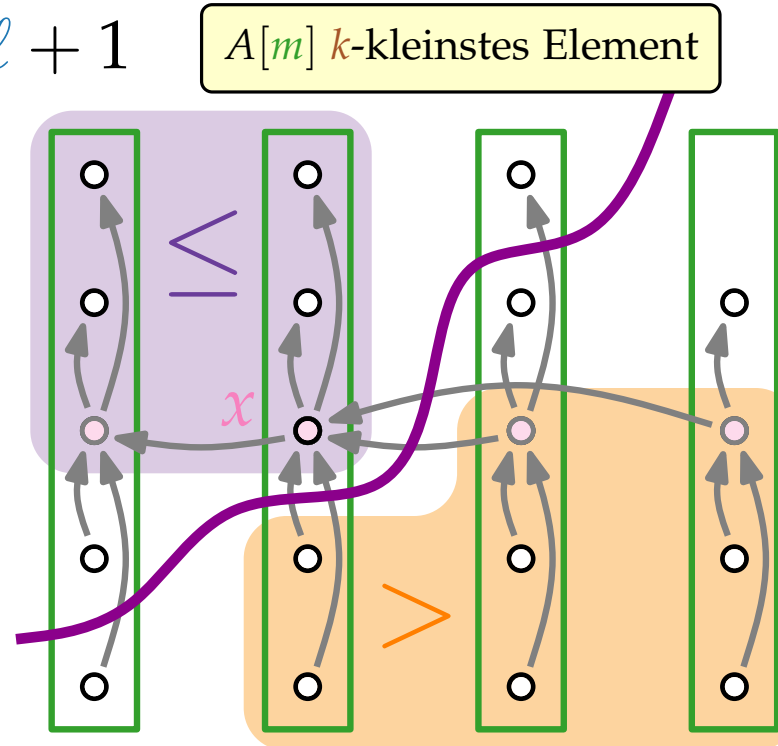
SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lceil n/5 \rceil$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$

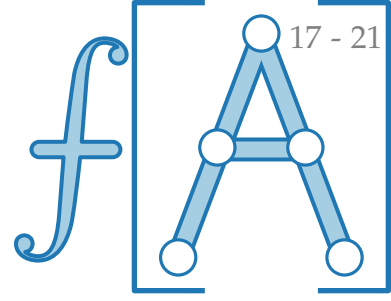
5.

```
if  $i == k$  then return  $A[m]$ 
else
  if  $i < k$  then
    return SELECT( $A, \ell, m - 1, i$ )
  else
    return SELECT( $A, m + 1, r, i - k$ )
```

$$\text{Anzahl}(\bullet) \geq 3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$$



SELECT: deterministisch



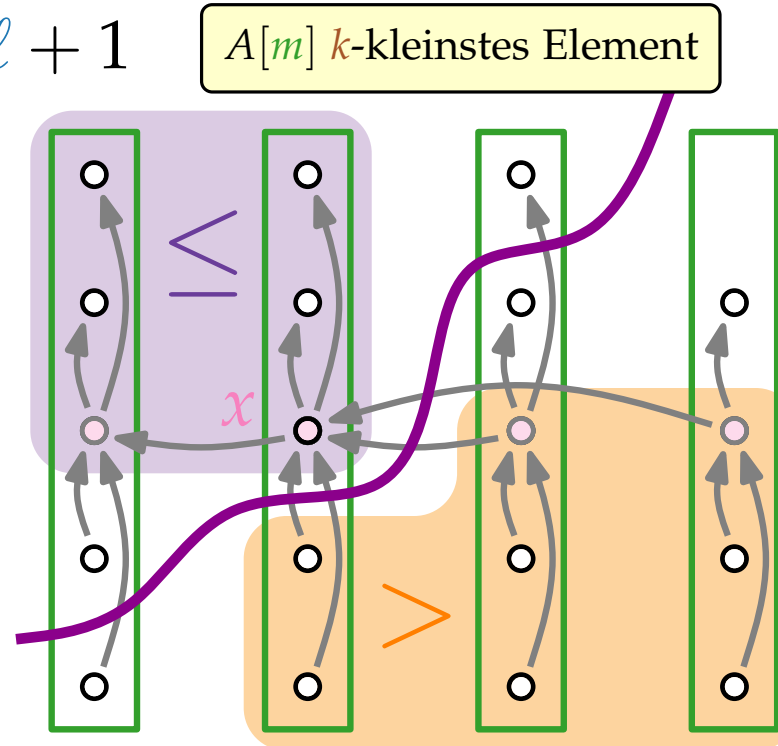
SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lceil n/5 \rceil$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$

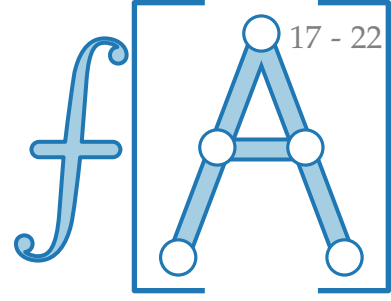
5.

```
if  $i == k$  then return  $A[m]$ 
else
  if  $i < k$  then
    return SELECT( $A, \ell, m - 1, i$ )
  else
    return SELECT( $A, m + 1, r, i - k$ )
```

Anzahl($\bullet$) $\geq 3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$



SELECT: deterministisch



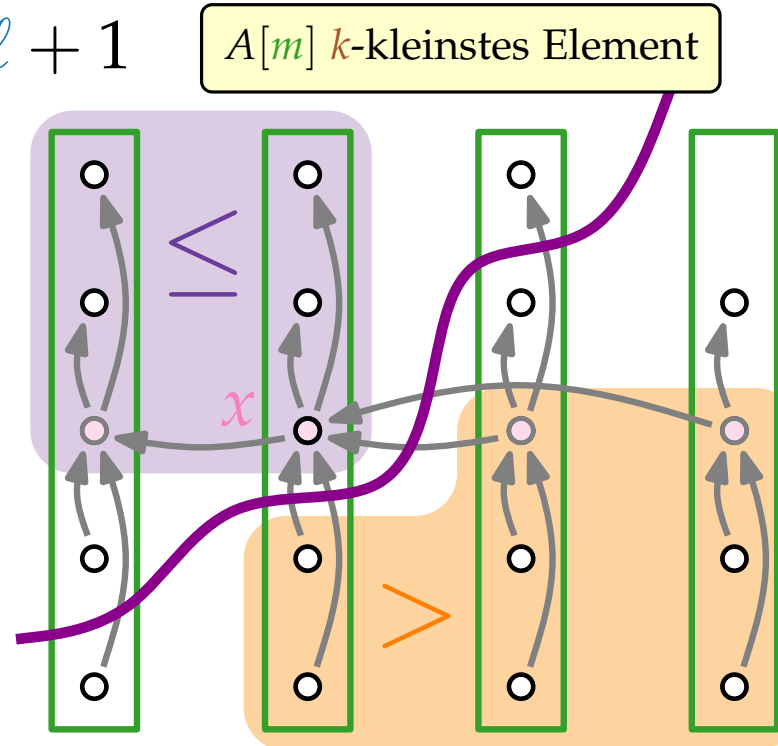
SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lceil n/5 \rceil$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$

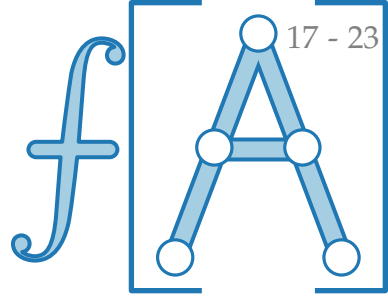
5.

```
if  $i == k$  then return  $A[m]$ 
else
  if  $i < k$  then
    return SELECT( $A, \ell, m - 1, i$ )
  else
    return SELECT( $A, m + 1, r, i - k$ )
```

$$\text{Anzahl}(\bullet) \geq 3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$$



SELECT: deterministisch

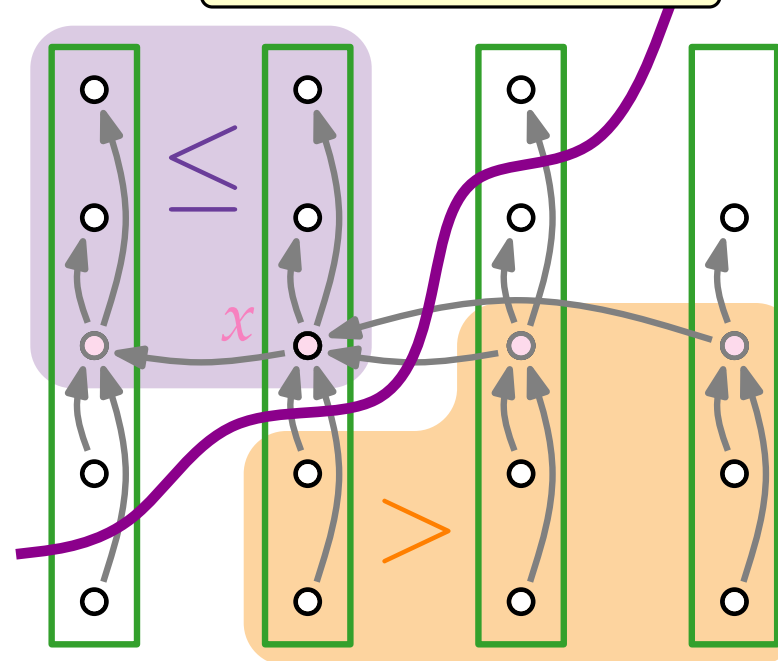


SELECT(A, ℓ, r, i)

1. Teile die n Elem. der Eingabe in $\lfloor n/5 \rfloor$ 5er-Gruppen und eine Gruppe mit den restlichen ($n \bmod 5$) Elem.
2. Sortiere jede der $\lceil n/5 \rceil$ Gruppen und bestimme ihren Median.
3. Bestimme **rekursiv** den Median x der Gruppen-Mediane.
4. $m = \text{PARTITION}'(A, \ell, r, x); k = m - \ell + 1$ $A[m]$ k -kleinstes Element

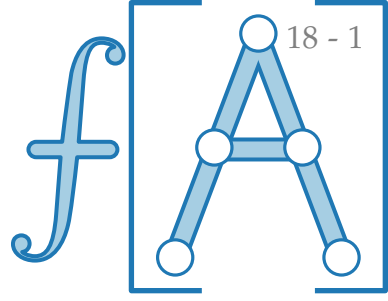
5. **if** $i == k$ **then return** $A[m]$
else x
 if $i < k$ **then**
 return SELECT($A, \ell, m - 1, i$)
 else $\leq 7n/10 + 6$ Elemente
 return SELECT($A, m + 1, r, i - k$)

$$\text{Anzahl}(\bullet) \geq 3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$$



Laufzeitanalyse

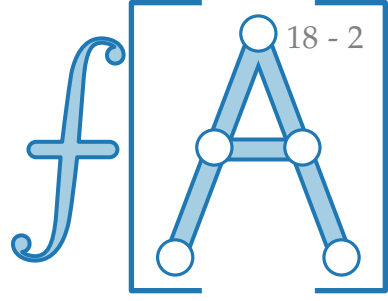
Beobachtung. Es genügt wieder, Vergleiche zu zählen!



Laufzeitanalyse

Beobachtung. Es genügt wieder, Vergleiche zu zählen!

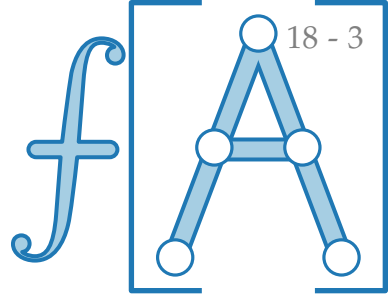
PARTITION': $\approx 1n$



Laufzeitanalyse

Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.



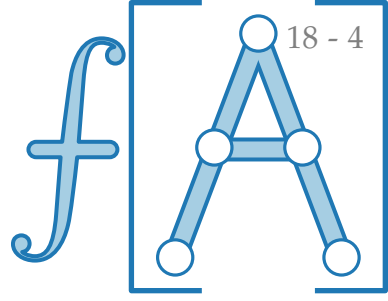
Laufzeitanalyse

Beobachtung. Es genügt wieder, Vergleiche zu zählen!

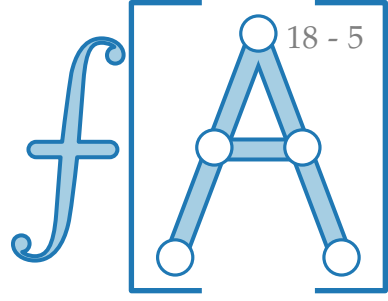
PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq$$



Laufzeitanalyse



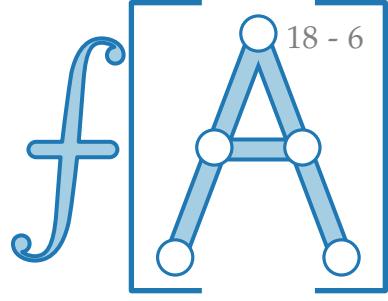
Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \left\{ \begin{array}{l} \text{falls } n \geq n_0, \end{array} \right.$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

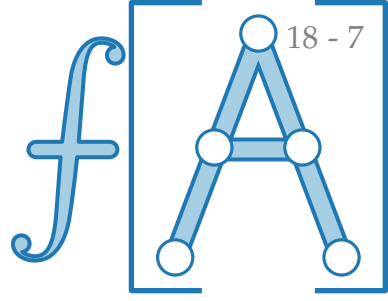
Ansatz.

Schritt 3

$$V(n) \leq \left\{ \overbrace{V(\lceil n/5 \rceil)}^{\text{Schritt 3}} \right.$$

falls $n \geq n_0$,

Laufzeitanalyse



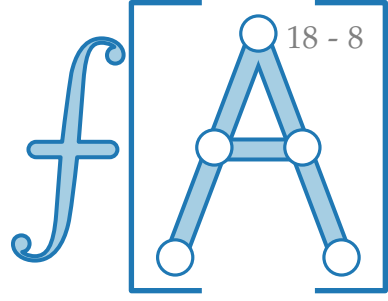
Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} \\ \underbrace{\hspace{10em}}_{\text{Schritt 5}} \end{cases} \quad \text{falls } n \geq n_0,$$

Laufzeitanalyse



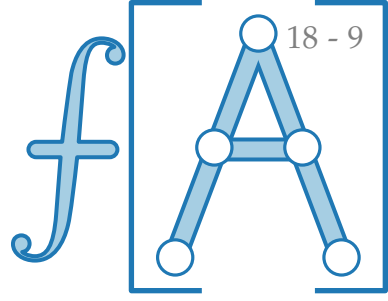
Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + \underbrace{3n}_{\text{Schritt 5}} & \text{falls } n \geq n_0, \end{cases}$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

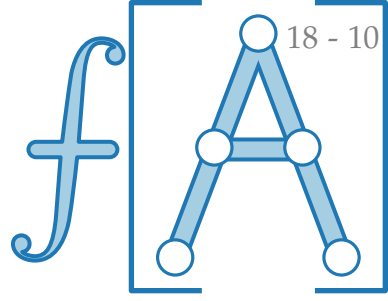
PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6) + 3n}^{\text{Schritt 3}} & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

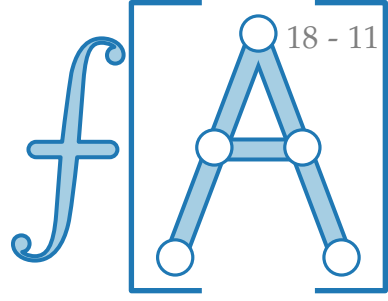
Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6) + 3n}^{\text{Schritt 3}} & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

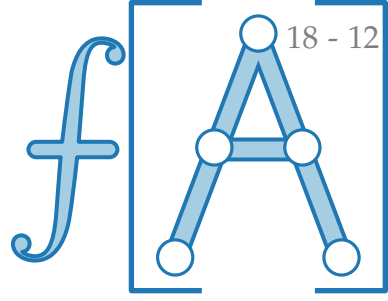
$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\Rightarrow V(n) \leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

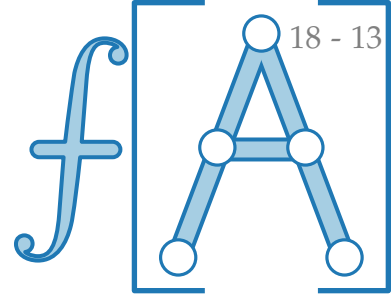
$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\Rightarrow V(n) \leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

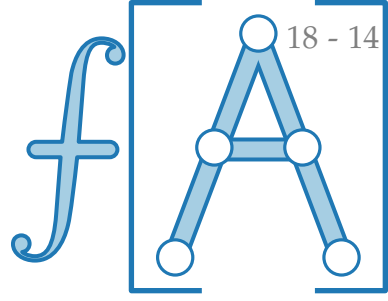
$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n \end{aligned}$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

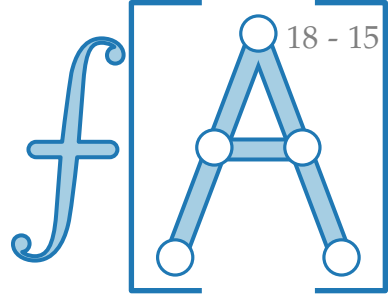
$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

$\underbrace{\hspace{10em}}_{\text{Schritt 5}}$

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n \end{aligned}$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

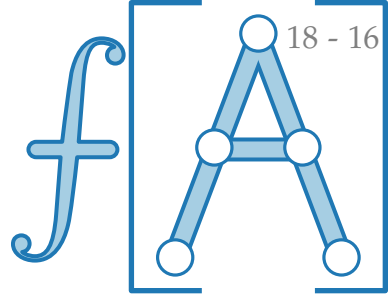
$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) \end{aligned}$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

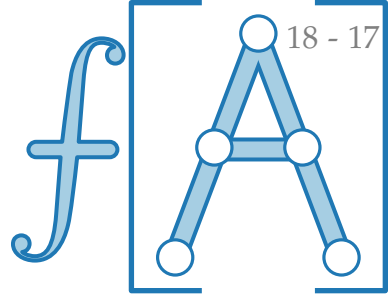
$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) \end{aligned}$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

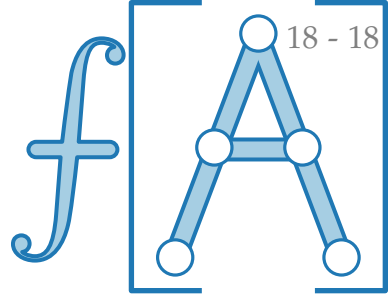
$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n \end{aligned}$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

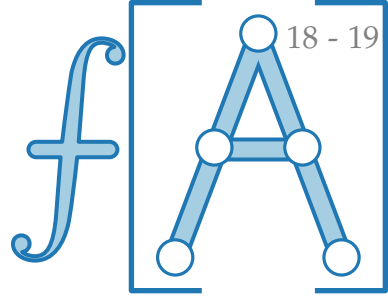
Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\Rightarrow V(n) \leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n$$

$$= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n)$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

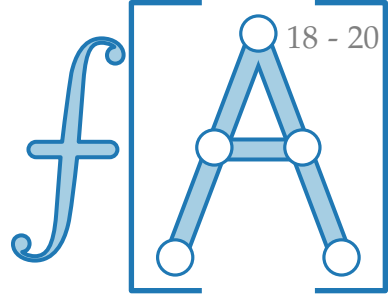
$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n) \end{aligned}$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

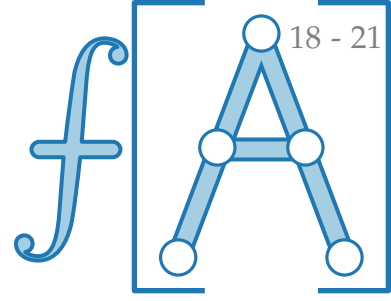
Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n) \end{aligned}$$

$\geq 0?! \quad \text{!}$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

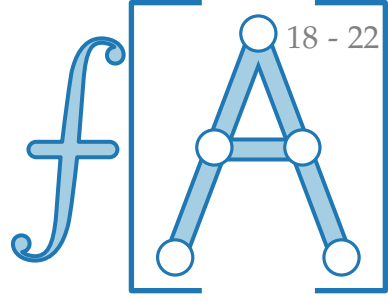
Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n) \end{aligned}$$

$\geq 0?! \quad \text{!}$

$$\text{falls } c \geq \frac{3n}{n/10 - 7} =$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

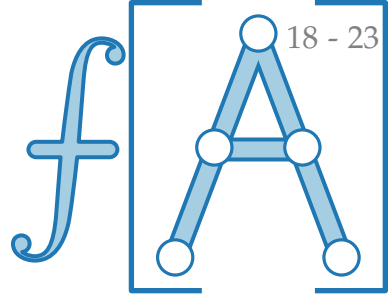
Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n) \end{aligned}$$

$\geq 0?! \quad \text{}$

$$\text{falls } c \geq \frac{3n}{n/10 - 7} = \frac{30}{1 - 70/n} \xrightarrow{n \rightarrow \infty}$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

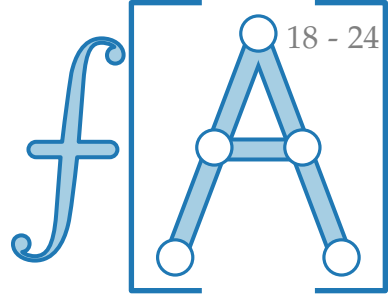
Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n) \end{aligned}$$

$\geq 0?! \quad \text{}$

$$\text{falls } c \geq \frac{3n}{n/10 - 7} = \frac{30}{1 - 70/n} \xrightarrow{n \rightarrow \infty} 30^+$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

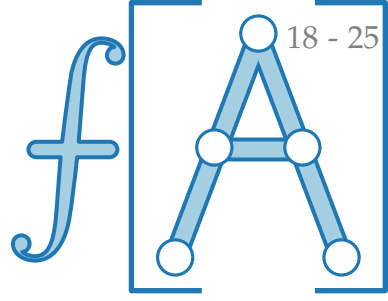
Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\begin{aligned} \Rightarrow V(n) &\leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n \\ &= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n) \\ \text{falls } c &\geq \frac{3n}{n/10 - 7} = \frac{30}{1 - 70/n} \xrightarrow{n \rightarrow \infty} 30^+ \end{aligned}$$

$\geq 0?!$

bzw. $n \geq \frac{70c}{c-30}$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

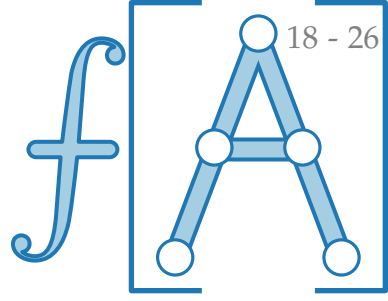
$$\Rightarrow V(n) \leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n$$

$$= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n)$$

$$\text{falls } c \geq \frac{3n}{n/10 - 7} = \frac{30}{1 - 70/n} \xrightarrow{n \rightarrow \infty} 30^+ \quad \boxed{\text{bzw. } n \geq \frac{70c}{c-30}}$$

$$\Rightarrow \text{für jedes } \varepsilon > 0 \quad \text{gilt: } V(n) \leq \underbrace{(30 + \varepsilon)}_c \cdot n$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\Rightarrow V(n) \leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n$$

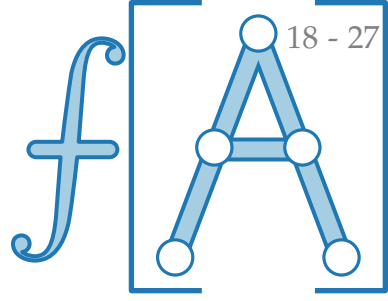
$$= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n)$$

$$\text{falls } c \geq \frac{3n}{n/10 - 7} = \frac{30}{1 - 70/n} \xrightarrow{n \rightarrow \infty} 30^+$$

$$\text{bzw. } n \geq \frac{70c}{c - 30}$$

$$\Rightarrow \text{für jedes } \varepsilon > 0 \text{ und } n \geq \frac{2100}{\varepsilon} + 70 \text{ gilt: } V(n) \leq \underbrace{(30 + \varepsilon)}_c \cdot n$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + 3n & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Schritt 5

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

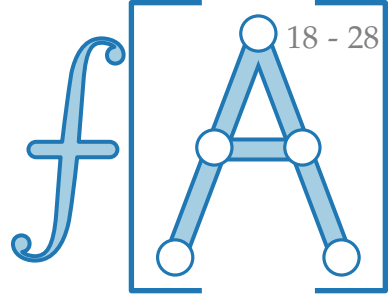
$$\Rightarrow V(n) \leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n$$

$$= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n)$$

$$\text{falls } c \geq \frac{3n}{n/10 - 7} = \frac{30}{1 - 70/n} \xrightarrow{n \rightarrow \infty} 30^+ \quad \boxed{\text{bzw. } n \geq \frac{70c}{c-30}}$$

$$\Rightarrow \text{für jedes } \varepsilon > 0 \text{ und } n \geq \frac{2100}{\varepsilon} + 70 \text{ gilt: } V(n) \leq (30 + \varepsilon) \cdot n$$

Laufzeitanalyse



Beobachtung. Es genügt wieder, Vergleiche zu zählen!

PARTITION': $\approx 1n$, Sortieren: $\approx \frac{n}{5} \cdot V_{IS}(5) = 2n$ Vgl.

Ansatz.

$$V(n) \leq \begin{cases} \overbrace{V(\lceil n/5 \rceil) + V(7n/10 + 6)}^{\text{Schritt 3}} + \underbrace{3n}_{\text{Schritt 5}} & \text{falls } n \geq n_0, \\ \mathcal{O}(1) & \text{sonst.} \end{cases}$$

Behauptung. Es gibt $c, n_0 > 0$, so dass für alle $n \geq n_0$ gilt: $V(n) \leq cn$.

$$\Rightarrow V(n) \leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 6) + 3n$$

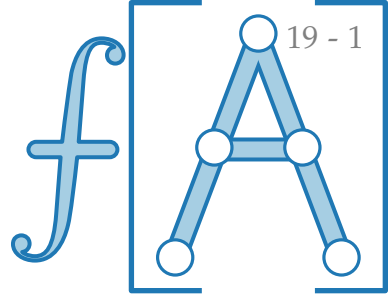
$$= c \cdot (9n/10 + 7) + 3n = cn - (c \cdot (n/10 - 7) - 3n)$$

$$\text{falls } c \geq \frac{3n}{n/10 - 7} = \frac{30}{1 - 70/n} \xrightarrow{n \rightarrow \infty} 30^+ \quad \boxed{\text{bzw. } n \geq \frac{70c}{c-30}}$$

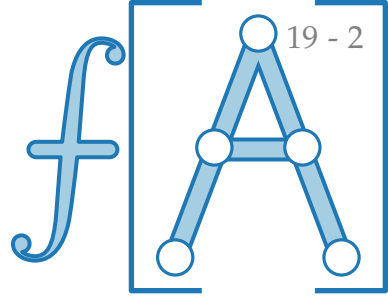
$$\Rightarrow \text{für jedes } \varepsilon > 0 \text{ und } n \geq \frac{2100}{\varepsilon} + 70 \text{ gilt: } V(n) \leq (30 + \varepsilon) \cdot n$$

Ergebnis und Diskussion

Satz. Das Auswahlproblem kann auch im schlechtesten Fall in linearer Zeit gelöst werden.



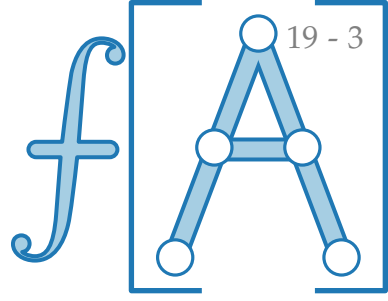
Ergebnis und Diskussion



Satz. Das Auswahlproblem kann auch im schlechtesten Fall in linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq 2100/\varepsilon + 70$ Zahlen die i -kleinste Zahl mit höchstens $(30 + \varepsilon)n$ Vergleichen finden kann.

Ergebnis und Diskussion

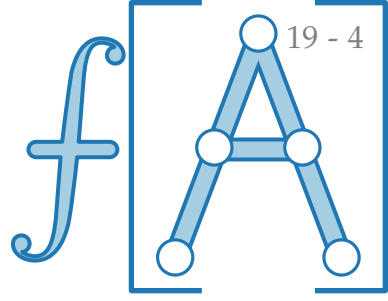


Satz. Das Auswahlproblem kann auch im schlechtesten Fall in linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq 2100/\varepsilon + 70$ Zahlen die i -kleinste Zahl mit höchstens $(30 + \varepsilon)n$ Vergleichen finden kann.

- Der Algorithmus LAZYSELECT [Floyd & Rivest, 1975] löst das Auswahlproblem mit Wahrscheinlichkeit $1 - \mathcal{O}(1/\sqrt[4]{n})$ mit $\frac{3}{2}n + o(n)$ Vergleichen

Ergebnis und Diskussion

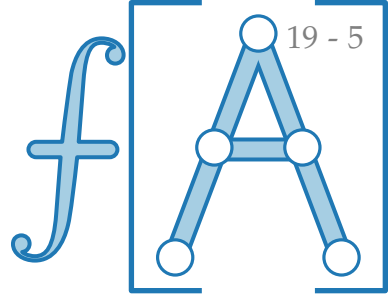


Satz. Das Auswahlproblem kann auch im schlechtesten Fall in linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq 2100/\varepsilon + 70$ Zahlen die i -kleinste Zahl mit höchstens $(30 + \varepsilon)n$ Vergleichen finden kann.

- Der Algorithmus LAZYSELECT [Floyd & Rivest, 1975] löst das Auswahlproblem mit Wahrscheinlichkeit $1 - \mathcal{O}(1/\sqrt[4]{n})$ mit $\frac{3}{2}n + o(n)$ Vergleichen
- Der besten bekannte deterministische Auswahl-Algorithmus [Dor & Zwick, 1999] (*sehr kompliziert!*) benötigt im schlechtesten Fall höchstens $2,942n$ Vergleiche.

Ergebnis und Diskussion

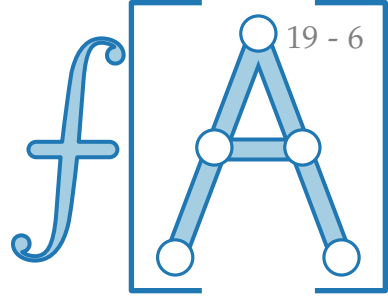


Satz. Das Auswahlproblem kann auch im schlechtesten Fall in linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq 2100/\varepsilon + 70$ Zahlen die i -kleinste Zahl mit höchstens $(30 + \varepsilon)n$ Vergleichen finden kann.

- Der Algorithmus LAZYSELECT [Floyd & Rivest, 1975] löst das Auswahlproblem mit Wahrscheinlichkeit $1 - \mathcal{O}(1/\sqrt[4]{n})$ mit $\frac{3}{2}n + o(n)$ Vergleichen
- Der besten bekannte deterministische Auswahl-Algorithmus [Dor & Zwick, 1999] (*sehr kompliziert!*) benötigt im schlechtesten Fall höchstens $2,942n$ Vergleiche.
- **Jeder** deterministische Auswahl-Alg. benötigt im schlechtesten Fall mindestens $(2 + 2^{-80})n$ Vergleiche. [Dor & Zwick, 2001]

Ergebnis und Diskussion



Satz. Das Auswahlproblem kann auch im schlechtesten Fall in linearer Zeit gelöst werden.

Genauer. Für jedes $\varepsilon > 0$ gilt, dass man in einer Folge von $n \geq 2100/\varepsilon + 70$ Zahlen die i -kleinste Zahl mit höchstens $(30 + \varepsilon)n$ Vergleichen finden kann.

- Der Algorithmus LAZYSELECT [Floyd & Rivest, 1975] löst das Auswahlproblem mit Wahrscheinlichkeit $1 - \mathcal{O}(1/\sqrt[4]{n})$ mit $\frac{3}{2}n + o(n)$ Vergleichen
- Der besten bekannte deterministische Auswahl-Algorithmus [Dor & Zwick, 1999] (*sehr kompliziert!*) benötigt im schlechtesten Fall höchstens $2,942n$ Vergleiche.
- **Jeder** deterministische Auswahl-Alg. benötigt im schlechtesten Fall mindestens $(2 + 2^{-80})n$ Vergleiche. [Dor & Zwick, 2001]

Satz. Durch Suche des Medians in $\mathcal{O}(n)$ Zeit kann QUICKSORT in Worst-Case $\mathcal{O}(n \log n)$ Zeit sortieren.